MICROSOFT CERTIFIED: SQL AI DEVELOPER ASSOCIATE

Exam DP-800 Study Guide

Developing AI-Enabled Database Solutions

EXAM **DP-800**	SKILLS OUTLINE VERSION **19 October 2026**
TIME TO COMPLETE **120 minutes, proctored**	PASSING SCORE **700 of 1000**
LEVEL **Intermediate · Associate**	RENEWAL **Free online, every 12 months**

T-SQL · JSON & regex · Indexes & partitioning · Always Encrypted · RLS & masking · Query Store · SQL database projects · GitHub Actions · Data API builder · GitHub Copilot & MCP · Embeddings · Vector & hybrid search · RAG

Every Microsoft link in this guide carries the Microsoft Learn Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Language-locale segments have been removed and the ID is appended with `?` or `&` exactly as the Registered Member Handbook requires, so you can copy any link straight out of this document and share it.

Built from the official skills outline published on the Microsoft Learn DP-800 study guide page.

Table of contents

Chapter 0 — How to use this guide

0.1 What DP-800 actually certifies

0.2 The three skill areas and their weights

0.3 Exam logistics you should know before booking

0.4 A six-week study plan

0.5 How to read the callouts

Chapter 1 — The SQL platform family and the words you need

1.1 One engine, four platforms

1.2 Database words you need

1.3 AI words you need

1.4 Chapter 1 self-check

1.5 Learn more

Chapter 2 — Tables, data types, and indexes

2.1 Designing a table: data types and sizes

2.2 How indexes work, in one picture

2.3 Rowstore indexes: covering, key order, and filtered

2.4 Columnstore indexes

2.5 Chapter 2 self-check

2.6 Learn more

Chapter 3 — Specialized tables: in-memory, temporal, ledger, graph, and external

3.1 Memory-optimized (In-Memory OLTP) tables

3.2 Temporal (system-versioned) tables

3.3 Ledger tables

3.4 Graph tables

3.5 External tables (data virtualization)

3.6 Chapter 3 self-check

3.7 Learn more

Chapter 4 — JSON columns, constraints, sequences, and partitioning

4.1 Storing JSON

4.2 Indexing JSON

4.3 Constraints

4.4 SEQUENCE vs IDENTITY

4.5 Partitioning tables and indexes

4.6 Managing partitions

4.7 Chapter 4 self-check

4.8 Learn more

Chapter 5 — Programmability objects: views, functions, procedures, and triggers

5.1 Views and indexed views

5.2 Scalar functions

5.3 Table-valued functions (TVFs)
5.4 Stored procedures
5.5 Triggers
5.6 Chapter 5 self-check
5.7 Learn more

Chapter 6 — Advanced T-SQL, part 1: CTEs, window functions, subqueries, and errors

6.1 Common table expressions (CTEs)
6.2 Window functions
6.3 Correlated subqueries and EXISTS
6.4 Error handling with TRY…CATCH
6.5 Writing SARGable predicates
6.6 Chapter 6 self-check
6.7 Learn more

Chapter 7 — Advanced T-SQL, part 2: JSON, regular expressions, fuzzy matching, and graphs

7.1 JSON functions
7.2 Regular expressions
7.3 Fuzzy string matching
7.4 Graph queries with MATCH
7.5 Chapter 7 self-check
7.6 Learn more

Chapter 8 — AI-assisted development: GitHub Copilot, Copilot in Fabric, and MCP

8.1 The tools
8.2 The security impact of AI-assisted tools
8.3 Enabling GitHub Copilot and Copilot in Fabric
8.4 Choosing models and MCP tools in a chat session
8.5 GitHub Copilot instruction files
8.6 Connecting to MCP servers
8.7 Chapter 8 self-check
8.8 Learn more

Chapter 9 — Data security: encryption, masking, row-level security, and permissions

9.1 Encryption: TDE, Always Encrypted, and column-level encryption
9.2 Dynamic data masking (DDM)
9.3 Row-level security (RLS)
9.4 Object-level permissions
9.5 Chapter 9 self-check
9.6 Learn more

Chapter 10 — Secure access, auditing, and securing model and API endpoints

10.1 Passwordless access with Microsoft Entra ID
10.2 Network access
10.3 Auditing and ledger
10.4 Securing model endpoints
10.5 Securing GraphQL, REST, and MCP endpoints
10.6 Chapter 10 self-check

10.7 Learn more

Chapter 11 — Optimize database performance

11.1 Recommending database configurations
11.2 Transactions, isolation levels, and concurrency
11.3 Reading execution plans
11.4 Dynamic management views (DMVs)
11.5 Query Store and Query Performance Insight
11.6 Blocking and deadlocks
11.7 Chapter 11 self-check
11.8 Learn more

Chapter 12 — CI/CD with SQL Database Projects

12.1 SQL database projects in one page
12.2 Build validation, references, and system objects
12.3 Source control and reference data
12.4 Branching, pull requests, and conflicts
12.5 Pipelines with GitHub Actions, and secrets
12.6 Detecting schema drift
12.7 Deploying changes
12.8 Pipeline controls: branch policies, approvals, code owners
12.9 Testing strategy
12.10 Chapter 12 self-check
12.11 Learn more

Chapter 13 — Integrate SQL solutions with Azure services

13.1 Data API builder (DAB): what and why
13.2 The configuration file
13.3 Entities, permissions, caching, pagination, and filtering
13.4 REST and GraphQL endpoints, stored procedures, views, and relationships
13.5 Deploying DAB
13.6 Recommending Azure Monitor configurations
13.7 Handling data changes
13.8 Chapter 13 self-check
13.9 Learn more

Chapter 14 — Models and embeddings

14.1 Evaluating models
14.2 Creating and managing external models
14.3 Which columns to embed
14.4 Chunking
14.5 Generating embeddings
14.6 Keeping embeddings fresh: maintenance methods
14.7 Chapter 14 self-check
14.8 Learn more

Chapter 15 — Intelligent search: full-text, vector, and hybrid

15.1 Choosing full-text, vector, or hybrid search

15.2 Full-text search
15.3 Designing for vector data
15.4 The vector functions
15.5 kNN vs ANN, vector indexes, and metrics
15.6 Writing vector search queries
15.7 Hybrid search and reciprocal rank fusion (RRF)
15.8 Evaluating vector and hybrid search
15.9 Azure AI Search: hybrid queries with semantic ranking
15.10 Chapter 15 self-check
15.11 Learn more

Chapter 16 — Retrieval-augmented generation (RAG) in T-SQL

16.1 What RAG is, and when to use it
16.2 Calling a model: `sp_invoke_external_rest_endpoint`
16.3 Converting rows to JSON for the model
16.4 Building the prompt and sending it
16.5 Extracting the response
16.6 The end-to-end stored procedure
16.7 Chapter 16 self-check
16.8 Learn more

Chapter 17 — Case-study walkthroughs

17.1 How to attack a case study
17.2 Case A — a fleet-telematics platform (Azure SQL Database)
17.3 Case B — a healthcare provider (Azure SQL Database + GitHub)

Chapter 18 — Practice questions

Chapter 19 — Labs, exam day, and quick reference

19.1 The hands-on lab track
19.2 The week before
19.3 The 60-second memory dump
19.4 Reading exam questions
19.5 Glossary
19.6 Where to go next

Appendix A — One-page decision tables

A.1 Which index or table feature?
A.2 Which programmability object?
A.3 Which T-SQL function?
A.4 Which security control?
A.5 Which performance tool or fix?
A.6 Which CI/CD action?
A.7 Which change-handling or embedding-maintenance method?
A.8 Which AI building block?

Appendix B — Where popular answer keys go wrong

Appendix C — Exam-dump coverage map

Appendix D — Syntax cheat sheet

Appendix E — Feature availability by platform

Chapter 0 — How to use this guide

0.1 What DP-800 actually certifies

Passing **Exam DP-800: Developing AI-Enabled Database Solutions** earns the Microsoft Certified: SQL AI Developer Associate credential. Microsoft describes the candidate like this:

> As a candidate for this Microsoft Certification, you should have subject matter expertise in designing and developing AI-enabled database solutions across Microsoft SQL platforms, including Microsoft SQL Server, Azure SQL, and SQL databases in Microsoft Fabric.

The official DP-800 study guide lists four responsibilities:

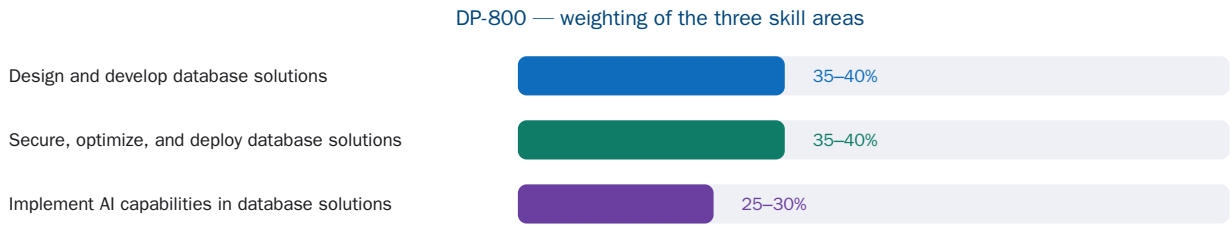
- Designing and developing database solutions that include both structured and semi-structured data.
- Integrating AI features into modern and highly scalable enterprise applications.
- Securing, optimizing, and deploying database solutions.
- Implementing AI capabilities in database solutions.

You are also expected to write **T-SQL**, use **CI/CD in GitHub**, use **AI-assisted development tools** (GitHub Copilot, MCP), and understand **AI concepts** such as embeddings, vectors, and models.

> **PLAIN ENGLISH**
>
> **T-SQL** (Transact-SQL) is the SQL language used by Microsoft SQL Server and Azure SQL. **Structured data** fits neatly into rows and columns (an Orders table). **Semi-structured data** has its own shape inside a value, like a JSON document stored in one column. **CI/CD** (continuous integration / continuous deployment) means every change is automatically built, tested, and deployed by a pipeline instead of by hand.

> **KEY TAKEAWAY**
>
> DP-800 is a **database developer** exam with an AI layer on top. About three quarters of it is classic SQL work done well (tables, indexes, T-SQL, security, performance, deployment). The last quarter is about putting AI *inside* the database: embeddings, vector search, and retrieval-augmented generation (RAG).

0.2 The three skill areas and their weights



Skills measured as of 19 October 2026. The first two areas carry the same weight; the AI area is smaller but very scenario-heavy.

Skill area	Weight	Chapters in this guide
Design and develop database solutions	35–40%	2, 3, 4, 5, 6, 7, 8
Secure, optimize, and deploy database solutions	35–40%	9, 10, 11, 12, 13
Implement AI capabilities in database solutions	25–30%	14, 15, 16

The change log on the official study guide shows **minor** updates in the 19 October 2026 version to *Design and implement database objects*, *Design and implement SQL solutions by using AI-assisted tools*, *Design and implement models and embeddings*, and *Design and implement intelligent search*. Microsoft also notes that most questions cover generally available (GA) features, but the exam *"may contain questions on Preview features if those features are commonly used."* Many of the AI features in this guide are new, so you will see the words **preview** and **GA** often.

> **PLAIN ENGLISH**
>
> **GA** (general availability) means a feature is finished and fully supported. **Preview** means you can try it, but it may still change. On SQL Server 2025 many preview features need a switch first: `ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;`.

0.3 Exam logistics you should know before booking

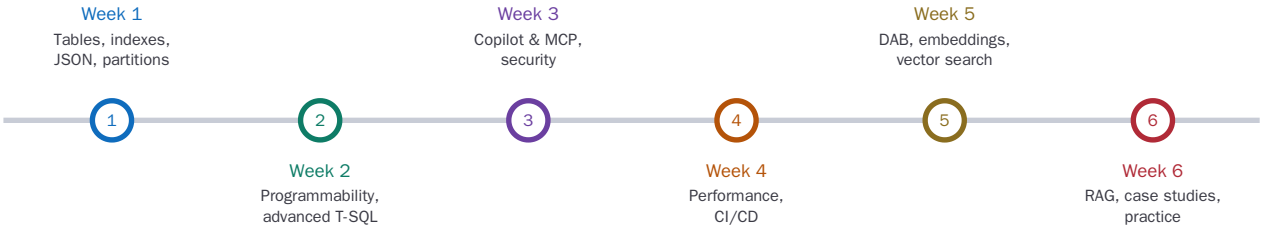
Item	Detail
Duration	**120 minutes** to complete the assessment; the appointment is longer — see exam duration and experience
Passing score	**700** on a scale of 1000 — a scaled score, not a percentage. See exam scoring and score reports
Question count	Not published. Expect roughly 40–60 items, usually with at least one case study
Question types	Multiple choice, multi-select, drag-and-drop, hot area (drop-down boxes inside code), Yes/No series, case studies, and possible interactive components
Languages	English, Japanese, Chinese (Simplified), Chinese (Traditional), Korean, German, French, Italian, Portuguese (Brazil), Spanish
Delivery	Proctored, online or at a Pearson VUE test centre
Retake	24 hours after a first failure, longer for later attempts — retake policy
Renewal	Free online assessment every 12 months — renew your certification
Accommodations	Request accommodations for extra time or assistive technology

> **WATCH OUT**
> When this guide was written (October 2026), the DP-800 certification page offered the **exam sandbox** (aka.ms/examdemo) but no DP-800 practice assessment of its own. Check the certification page again before you book. Until one appears, use the self-checks in each chapter and the practice questions in Chapter 18. (The "Take a free Practice Assessment" link on the DP-800 study guide page currently opens the Azure Administrator assessment, not a DP-800 one.)

How the question types feel on this exam. DP-800 loves code. Many items show a T-SQL script, a GitHub Actions YAML file, or a `dab-config.json` file with one or more blanks, and you choose the missing piece from a drop-down. Others show code and ask Yes/No questions about what it does. Read code slowly: one wrong argument order (for example `DATEDIFF(year, GETDATE(), @OrderDate)` returns a *negative* number) is often the whole question.

Yes/No series. Some questions are part of a *series*: the same scenario repeats with a different proposed solution each time, and you cannot go back. Judge each solution on its own. More than one solution in a series can be correct.

0.4 A six-week study plan



Roughly 6–8 hours per week. Do the exercise unit at the end of every Microsoft Learn module — DP-800 questions are written from hands-on experience.

Week	Focus	This guide	Microsoft Learn
1	Platforms, tables, indexes, specialized tables, JSON, constraints, partitions	Ch. 1–4	Design and implement database objects with SQL
2	Views, functions, procedures, triggers, advanced T-SQL	Ch. 5–7	Implement programmability objects · Write advanced T-SQL code
3	GitHub Copilot, MCP, data security, passwordless access	Ch. 8–10	Implement SQL solutions by using AI-assisted tools · Implement data security and compliance
4	Performance tuning, CI/CD with SQL database projects	Ch. 11–12	Optimize database performance · Implement CI/CD by using SQL database projects
5	Data API builder, change handling, models, embeddings, search	Ch. 13–15	Integrate SQL solutions with Azure services · Models and embeddings · Intelligent search
6	RAG, case studies, practice, weak spots	Ch. 16–19	Design and implement RAG with SQL

The three official learning paths line up exactly with the three skill areas: Design and develop database solutions, Secure, optimize, and deploy database solutions, and Implement AI capabilities in database solutions.

> **TIP**
> Many exam questions follow the official training units almost word for word — sometimes a one-line description from a unit *is* the answer. When this guide quotes a training unit, treat that sentence as high value.

0.5 How to read the callouts

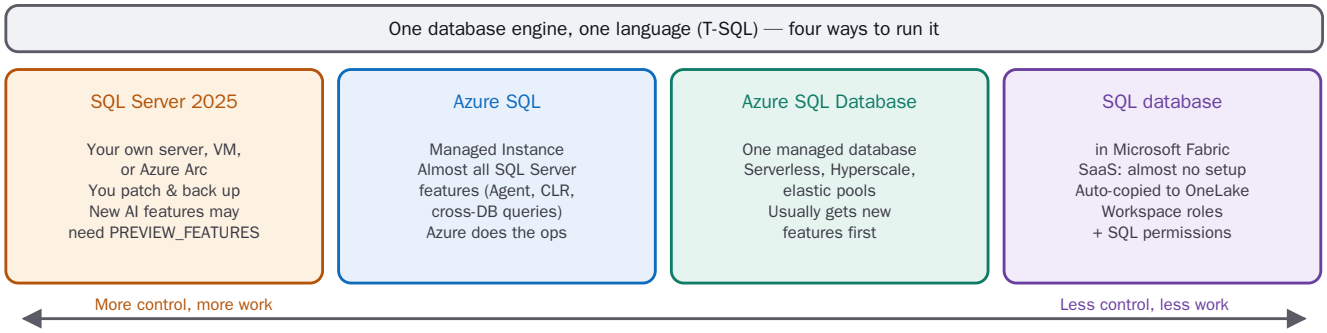
Callout	What it means
Exam focus	Wording from the official skills outline
Plain English	A technical word or idea explained simply
Example	A small, concrete example you can picture or run
Scenario pattern	A requirement you will meet on the exam, and the answer that fits it
Common exam trap	A distinction the exam deliberately tests; most wrong answers come from these
Know the difference	Two features that look alike under time pressure
Key takeaway	The one sentence to remember from the section
Hands-on lab	Do it yourself; reading alone will not get you to 700
Tip / **Watch out**	Practical advice and known gotchas

> **HANDS-ON LAB**
>
> Set up a free practice environment. Any one of these is enough: the Azure SQL Database free offer, a Microsoft Fabric trial with a SQL database in Fabric, or SQL Server 2025 Developer edition on your own PC. Install Visual Studio Code with the **MSSQL** extension and **GitHub Copilot**. You will use them in almost every chapter.

Chapter 1 — The SQL platform family and the words you need

Before the details, two things: which **platforms** the exam talks about, and the **vocabulary** that every later chapter uses. If a word in a question is unfamiliar, it is probably explained here or in the glossary (19.5).

1.1 One engine, four platforms



All four run the same SQL Database Engine and the same T-SQL. They differ in who manages the server and how quickly new features arrive.

Platform	What it is	Things the exam expects you to know
SQL Server 2025	The product you install yourself (on a PC, a server, a VM, or connected to Azure with Azure Arc)	You manage patching, backups, and configuration. Many new AI features are **preview** here and need `PREVIEW_FEATURES = ON`. Some features need `sp_configure` switches (for example `external rest endpoint enabled`)
Azure SQL Managed Instance	A managed SQL Server instance in Azure	*"Near 100% compatibility with the latest SQL Server Enterprise Edition"* — SQL Agent, CLR, cross-database queries. In-Memory OLTP needs the **Business Critical** tier. Newest features need the *SQL Server 2025* or *Always-up-to-date* update policy
Azure SQL Database	A single managed database (or elastic pool) in Azure	Serverless (pauses when idle) and Hyperscale (no fixed maximum size) options. Usually the first place new SQL features appear. RCSI and TDE are on by default
SQL database in Microsoft Fabric	A software-as-a-service (SaaS) database inside a Fabric workspace	*"Built on Azure SQL Database technology."* Changes are automatically mirrored to **OneLake** as Delta Parquet files. Microsoft Entra authentication only. Some features are not available (see the note below)

> PLAIN ENGLISH
>
> **SaaS** (software as a service) means you just use the product — there is no server to configure. **Microsoft Fabric** is Microsoft's analytics platform, and **OneLake** is its single data lake. **Azure Arc** lets you manage servers outside Azure as if they were Azure resources.

WATCH OUT

SQL database in Fabric can't create memory-optimized tables, ledger tables, or Always Encrypted tables, and it does not allow `ALTER TABLE … SWITCH PARTITION`. Its DDL triggers are database-scoped only, and it has no change data capture. Tables with `json` or `vector` columns, or with a clustered columnstore index (which can only be created inside `CREATE TABLE` there), are not mirrored to OneLake. If a question is set in Fabric and an answer depends on one of these, that answer is wrong. Appendix E has the full feature-by-platform table.

1.2 Database words you need

Word	Meaning in plain English
Table, row, column	A table stores data in rows (records) and columns (fields)
OLTP vs analytics	*OLTP* (online transaction processing) = many small, fast reads and writes, like placing orders. *Analytics* = scanning and summarizing many rows, like monthly revenue reports
Data type	What kind of value a column holds: `int`, `decimal(10,2)`, `nvarchar(100)`, `datetime2`, `json`, `vector(1536)` …
Primary key (PK)	The column(s) that identify each row uniquely. No NULLs, one per table
Index	An extra structure that helps SQL find rows fast, like the index at the back of a book
Clustered index	The table's rows stored in key order — the index *is* the table. One per table
Seek vs scan	A *seek* jumps straight to the rows it needs; a *scan* reads every row. Seeks are what you want for selective queries
Transaction	A group of changes that either all succeed (`COMMIT`) or all are undone (`ROLLBACK`)
Lock / blocking	A transaction locks data it changes; another transaction that needs the same data must wait — that wait is *blocking*
Execution plan	The step-by-step recipe SQL chose to run a query (which indexes, which joins, in what order)
DMV	*Dynamic management view*: a system view such as `sys.dm_exec_requests` that shows what the server is doing right now
Stored procedure / function / view / trigger	Saved T-SQL objects (Chapter 5)
LOB	*Large object*: `varchar(max)`, `nvarchar(max)`, `varbinary(max)` — values up to 2 GB that some features can't use directly
Schema	Two meanings! (1) a *namespace* that groups objects (`Sales.Orders` is in the `Sales` schema); (2) the *design* of a database (its tables and columns). The sentence tells you which
Principal / securable	A *principal* is who (a login, user, role, or app); a *securable* is what (a database, schema, table, procedure)

1.3 AI words you need

Word	Meaning in plain English	Example
Model	A trained AI program that turns input into output	A chat model, an embedding model
LLM / SLM	Large / small language model — a model that reads and writes text	GPT-family chat models; small models that can run locally
Prompt	The text you send to a model, often with a **system** message (rules) and a **user** message (the question)	"You are a helpful HR assistant…"
Token	A small piece of text (about part of a word). Models have token limits, and you pay per token	Shorter context = fewer tokens = cheaper and faster
Embedding	A list of numbers that represents the **meaning** of a text. Similar meanings → similar numbers	"car crashed in rain" and "vehicle accident in wet weather" get close embeddings
Vector	The list of numbers itself; its length is the **dimension** count	`text-embedding-3-small` returns 1,536 numbers by default
Vector search / similarity search	Find rows whose vectors are closest to the question's vector	"Find incidents similar to this one"
Distance metric	How "closeness" is measured — cosine, Euclidean, or dot product	Cosine is the usual choice for text
Chunk	A smaller piece of a long document, embedded separately	One embedding per paragraph instead of per 50-page manual
Full-text search	Classic keyword search over words in text	Find documents containing "brake"
Hybrid search	Keyword search and vector search combined	Exact product codes *and* similar meaning
RAG	*Retrieval-augmented generation*: retrieve relevant data first, then let the model answer using it	A chatbot that answers from your current policy documents
Grounding	Giving the model trusted facts to base its answer on	The retrieved policy text
Hallucination	A confident answer that is not true	The model invents a policy that doesn't exist
Fine-tuning	Retraining a model on your data to change its behaviour or style	Not needed for RAG
External model	A database object that records where a model lives and how to call it	`CREATE EXTERNAL MODEL` (Chapter 14)

Word	Meaning in plain English	Example
MCP	*Model Context Protocol*: a standard way for AI assistants to call tools and read live data	An MCP server that gives Copilot your live database schema
Agent	An AI assistant that can plan steps and call tools on its own (with your approval)	Copilot agent mode running `list_tables`

> **KEY TAKEAWAY**
>
> Three ideas carry the whole AI part of the exam: **embeddings capture meaning**, **vector search finds similar meaning**, and **RAG feeds what you found into the model** so it answers from your data instead of guessing.

1.4 Chapter 1 self-check

1. Which platform is SaaS and mirrors its data to OneLake automatically?
2. A question says "the solution must use In-Memory OLTP on Azure SQL Managed Instance". Which tier is needed?
3. What is the difference between a seek and a scan?
4. What does a 1,536-dimension embedding look like?
5. Name the two meanings of the word *schema*.

Answers: 1) SQL database in Microsoft Fabric. 2) Business Critical. 3) A seek jumps to the needed rows using an index; a scan reads all rows. 4) A list of 1,536 numbers (floats). 5) A namespace that groups objects, and the design (structure) of a database.

1.5 Learn more

- Training: Understand your SQL platform choices
- What's new in SQL Server 2025 · SQL database in Fabric overview · Limitations in SQL database in Fabric · Azure SQL: IaaS vs PaaS

Chapter 2 — Tables, data types, and indexes

> **EXAM FOCUS**
> **Design and implement database objects** — design and implement tables, including data types, size, columns, indexes, and columnstore indexes.

2.1 Designing a table: data types and sizes

The rule is simple: **pick the smallest data type that safely holds the data.** Small rows mean less storage, less I/O, smaller indexes, and faster joins — and data types are hard to change later (*"changing column data types in production databases often requires table rebuilds"*).

Data	Good choice	Avoid	Why
Money	`decimal(10,2)`	`float`	*"using FLOAT for financial data instead of DECIMAL can introduce rounding errors"*
Dates and times	`datetime2` (or `date` if no time)	`datetime`	`datetime2` has better precision; `date` is only 3 bytes
Text in many languages	`nvarchar(n)` (2 bytes per character)	—	Unicode
ASCII-only text	`varchar(n)` (1 byte per character)	`nvarchar` when storage matters	Half the size
Surrogate key	`int` / `bigint` with `IDENTITY`	`uniqueidentifier` (GUID)	*"A UNIQUEIDENTIFIER primary key when INT suffices triples your index size and slows every JOIN operation."*
Flexible attributes	`json` (native type)	Many sparse columns	See 4.1
Embeddings	`vector(1536)`	`varbinary`, `nvarchar(max)`	Vector functions and indexes need the `vector` type (15.3)

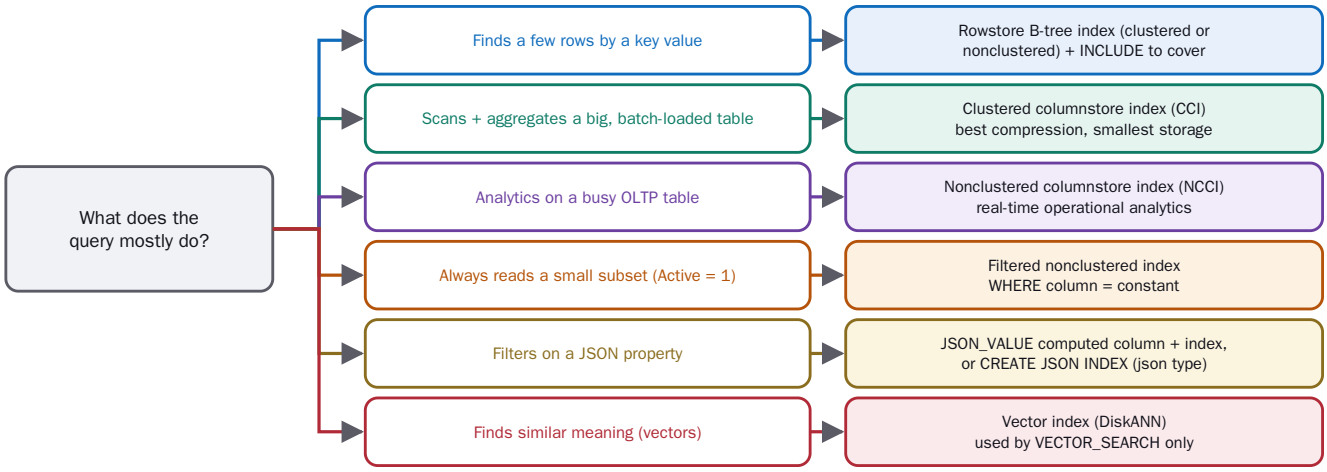
Estimating size. Add the fixed-size columns, add the average size of variable-length columns, and add about 7 bytes of row overhead. Example from the training: about 99 bytes per row × 1 million rows ≈ 94 MB.

A good clustered key is narrow, unique, static, and ever-increasing — exactly what an `int` or `bigint` `IDENTITY` column (or a `SEQUENCE` default) gives you.

Computed columns are calculated from other columns. *"A computed column is a virtual column that isn't physically stored in the table, unless the column is marked PERSISTED."* A computed column must be `PERSISTED` to be used in a `CHECK` or `FOREIGN KEY` constraint, or as a partitioning column, and a `PERSISTED` expression must be deterministic (always the same result for the same input).

```
CREATE TABLE dbo.Product (
    ProductID     int IDENTITY(1,1) PRIMARY KEY,
    ProductName   nvarchar(100) NOT NULL,
    Price         decimal(10,2) NOT NULL,
    StockQuantity int NOT NULL CONSTRAINT DF_Product_Stock DEFAULT 0,
    LastRestocked datetime2 CONSTRAINT DF_Product_Restocked DEFAULT SYSUTCDATETIME()
);
```

2.2 How indexes work, in one picture



Pick the index from what the query does most: find a few rows, scan many rows, read a subset, filter on JSON, or find similar vectors.

- A **clustered index** sorts and stores the table rows by its key. *"There can be only one clustered index per table, because the data rows themselves can be stored in only one order."* A table without one is a **heap** (*"generally not recommended"*).
- A **nonclustered index** is a separate, sorted copy of a few columns plus a pointer back to the row (the clustered key).
- A **key lookup** happens when a nonclustered index finds the row but the query needs columns the index doesn't have, so SQL goes back to the clustered index for each row. Many key lookups = slow.

2.3 Rowstore indexes: covering, key order, and filtered

Covering index rule: put the columns you **search on** in the key — **equality columns first, then range or sort columns** — and put the columns you only **return** in `INCLUDE`.

> *"If a column must be retrieved by a query, but isn't used in the query predicates, aggregations, and sorts, add it as an included column and not as a key column."*

EXAMPLE

The query:

```
SELECT OrderId, OrderDate, Status, TotalAmount
FROM dbo.Orders
WHERE CustomerId = @CustomerId AND OrderDate >= @SinceDate
ORDER BY OrderDate DESC;
```

The index that removes both the key lookup and the sort:

```
CREATE NONCLUSTERED INDEX IX_Orders_Customer_OrderDate
ON dbo.Orders (CustomerId, OrderDate DESC)    -- equality column, then range/sort column
INCLUDE (Status, TotalAmount);                -- returned-only columns
-- OrderId comes along automatically if it is the clustered key
```

SQL seeks to the customer, reads rows already in `OrderDate DESC` order, and never visits the table: *"If a disk-based rowstore index is created with key columns that match those in the ORDER BY clause in the query, the Sort operator in the query plan is eliminated."*

Facts that decide index questions:

- **Included columns can't be sought or sorted.** *"Sort order can be specified only for the key columns in index."* Adding `CustomerId` as an *included* column to an `OrderDate` index only makes it a filter applied after the seek.
- **Leftmost prefix.** An index on `(LastName, FirstName)` helps `WHERE LastName = …` but not `WHERE FirstName = …` alone.
- **The clustered key is free.** *"If the table has a clustered index, the column or columns defined in the clustered index key are automatically added to each nonunique nonclustered index on the table."*
- **Direction.** An index can be read in either direction, so `(A DESC)` also serves `ORDER BY A ASC`.
- **Limits.** Up to 32 key columns; 900 bytes for a clustered key, 1,700 bytes for a nonclustered key. `varchar(max)`, `nvarchar(max)`, `varbinary(max)`, `json`, and `vector` can't be **key** columns, but they can be **included** columns (*"All data types except text, ntext, and image can be used as nonkey columns"*).
- **Only one clustered index.** If the table already has a clustered primary key, "create a clustered index on another column" is not an option.

Filtered indexes index only the rows that match a `WHERE` predicate:

> *"A filtered index is an optimized disk-based rowstore nonclustered index especially suited to cover queries that select from a well-defined subset of data."*

```
CREATE NONCLUSTERED INDEX IX_Orders_Active
ON dbo.Orders (CustomerId, OrderDate)
WHERE Active = 1;                        -- only active rows are in the index
```

Filtered indexes are smaller and cheaper to maintain. The predicate must use **simple comparisons with constants** — no `LIKE`, no computed columns, and no functions such as `DATEADD(DAY, -7, SYSUTCDATETIME())`. They can't be built on a view, and the query's `WHERE` must fall inside the filter to use the index.

> **SCENARIO PATTERN**
>
> **Dashboard query scans the clustered index** — `SELECT VehicleId, LastUpdatedUtc, EngineStatus, BatteryHealth FROM dbo.VehicleHealthSummary WHERE FleetId = @FleetId ORDER BY LastUpdatedUtc DESC` with a clustered PK on `VehicleId` → a nonclustered index **on** `FleetId` that **includes** `LastUpdatedUtc`, `EngineStatus`, `BatteryHealth`. Leading with `LastUpdatedUtc` can't seek on `FleetId`; a filtered index with `DATEADD` isn't even valid.

2.4 Columnstore indexes

Columnstore stores each column separately and compresses it heavily. It is excellent for scanning and aggregating millions of rows, and poor for single-row lookups and heavy updates.

	Clustered columnstore index (CCI)	Nonclustered columnstore index (NCCI)
What it is	**The table itself**, stored as columns	An **extra** column copy on top of a normal rowstore table
Storage	Smallest: *"up to 10 times"* compression, only one copy	About *"10% more storage on average"* — the rowstore stays
Best for	Big fact tables, data warehouse, IoT/log tables loaded in batches with few updates	**Real-time operational analytics**: an OLTP table that also needs analytic queries
Avoid when	*"More than 10% of the operations on the table are updates and deletes"*, or fewer than ~1 million rows per partition	The table is only OLTP and nobody runs analytics

> *"Real-time analytics uses an updateable nonclustered columnstore index on a rowstore table. The columnstore index maintains a copy of the data, so the OLTP and analytics workloads run against separate copies of the data. This minimizes the performance impact of both workloads running at the same time."*

```
-- Data warehouse fact table
CREATE CLUSTERED COLUMNSTORE INDEX CCI_FactSales ON dbo.FactSales;

-- OLTP table that also needs live reporting
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders
ON dbo.Orders (OrderDate, CustomerId, ProductId, Quantity, TotalAmount);

-- Filtered NCCI on "warm" rows only
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI_Orders_Shipped
ON dbo.Orders (OrderDate, CustomerId, TotalAmount)
WHERE OrderStatus = 5;
```

How it works, briefly: rows are grouped into **rowgroups** of up to 1,048,576 rows, each column of a rowgroup is compressed separately, and queries run in **batch mode** (*"two to four times"* faster). Small inserts first land in a **delta rowgroup** (row storage) and are compressed later by the *tuple mover*.

> **KNOW THE DIFFERENCE**
>
> **Which index for which table?** A 10 TB fact table loaded nightly and scanned with `SUM()` → **CCI**. A busy orders table that must also feed near-real-time revenue reports → **NCCI**. A small product lookup table used for point lookups and joins, with "minimize storage" → **clustered rowstore** index (a nonclustered index would add a second structure; columnstore is for big scans). "Only index rows where `Active = 1`" → **filtered** nonclustered index.

> **COMMON EXAM TRAP**
>
> - An NCCI **adds** storage; only a CCI **replaces** the rowstore and minimizes storage.
> - A CCI on a table with continuous OLTP updates breaks "without significantly affecting OLTP throughput".
> - A filtered *rowstore* index does not reduce read time for big analytic scans — columnstore does.
> - An NCCI can't use `INCLUDE` or `ASC` / `DESC`, and columnstore indexes can't enforce constraints (add a B-tree index for a primary key).

2.5 Chapter 2 self-check

1. A query filters `WHERE Region = @r AND SaleDate BETWEEN @a AND @b` and returns `Amount`. Write the best index.
2. Why doesn't adding `CustomerId` as an included column to an index on `OrderDate` turn a range scan into a customer seek?
3. Which index type is "an additional copy" and which one "is the table"?
4. Can a filtered index use `WHERE CreatedAt > DATEADD(day, -7, SYSUTCDATETIME())`?
5. Which data type should hold currency amounts?

Answers: 1) `CREATE INDEX IX ON dbo.Sales (Region, SaleDate) INCLUDE (Amount);` 2) Included columns are stored only at the leaf level and are not part of the key, so they can't be sought. 3) NCCI is an extra copy; CCI is the table. 4) No — the filter must compare with constants. 5) `decimal(p,s)`.

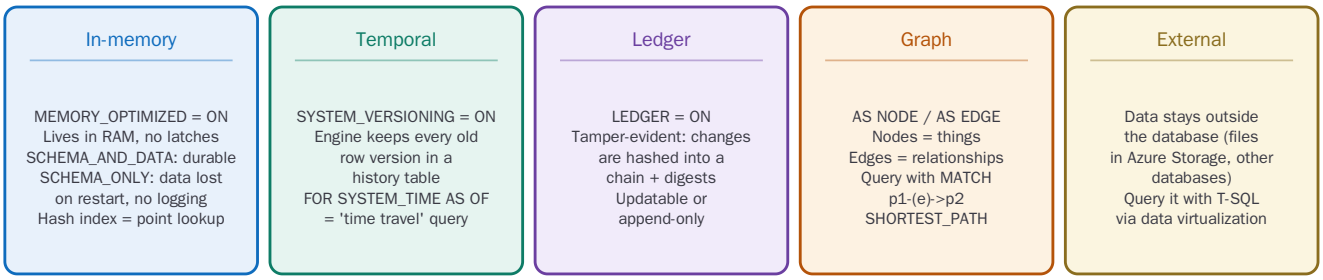
2.6 Learn more

- Training: Build effective tables · Optimize with indexes
- Index architecture and design guide · Create indexes with included columns · Create filtered indexes
- Columnstore indexes overview · Columnstore design guidance · Real-time operational analytics

---

Chapter 3 — Specialized tables: in-memory, temporal, ledger, graph, and external

> **EXAM FOCUS**
>
> **Design and implement database objects** — design and implement specialized tables, including in-memory, temporal, external, ledger, and graph.



Five special table types, each solving one problem: speed, history, tamper-evidence, relationships, or data that stays outside the database.

You need…	Use	One-line reason
The lowest latency for hot OLTP data, session state, or ETL staging	**Memory-optimized table**	Lives in memory, lock- and latch-free
"What did this row look like on date X?" without writing history code	**Temporal table**	The engine keeps every old version automatically
Proof for auditors that nobody (not even a DBA) changed the data	**Ledger table**	Every change is cryptographically chained
Many-to-many, multi-hop relationships ("friends of friends")	**Graph tables**	Nodes + edges, queried with `MATCH`
Query files in a data lake without loading them	**External table**	Data stays where it is; read-only

3.1 Memory-optimized (In-Memory OLTP) tables

A memory-optimized table lives entirely in memory and uses lock-free, latch-free structures, so it is extremely fast for high-throughput work such as gaming, trading, session state, and caching.

Durability — the big choice:

Option	What survives a restart	Logging	Use for
`DURABILITY = SCHEMA_AND_DATA` (default)	Table **and** data	Fully logged, like a disk table	Hot OLTP data you can't lose
`DURABILITY = SCHEMA_ONLY`	Table definition only — **the data is lost**	*"avoids transaction logging, which can significantly reduce I/O operations"*	Transient data: caches, session state, staging, leaderboards that are rebuilt

Indexes on memory-optimized tables: every memory-optimized table needs at least one index (and a durable one needs a primary key).

- **Hash index** — *"Hash indexes are primarily used for point lookups and not for range scans."* It needs an equality test on **all** its key columns and a `BUCKET_COUNT` (ideally 1–2 times the number of distinct key values).
- **Memory-optimized nonclustered index** — for range queries, inequality, and `ORDER BY`. When unsure, *"the general recommendation is to start with nonclustered indexes."*

```
CREATE TABLE dbo.Leaderboard
(
    PlayerId  int NOT NULL PRIMARY KEY NONCLUSTERED HASH WITH (BUCKET_COUNT = 1048576),
    Score     int NOT NULL,
    UpdatedAt datetime2 NOT NULL,
    INDEX IX_Score NONCLUSTERED (Score)             -- range / ORDER BY queries
)
WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_ONLY);
```

Where it runs: SQL Server; Azure SQL Database **Premium** (DTU) and **Business Critical** (vCore) — Hyperscale supports a subset; Azure SQL Managed Instance **Business Critical only**. **Not** in SQL database in Fabric, and not in General Purpose, Standard, or Basic tiers. Unsupported column types include `json`, `vector`, `xml`, and `datetimeoffset`.

> **SCENARIO PATTERN**
> "Reduce write latency during peak events, point lookups, data does NOT need to persist after a restart" → **`SCHEMA_ONLY` memory-optimized table + `HASH` index**. `SCHEMA_AND_DATA` logs every change for durability you don't need; a nonclustered columnstore index is for analytics, not lookups.

3.2 Temporal (system-versioned) tables

A **temporal table** keeps a full history of every row automatically. When a row changes, SQL Server moves the old version into a **history table** and stamps it with the time range when it was valid (in UTC).

```
CREATE TABLE dbo.Employee
(
    EmployeeID   int NOT NULL PRIMARY KEY CLUSTERED,
    Name         nvarchar(100) NOT NULL,
    AnnualSalary decimal(10,2) NOT NULL,
    ValidFrom    datetime2 GENERATED ALWAYS AS ROW START,
    ValidTo      datetime2 GENERATED ALWAYS AS ROW END,
    PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)
)
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = dbo.EmployeeHistory));

-- "Time travel" query
SELECT * FROM dbo.Employee FOR SYSTEM_TIME AS OF '2026-01-01' WHERE EmployeeID = 1;
```

Requirements and rules: a **primary key**; exactly one `PERIOD FOR SYSTEM_TIME` with two `datetime2` columns; other `FOR SYSTEM_TIME` options are `FROM … TO`, `BETWEEN … AND`, `CONTAINED IN`, and `ALL`. Limits that show up in questions:

- *"TRUNCATE TABLE isn't supported while SYSTEM_VERSIONING is ON."*
- *"Direct modification of the data in a history table isn't permitted."*
- `INSTEAD OF` triggers aren't allowed; `AFTER` triggers only on the current table.
- History grows forever unless you set `HISTORY_RETENTION_PERIOD` (for example `6 MONTHS`).
- In SQL database in Fabric the history table isn't mirrored to OneLake.

> **KNOW THE DIFFERENCE**
>
> **Temporal vs partitioning.** A temporal table keeps *history*. It is **not** partitioning and gives you no partition-level retention. A requirement such as *"Telemetry data must be stored in a partitioned table"* is only met by a partition scheme (4.5).

3.3 Ledger tables

Ledger makes data **tamper-evident**: *"You can cryptographically attest to other parties, such as auditors or other business parties, that your data hasn't been tampered with."* Each transaction's changes are hashed (SHA-256) into a chain of blocks, like a blockchain. Ledger can't *prevent* tampering by a powerful attacker, but it *"guarantees that any tampering will be detected."*

Type	Allows	History	Typical use
Updatable ledger table	INSERT, UPDATE, DELETE	Yes — a history table and a ledger view	System-of-record data that changes (balances, employee records)
Append-only ledger table	INSERT only	No history table	Logs, security events (SIEM)

```
-- Updatable ledger table (system-versioned, but it doesn't need a PERIOD)
CREATE TABLE HR.EmployeeSalary
(   EmployeeID int NOT NULL PRIMARY KEY CLUSTERED,
    Salary     decimal(10,2) NOT NULL )
WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = HR.EmployeeSalaryHistory), LEDGER = ON);

-- Append-only ledger table
CREATE TABLE Security.KeyCardEvents
(   EmployeeID int NOT NULL, Operation nvarchar(1024) NOT NULL, EventTime datetime2 NOT NULL
)
WITH (LEDGER = ON (APPEND_ONLY = ON));

-- Generate a database digest (store it outside the database); verify later with
sp_verify_database_ledger
EXECUTE sp_generate_database_ledger_digest;
```

The **database digest** (the hash of the latest block) is stored outside the database — for example in immutable Azure Blob Storage or Azure Confidential Ledger — and verification (`sp_verify_database_ledger`, or `sys.sp_verify_database_ledger_from_digest_storage` with automatic digest storage) recomputes the hashes and compares them.

> **COMMON EXAM TRAP**
>
> - Ledger ≠ auditing. **Auditing** records *who did what*; **ledger** *proves the data wasn't changed*. "Tamper-evident history with cryptographic proof" → ledger.
> - Existing tables **can't be converted** to ledger tables, and a ledger table **can't be turned back** into a normal table.
> - Ledger tables don't support `TRUNCATE`, partition `SWITCH`, memory-optimized tables, graph tables, or `vector` / `xml` columns.
> - Ledger is available in SQL Server 2022+, Azure SQL Database, and Azure SQL Managed Instance — **not** in SQL database in Fabric.

3.4 Graph tables

SQL Graph models **things** as **node** tables and **relationships** as **edge** tables. It is easier than long chains of self-joins for many-to-many, multi-hop questions. For a simple parent-child relationship, a normal foreign key is still the right choice.

```
CREATE TABLE dbo.Person (PersonID int PRIMARY KEY, DisplayName nvarchar(100)) AS NODE;
CREATE TABLE dbo.Knows  (Since date) AS EDGE;

-- Insert an edge by selecting the two nodes' hidden $node_id values
INSERT INTO dbo.Knows ($from_id, $to_id, Since)
VALUES ((SELECT $node_id FROM dbo.Person WHERE PersonID = 1),
        (SELECT $node_id FROM dbo.Person WHERE PersonID = 2), '2024-05-01');
```

Every node gets a hidden `$node_id`; every edge gets `$edge_id`, `$from_id`, and `$to_id`. Edges are always **directed**. You can't `UPDATE` `$from_id` / `$to_id` — delete and re-insert the edge. An **edge**

constraint (`CONNECTION (Customer TO Product)`) restricts which node types an edge may connect. Graph tables can't be temporal or memory-optimized, and in SQL database in Fabric they aren't mirrored to OneLake. Querying with `MATCH` is in 7.4.

3.5 External tables (data virtualization)

An **external table** is a table definition that points to files (Parquet or CSV, and Delta in Azure SQL Database) in a data lake. Queries read the files **in place**; the data is not copied into the database, and the table is **read-only**.

```
-- Only for PRIVATE storage: create a credential and add CREDENTIAL = LakeCredential to the
data source
-- (this public dataset needs none)
CREATE DATABASE SCOPED CREDENTIAL LakeCredential WITH IDENTITY = 'Managed Identity';
CREATE EXTERNAL DATA SOURCE TaxiLake
    WITH (LOCATION = 'abs://nyctlc@azureopendatastorage.blob.core.windows.net');
CREATE EXTERNAL FILE FORMAT ParquetFormat WITH (FORMAT_TYPE = PARQUET);
CREATE EXTERNAL TABLE dbo.TaxiRides (vendorID varchar(100), pickupTime datetime2,
passengerCount int)
WITH (LOCATION = 'yellow/puYear=*/puMonth=*/*.parquet',
      DATA_SOURCE = TaxiLake, FILE_FORMAT = ParquetFormat);

-- Ad hoc alternative, no table definition
SELECT TOP 10 * FROM OPENROWSET(BULK 'yellow/*/*/*.parquet',
       DATA_SOURCE = 'TaxiLake', FORMAT = 'parquet') AS r;
```

`OPENROWSET` is *"optimized for ad hoc querying"*; `CREATE EXTERNAL TABLE` is *"optimized for repetitive querying."* In SQL database in Fabric the only data source is **OneLake**. External tables support no DML, no row-level security or masking (in Azure SQL Database), and no temporal `FOR SYSTEM_TIME` clause.

3.6 Chapter 3 self-check

1. Which durability option skips transaction logging, and what happens to the data on restart?
2. Which index type suits `WHERE PlayerId = @id` on a memory-optimized table, and which suits `ORDER BY Score DESC`?
3. Auditors want cryptographic proof that employee rows weren't altered. Which feature, and which type of table if rows are updated?
4. Can you `TRUNCATE` a temporal table while system versioning is on?
5. Which platform can't create memory-optimized, ledger, or Always Encrypted tables?

Answers: 1) `SCHEMA_ONLY`; the table stays but its data is lost. 2) Hash index; memory-optimized nonclustered index. 3) Ledger; an updatable ledger table. 4) No. 5) SQL database in Microsoft Fabric.

3.7 Learn more

- Training: Use specialized table types
- In-memory: Defining durability · Indexes for memory-optimized tables · In-memory technologies in Azure SQL

- Temporal: Temporal tables · Ledger: Ledger overview · Updatable ledger tables
- Graph: SQL Graph architecture · External: Data virtualization in Azure SQL Database · CREATE EXTERNAL TABLE

Chapter 4 — JSON columns, constraints, sequences, and partitioning

> **EXAM FOCUS**
> **Design and implement database objects** — design and implement JSON columns and indexes · design and implement database constraints, including PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, and DEFAULT · design and implement SEQUENCES · design and implement partitioning for tables and indexes.

4.1 Storing JSON

Keep predictable fields in normal typed columns, and put the variable "bag of attributes" in a JSON column. (If every row has the same shape, use normal columns, not JSON.) You have two ways to store it:

	Native `json` data type	`nvarchar(max)` text
Format	Binary, already parsed: *"More efficient reads … More efficient writes … More efficient storage"*	Plain text, parsed on every read
Validation	Automatic — input must be a JSON **object or array** (a bare scalar such as `'1'` raises error 13609)	You add `CHECK (ISJSON(col) = 1)`
Indexing	`CREATE JSON INDEX`	Computed column with `JSON_VALUE` + normal index
Platforms	GA in SQL Server 2025, Azure SQL Database, Azure SQL MI (2025 / Always-up-to-date policy), SQL database in Fabric	Everywhere
Update in place	`UPDATE t SET doc.modify('$.a', 14859)` — *"the preferred way to modify a JSON document in a json column"*	`JSON_MODIFY`

A `json` column can be up to 2 GB with up to 128 nesting levels. It **can't** be a key column in a normal index (it can be an `INCLUDE` column), and it isn't supported in memory-optimized tables.

Rejecting bad data at the door:

```
-- Malformed JSON never gets in (nvarchar storage)
ALTER TABLE dbo.Events ADD CONSTRAINT CK_Events_Payload_IsJson CHECK (ISJSON(Payload) = 1);

-- A required property must exist
ALTER TABLE dbo.Orders ADD CONSTRAINT CK_Orders_Basket
    CHECK (JSON_PATH_EXISTS(OrderDetails, '$.basket') = 1);

-- Duplicate payloads never get in: hash the LOB, put a unique index on the hash
ALTER TABLE dbo.Events
    ADD PayloadHash AS CAST(HASHBYTES('SHA2_256', Payload) AS binary(32)) PERSISTED;
CREATE UNIQUE INDEX UX_Events_PayloadHash ON dbo.Events (PayloadHash);
```

> **PLAIN ENGLISH**
> A **hash** (here SHA-256) turns any input into a short, fixed-size fingerprint — 32 bytes for `SHA2_256`. The same payload always gives the same fingerprint, so a unique index on the fingerprint blocks duplicates, even though the payload itself (an `nvarchar(max)` LOB) can't be an index key. Only `SHA2_256` and `SHA2_512` are not deprecated.

> **SCENARIO PATTERN**
> "The ingestion pipeline fails because of **malformed JSON** and **duplicate payloads**" → a `CHECK` constraint with `ISJSON`, **and** a unique index on a hash of the payload. Snapshot isolation, foreign keys, and a trigger that "repairs" JSON don't solve either problem.

4.2 Indexing JSON

Pattern 1 — JSON stored as text: a computed column plus a normal index.

> *"When you store JSON as varchar or nvarchar text, or when you want to index a single JSON property as a strongly typed column, create a computed column with JSON_VALUE and build a standard index on that column."*

```
ALTER TABLE WebSite.Logs ADD severity AS JSON_VALUE([log], '$.severity');   -- PERSISTED is
optional
CREATE INDEX ix_severity ON WebSite.Logs (severity)
    INCLUDE (LogId, LogDateTime, [log]);      -- cover the report: no key lookups
```

Queries keep using `WHERE JSON_VALUE([log], '$.severity') = 'Error'`; SQL Server sees the matching computed column and uses the index. Use `JSON_VALUE`, not `JSON_QUERY` (which returns NULL for a scalar and returns `nvarchar(max)`, which can't be a key). Cast to a small type when you can, because index keys over 1,700 bytes can't be indexed.

Pattern 2 — native `json` type: `CREATE JSON INDEX`.

```
CREATE TABLE dbo.VehicleTelemetry (
    TelemetryId  bigint IDENTITY PRIMARY KEY CLUSTERED,      -- a clustered PK is REQUIRED
    VehicleId    int NOT NULL,
    LocationJson json NOT NULL
);

CREATE JSON INDEX JI_VehicleTelemetry_Location ON dbo.VehicleTelemetry (LocationJson)
    FOR ('$.location.latitude', '$.location.longitude', '$.location.accuracy');

SELECT TelemetryId FROM dbo.VehicleTelemetry
WHERE JSON_VALUE(LocationJson, '$.location.latitude' RETURNING decimal(9,6)) = 52.370216;
```

Rules for `CREATE JSON INDEX`:

- The table **must have a clustered primary key**.

- `FOR (…)` lists the paths to index; each path *"Recursively indexes all keys/values from the specified path onward."* Leave out `FOR` and the whole document (`$`) is indexed.
- **Only the listed paths and their children are indexed.** In the example above, `$.location.heading` is a *sibling* of `latitude`, so a filter on `heading` is **not** served by the index. To cover it, index `$.location` instead.
- Paths can't overlap (`'$.a'` and `'$.a.b'` together is an error), and *"Modification of paths requires recreating the JSON index."*
- Only one JSON index per `json` column; it is built **offline**; it can't be used in index hints or on views; it helps `JSON_VALUE(…) = value`, `JSON_PATH_EXISTS`, and `JSON_CONTAINS` predicates (not `LIKE`).

> **COMMON EXAM TRAP**
> "Queries that filter on `$.location.heading` will use the index" when the `FOR` list names only latitude, longitude, and accuracy → **No**. A JSON index covers the listed paths and what is *under* them, never their siblings.

4.3 Constraints

Constraints are rules the engine enforces on **every** insert and update, whatever app or script writes the data.

Constraint	Rule	Facts the exam checks
PRIMARY KEY	Uniquely identifies each row; no NULLs	One per table; creates a unique index (clustered by default if none exists)
FOREIGN KEY	Value must exist in the parent table	**A NULL foreign key isn't checked**, so an FK doesn't make a value mandatory. Doesn't create an index automatically. Cascades: `NO ACTION` (default), `CASCADE`, `SET NULL`, `SET DEFAULT`
UNIQUE	No duplicate values	Allows **one** NULL per column. Creates a unique nonclustered index by default
CHECK	A true/false rule on the row	Rejects only FALSE — *NULL evaluates to UNKNOWN and passes*. Can't reference another table
DEFAULT	Value used when none is supplied	One per column; can use `NEXT VALUE FOR` a sequence
NOT NULL	A value is required	Part of the column definition

Adding a rule for future rows only — `WITH NOCHECK`:

```
-- Some existing rows have NULL owners; every NEW or UPDATED row must have one
ALTER TABLE dbo.Rooms WITH NOCHECK
    ADD CONSTRAINT CK_Rooms_Owner CHECK (Owner IS NOT NULL);

-- Later, after cleaning the data, validate everything and make the constraint trusted
ALTER TABLE dbo.Rooms WITH CHECK CHECK CONSTRAINT CK_Rooms_Owner;
```

> *"If you don't want to verify new CHECK or FOREIGN KEY constraints against existing data, use WITH NOCHECK … The new constraint is evaluated in all later data updates."*

New constraints are `WITH CHECK` by default, so without `NOCHECK` the `ALTER` fails while bad rows exist. A `NOCHECK` constraint is *untrusted* — the optimizer ignores it until you re-check it.

TIP

Name every constraint explicitly (`CK_Rooms_Owner` , `DF_Product_Stock`). System-generated names differ between environments, which causes noise in database projects and deployments.

4.4 SEQUENCE vs IDENTITY

A **SEQUENCE** is a standalone number generator that isn't tied to any table. **IDENTITY** is the simple per-table auto-number.

Feature	SEQUENCE	IDENTITY
Tied to one table	No	Yes (one identity column per table)
Shared across tables	Yes	No
Get the next number **before** inserting	Yes — `NEXT VALUE FOR`	No
Reserve a block of numbers	Yes — `sp_sequence_get_range`	No
Cycle / restart / change increment later	Yes — `CYCLE` , `ALTER SEQUENCE … RESTART`	No

```
CREATE SEQUENCE dbo.OrderNumber AS int START WITH 1000 INCREMENT BY 1 NO CYCLE;

DECLARE @next int = NEXT VALUE FOR dbo.OrderNumber;                 -- number before insert
INSERT dbo.Orders (OrderNumber, CustomerID) VALUES (@next, 100);

-- Shared series via defaults in two tables
CREATE TABLE dbo.WebOrders   (OrderNo int DEFAULT (NEXT VALUE FOR dbo.OrderNumber) PRIMARY
KEY, …);
CREATE TABLE dbo.PhoneOrders (OrderNo int DEFAULT (NEXT VALUE FOR dbo.OrderNumber) PRIMARY
KEY, …);
```

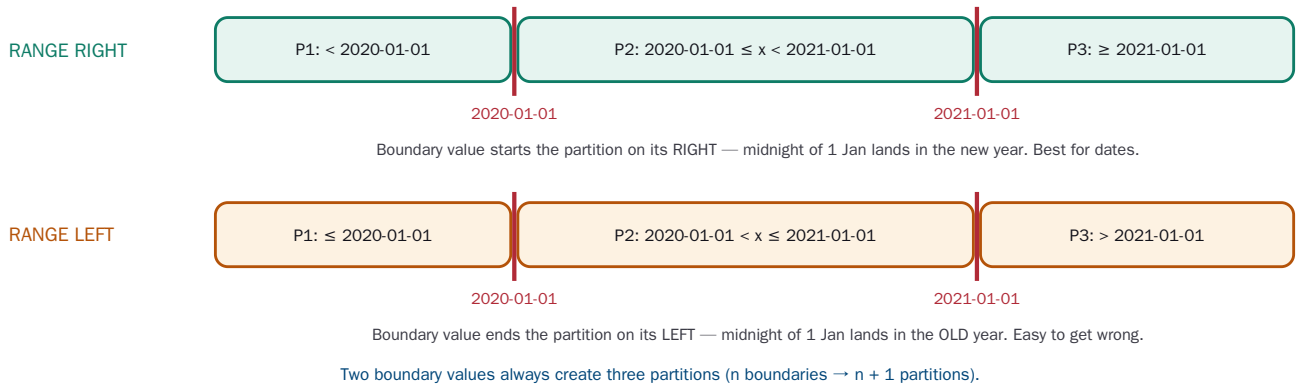
COMMON EXAM TRAP

- Both have **gaps**: *"Sequence numbers are generated outside the scope of the current transaction"* — a rolled-back insert still uses the number, and cached values can be lost on an unexpected shutdown.
- A sequence doesn't enforce uniqueness — add a `PRIMARY KEY` or `UNIQUE` constraint.
- With no `AS` type, a sequence is `bigint`; with no `START WITH`, an ascending sequence starts at the type's **minimum** value, not 1.
- `NEXT VALUE FOR` can't be used in views, functions, computed columns, `CHECK` constraints, or a `WHERE` clause — but it can be used in a `DEFAULT` constraint.

4.5 Partitioning tables and indexes

Partitioning splits one logical table into many physical slices by one column (usually a date). Three pieces:

1. A **partition function** defines the boundary values.
2. A **partition scheme** maps each partition to a filegroup (in Azure SQL Database and Fabric, only `PRIMARY`).
3. The table or index is created `ON scheme(column)`.



RANGE RIGHT puts each boundary value at the START of a partition, so all of 1 January stays with the new year. That is why it is the usual choice for dates.

```
CREATE PARTITION FUNCTION PF_Year (datetime2)
    AS RANGE RIGHT FOR VALUES ('2020-01-01', '2021-01-01');   -- 2 boundaries = 3 partitions

CREATE PARTITION SCHEME PS_Year AS PARTITION PF_Year ALL TO ([PRIMARY]);

CREATE TABLE dbo.Telemetry (
    TelemetryId bigint NOT NULL,
    DateKey     datetime2 NOT NULL,
    Reading     decimal(9,3) NULL,
    CONSTRAINT PK_Telemetry PRIMARY KEY CLUSTERED (TelemetryId, DateKey)   -- must include
DateKey
) ON PS_Year (DateKey);
```

Partition	Range (RANGE RIGHT)	Holds
1	`DateKey < 2020-01-01`	anything before 2020 (empty if the data starts in 2020)
2	`2020-01-01 ≤ DateKey < 2021-01-01`	all of 2020
3	`DateKey ≥ 2021-01-01`	all of 2021

Rules and facts:

- **n boundaries → n + 1 partitions** (up to 15,000).
- **`RANGE LEFT` is the default** if you don't say. For dates, choose **RIGHT**: *"rows with a value of midnight are stored in the same partition as rows with the later values on the same day."* With LEFT, a midnight value on 1 January lands in the previous year.
- The partitioning column must be part of the clustered key and of any unique index; a computed partitioning column must be `PERSISTED`.
- Indexes created on the same scheme are **aligned**. Alignment is required for partition switching and for truncating partitions.
- Partitioning helps maintenance and retention; it *"rarely improves query performance in OLTP systems."* Use it when most queries filter on one column (such as date) or when you archive by period.

4.6 Managing partitions

Goal	Statement	Notes
Remove all rows of one period, fast	`TRUNCATE TABLE dbo.Orders WITH (PARTITIONS (1));`	Deallocates pages; minimal logging; table and indexes must be aligned; works in SQL Server, Azure SQL Database, **and SQL database in Fabric**
Find the partition number for a value	`SELECT $PARTITION.PF_Year('2020-06-15');`	Returns the partition number
Move a partition to an archive table	`ALTER TABLE dbo.Orders SWITCH PARTITION 1 TO dbo.OrdersArchive;`	Metadata only; target must be empty. **Not allowed in SQL database in Fabric**
Add the next period	`ALTER PARTITION SCHEME … NEXT USED [PRIMARY];` then `ALTER PARTITION FUNCTION … SPLIT RANGE ('2022-01-01');`	Best practice: split only an **empty** partition and keep empty partitions at both ends — splitting a populated one moves rows and can log up to 4× more. With a columnstore index, only empty partitions can be split
Remove an old boundary	`ALTER PARTITION FUNCTION PF_Year() MERGE RANGE ('2020-01-01');`	Must be an existing boundary value
Drop partitioning metadata	`DROP PARTITION SCHEME name;`	Only when no table or index uses it. There is **no** `ALTER TABLE … DROP PARTITION SCHEME`

COMMON EXAM TRAP

Removing the oldest month with minimal impact (a classic Yes/No series):

- `TRUNCATE TABLE dbo.Orders WITH (PARTITIONS (n));` → **Yes.** *"TRUNCATE TABLE removes the data by deallocating the data pages … and records only the page deallocations in the transaction log."*
- `DELETE FROM dbo.Orders WHERE OrderDate < DATEADD(month, -36, SYSUTCDATETIME());` → **No.** *"The DELETE statement removes rows one at a time and records an entry in the transaction log for each deleted row"* — and the rolling timestamp isn't even a month boundary.
- `ALTER TABLE dbo.Orders DROP PARTITION SCHEME (…)` → **No.** Invalid syntax, and dropping a scheme removes no rows.
- `TRUNCATE TABLE dbo.Orders;` → **No.** *"If the WITH PARTITIONS clause isn't provided, the entire table is truncated."*

4.7 Chapter 4 self-check

1. Which two features of a native `json` column make it better than `nvarchar(max)` for JSON?
2. What must a table have before you can create a JSON index?

3. A UNIQUE column already has one NULL. Can you insert a second NULL?
4. Why might `CHECK (Price > 0)` still allow a row?
5. You need the next order number before inserting the row. IDENTITY or SEQUENCE?
6. `RANGE LEFT FOR VALUES ('2025-01-01')`: which partition holds `2025-01-01 00:00:00`?
7. Which partition operation is not allowed in SQL database in Fabric?

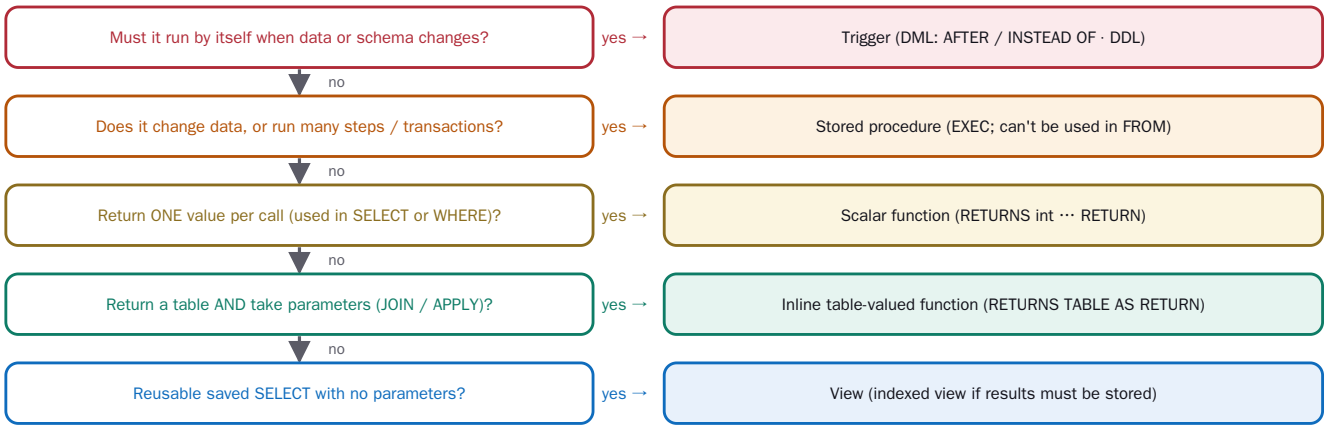
Answers: 1) It is stored already parsed (faster reads/writes, better compression) and validated automatically. 2) A clustered primary key. 3) No — UNIQUE allows only one NULL. 4) If `Price` is NULL, the check is UNKNOWN, which passes. 5) SEQUENCE (`NEXT VALUE FOR`). 6) The first (left) partition — with LEFT the boundary value belongs to the partition on its left. 7) `SWITCH PARTITION` (also partition compression).

4.8 Learn more

- Training: Manage JSON columns and indexes · Enforce data integrity with constraints · Partition tables for scale
- JSON: json data type · CREATE JSON INDEX · Index JSON data · Store JSON documents
- Constraints: Unique and check constraints · Primary and foreign keys · ALTER TABLE
- Sequences: Sequence numbers · CREATE SEQUENCE
- Partitioning: Partitioned tables and indexes · CREATE PARTITION FUNCTION · TRUNCATE TABLE

Chapter 5 — Programmability objects: views, functions, procedures, and triggers

> **EXAM FOCUS**
>
> **Implement programmability objects** — create views · create scalar functions · create table-valued functions · create stored procedures · create triggers.



Avoid multi-statement TVFs in big joins: the optimizer guesses their row count (fixed estimate).

Ask the questions from the top. The first "yes" tells you which object to build.

Capability	View	Stored procedure	Function	Trigger
Accepts parameters	No	Yes	Yes	No
Modifies data	Limited (one base table)	Yes	**No**	Yes
Returns a result set	Yes	Yes	Yes (table-valued)	No
Usable in `SELECT` / `JOIN` / `FROM`	Yes	**No**	Yes	No
Controls transactions	No	Yes	No	Yes
Runs automatically	No	No	No	**Yes**

5.1 Views and indexed views

A **view** is a saved `SELECT` that you query like a table. It doesn't store data. It hides complex joins, exposes only some columns, and can act as a security boundary.

```
CREATE VIEW Sales.HighValueOrders
AS
SELECT OrderID, CustomerID, OrderDate, TotalAmount
FROM Sales.Orders
WHERE TotalAmount > 1000
WITH CHECK OPTION;        -- inserting or updating a row so it would fall outside the view
fails
```

Rules worth knowing: name the columns (don't use `SELECT *`); no `ORDER BY` unless the view also has `TOP` (and even then the order isn't guaranteed); `CREATE VIEW` must be the first statement in its batch; data changes through a view may touch only **one** base table (an `INSTEAD OF` trigger can make a multi-table view updatable). In SQL database in Fabric, views aren't mirrored to OneLake.

Indexed views store the view's result on disk with a clustered index — great for aggregates that are read often, at the cost of slower writes to the base tables. *"Creating a unique clustered index on a view improves query performance, because the view is stored in the database in the same way a table with a clustered index is stored."* In Azure SQL Database, Azure SQL Managed Instance, and SQL Server Enterprise edition, the optimizer can use it even when a query doesn't name the view (on SQL Server Standard edition, query the view `WITH (NOEXPAND)`).

```
CREATE VIEW dbo.vMonthlyRevenue
WITH SCHEMABINDING                                  -- 1. required
AS
SELECT CustomerID, YEAR(OrderDate) AS OrderYear, MONTH(OrderDate) AS OrderMonth,
       SUM(ISNULL(TotalDue, 0)) AS Revenue,         -- SUM of a nullable column needs
ISNULL
       COUNT_BIG(*) AS RowCnt                       -- 2. required with GROUP BY
FROM dbo.Orders                                     -- two-part names
GROUP BY CustomerID, YEAR(OrderDate), MONTH(OrderDate);
GO
CREATE UNIQUE CLUSTERED INDEX IX_vMonthlyRevenue    -- 3. the FIRST index must be unique
clustered
    ON dbo.vMonthlyRevenue (CustomerID, OrderYear, OrderMonth);
```

Not allowed in an indexed view: `ORDER BY`, `TOP`, `DISTINCT`, `HAVING`, `COUNT` (use `COUNT_BIG`), `AVG` / `MIN` / `MAX`, outer joins, subqueries, CTEs, `UNION`, window functions, other views, or nondeterministic expressions.

> **SCENARIO PATTERN**
> "Reduce the time to retrieve calculated monthly revenue per customer **without altering any table**" → a view `WITH SCHEMABINDING` that includes `COUNT_BIG(*)`, plus a **unique clustered index** on it. A view with `ORDER BY` and no `TOP`, a view without `SCHEMABINDING`, a nonclustered first index, or `GROUP BY` without `COUNT_BIG(*)` all fail.

5.2 Scalar functions

A **scalar user-defined function (UDF)** takes parameters and returns **one** value. You can use it wherever an expression is allowed.

```
CREATE FUNCTION dbo.ufn_DaysSinceOrder (@OrderDate datetime2(0))
RETURNS int                                 -- a scalar UDF must declare its return type
AS                                          -- AS is optional
BEGIN
    DECLARE @Days int;
    SELECT @Days = DATEDIFF(day, @OrderDate, GETDATE());   -- start date first, end date
second
    RETURN @Days;                           -- the LAST statement must be RETURN
END;
GO
SELECT OrderID, dbo.ufn_DaysSinceOrder(OrderDate) AS DaysOld FROM Sales.Orders;   -- two-
part name
```

Rules the exam tests:

Rule	Detail
Must end with RETURN	Error 455: *"The last statement included within a function must be a return statement."* A bare expression on its own line won't compile
Two-part name to call it	*"Scalar functions must be invoked by using at least the two-part name of the function (<schema>.<function>)."* `EXEC @ret = ufn_GetTax …` fails; `EXEC @ret = dbo.ufn_GetTax …` works
No side effects	Can't modify database state, use `TRY…CATCH`, dynamic SQL, temp tables, or call stored procedures
Some built-ins are banned	`NEWID`, `NEWSEQUENTIALID`, `RAND`, `TEXTPTR` raise error 443. **`GETDATE()`, `CURRENT_TIMESTAMP`, `GETUTCDATE()` are allowed** — but they make the function nondeterministic
Defaults need `DEFAULT`	To use a parameter's default value, pass the keyword `DEFAULT`
`RETURNS TABLE` is not scalar	That makes it a table-valued function

`DATEDIFF(datepart, startdate, enddate)` returns *"the count (as a signed integer value) of the specified datepart boundaries crossed."* If the start is later than the end, the result is **negative**: `DATEDIFF(year, GETDATE(), @OrderDate)` is negative for a past order. It counts *boundaries*, not full periods — `DATEDIFF(year, '2025-12-31', '2026-01-01')` is 1.

Scalar UDF inlining (compatibility level 150+): SQL Server can rewrite a simple scalar UDF into the calling query, removing the old row-by-row cost. A UDF can't be inlined if it calls a time-dependent function such as `GETDATE()`, has more than one `RETURN`, uses table variables, or is used in certain places (for example a computed column). Check `sys.sql_modules.is_inlineable`. Tip from the training: pass "today" in as a parameter if you need a deterministic, inlinable function.

5.3 Table-valued functions (TVFs)

A **TVF** returns a **table** that you can use in `FROM`, `JOIN`, or `APPLY`.

	Inline TVF	Multi-statement TVF (MSTVF)
Body	One `RETURN (SELECT …)` — no `BEGIN…END`, no declared table	`RETURNS @t TABLE (…)`, then `BEGIN … INSERT @t … RETURN END`
How the optimizer sees it	Expanded into the calling query *"like views with parameters"* — real statistics, good plans	A black box: *"MSTVFs have a fixed cardinality guess of 100 starting with SQL Server 2014"*
Use when	A parameterized, joinable query — almost always	You truly need procedural steps (IF, loops) to build the result

```
-- Inline TVF: one customer's (in the case study, a patient's) transactions between two
dates, with a running total
CREATE FUNCTION dbo.CustomerTransactionsBetweenDates
    (@CustomerId int, @StartDate datetime2, @EndDate datetime2)
RETURNS TABLE
AS
RETURN
(
    SELECT t.TransactionId, t.CustomerId, t.TransactionDate, t.Amount,
           SUM(t.Amount) OVER (PARTITION BY t.CustomerId
                               ORDER BY t.TransactionDate, t.TransactionId
                               ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS
RunningTotal
    FROM dbo.Transactions AS t
    WHERE t.CustomerId = @CustomerId
      AND t.TransactionDate >= @StartDate AND t.TransactionDate < @EndDate
);
GO
-- Developers can join to it…
SELECT x.*, c.Name
FROM dbo.CustomerTransactionsBetweenDates(42, '2026-01-01', '2026-02-01') AS x
JOIN dbo.Customers AS c ON c.CustomerId = x.CustomerId;
-- …or call it once per outer row with APPLY
SELECT c.Name, x.TransactionDate, x.RunningTotal
FROM dbo.Customers AS c
CROSS APPLY dbo.CustomerTransactionsBetweenDates(c.CustomerId, '2026-01-01', '2026-02-01')
AS x;
```

KNOW THE DIFFERENCE

Procedure vs inline TVF vs view. A stored procedure returns rows but *can't* be used in `FROM` / `JOIN`. A view can be joined but *can't* take parameters. An inline TVF does both — it is a **parameterized view**. When a question says "developers must join the result to other tables" and the code says `RETURNS TABLE AS RETURN (…)`, the answer is **FUNCTION**.

TIP

Use `CROSS APPLY` to pass a column from the outer table into a TVF (`OUTER APPLY` keeps outer rows when the function returns nothing). A plain `JOIN … ON 1 = 1` can't pass outer columns as arguments.

5.4 Stored procedures

A **stored procedure** is a named batch of T-SQL you run with `EXEC`. It can take input and **output** parameters, change data in many tables inside one transaction, handle errors, and return a status code.

```
CREATE OR ALTER PROCEDURE dbo.CalculateOrderTotal
    @OrderID int,
    @TotalAmount decimal(10,2) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SELECT @TotalAmount = SUM(Quantity * UnitPrice)
    FROM dbo.OrderDetails WHERE OrderID = @OrderID;
    RETURN 0;
END;
GO
DECLARE @t decimal(10,2);
EXEC dbo.CalculateOrderTotal @OrderID = 42, @TotalAmount = @t OUTPUT;   -- OUTPUT on the
call too
```

Good practice from the training: schema-qualify names, validate parameters early, wrap changes in `TRY…CATCH` with a guarded rollback (6.4), avoid `SELECT *`, and **don't name procedures `sp_…`** (that prefix is for system procedures and can clash with them). Procedures are also a security tool: with **ownership chaining**, a user with only `EXECUTE` on a procedure can work with tables they have no direct permission on (9.4).

5.5 Triggers

A **trigger** is code that runs **automatically**.

Kind	Fires on	Notes
DML `AFTER` trigger	`INSERT`, `UPDATE`, `DELETE` on a **table**	Runs after constraints pass. Can't be defined on views
DML `INSTEAD OF` trigger	`INSERT`, `UPDATE`, `DELETE` on a table **or view**	Replaces the original statement. One per action per object
DDL trigger	`CREATE`, `ALTER`, `DROP`, `GRANT`, `DENY`, `REVOKE`, `UPDATE STATISTICS` …	`ON DATABASE` or `ON ALL SERVER` (Azure SQL Database and Fabric: database only). Uses `EVENTDATA()`; never `INSTEAD OF`

Inside a DML trigger you read two pseudo-tables: **`inserted`** (new rows — filled by INSERT and UPDATE) and **`deleted`** (old rows — filled by DELETE and UPDATE). A trigger fires **once per statement**, so always join to these tables; never assume a single row.

```
-- Stamp LastModifiedUtc whenever a maintenance row changes
CREATE OR ALTER TRIGGER dbo.trg_MaintenanceEvents_SetLastModified
ON dbo.MaintenanceEvents
AFTER UPDATE
AS
BEGIN
    SET NOCOUNT ON;
    IF TRIGGER_NESTLEVEL(@@PROCID) > 1 RETURN;          -- optional re-entry guard
    UPDATE m
    SET    LastModifiedUtc = SYSUTCDATETIME()
    FROM   dbo.MaintenanceEvents AS m
    INNER JOIN inserted AS i ON m.MaintenanceId = i.MaintenanceId;   -- join on the PRIMARY
KEY
END;
GO
-- Audit only real price changes, using old (deleted) and new (inserted) values
CREATE TRIGGER trg_LogPriceChanges ON Products.Catalog AFTER UPDATE
AS
BEGIN
    IF UPDATE(Price)
        INSERT Audit.PriceHistory (ProductID, OldPrice, NewPrice, ChangeDate)
        SELECT d.ProductID, d.Price, i.Price, SYSUTCDATETIME()
        FROM deleted AS d JOIN inserted AS i ON d.ProductID = i.ProductID
        WHERE d.Price <> i.Price;
END;
```

Recursion. *"An AFTER trigger does not call itself recursively unless the RECURSIVE_TRIGGERS database option is set."* `RECURSIVE_TRIGGERS` is **OFF** by default (OFF stops *direct* recursion; indirect recursion is controlled by the server option `nested triggers`). `TRIGGER_NESTLEVEL()` tells you how deeply triggers are nested, so `IF TRIGGER_NESTLEVEL(@@PROCID) > 1 RETURN;` is a common guard.

Switching a trigger off without dropping it:

```
DISABLE TRIGGER ddl_Audit ON DATABASE;          -- database-scoped DDL trigger (no schema
prefix)
-- … deploy …
ENABLE TRIGGER ddl_Audit ON DATABASE;
DISABLE TRIGGER Sales.trg_Orders ON Sales.Orders;   -- a DML trigger: ON <table>
```

> *"Disabling a trigger does not drop it. The trigger still exists as an object in the current database. However, the trigger does not fire."*

COMMON EXAM TRAP

- **Join `inserted` on the primary key.** Joining on a non-unique column (for example `VehicleId` in a table of maintenance events) updates every event of that vehicle when only one changed.
- **The "avoid recursive updates" drag-and-drop (fleet case study).** The answer key pairs the primary-key join with `WHERE m.LastModifiedUtc <> i.LastModifiedUtc` as the "recursion guard", so that is the most likely expected answer. Know the real behaviour too: in an `AFTER` trigger the table row and `inserted` already hold the same new value, so that predicate filters out every row, and a `WHERE` clause can't stop a trigger from firing anyway. In real code, rely on `RECURSIVE_TRIGGERS` OFF (the default) or `IF TRIGGER_NESTLEVEL(@@PROCID) > 1 RETURN;`.
- **`ALTER TRIGGER` re-enables a disabled trigger.** To stop a trigger for a deployment, use `DISABLE TRIGGER`. `ALTER DATABASE AUDIT SPECIFICATION` belongs to SQL Server Audit, not triggers.
- **DDL triggers ignore row changes.** Row `INSERT` / `UPDATE` / `DELETE` never fires a DDL trigger.
- **Prefer constraints.** Use `CHECK` for single-row rules and `FOREIGN KEY` for relationships; use triggers only when constraints can't express the rule (for example, a rule that spans tables).
- `TRUNCATE TABLE` doesn't fire triggers. Temporal tables allow no `INSTEAD OF` triggers.

5.6 Chapter 5 self-check

1. Name the three requirements for indexing an aggregating view.
2. A scalar UDF's body ends with `IF @x > 0 RETURN 1 ELSE RETURN 0`. Does it compile?
3. Why does `EXEC @ret = ufn_GetTaxMultiplier @TaxAmt = 100, @StateCode = 'WA';` fail?
4. Reports join an MSTVF to a large table and get inconsistent plans. What do you change and why?
5. Which object takes parameters *and* can be joined in a `FROM` clause?
6. Inside an `AFTER UPDATE` trigger, where are the old values?
7. Which statement stops a database DDL trigger from firing but keeps it?

Answers: 1) `WITH SCHEMABINDING`, `COUNT_BIG(*)` (with `GROUP BY`), and a unique clustered index first. 2) No — error 455: the last statement must be `RETURN`. 3) A scalar function must be called with at least a two-part name (`dbo.ufn_GetTaxMultiplier`). 4) Rewrite it as an inline TVF; MSTVFs get a fixed estimate of 100 rows, while inline TVFs are expanded with real statistics. 5) An inline table-valued function. 6) In the `deleted` pseudo-table. 7) `DISABLE TRIGGER name ON DATABASE`.

5.7 Learn more

- Training: Implement programmability objects with SQL — views, stored procedures, scalar functions, TVFs, triggers, choosing the right object
- Reference: CREATE VIEW · Create indexed views · CREATE FUNCTION · Scalar UDF inlining · CREATE PROCEDURE
- Triggers: CREATE TRIGGER · inserted and deleted tables · DDL triggers · DISABLE TRIGGER · TRIGGER_NESTLEVEL

Chapter 6 — Advanced T-SQL, part 1: CTEs, window functions, subqueries, and errors

> **EXAM FOCUS**
> **Write advanced T-SQL code** — write common table expressions (CTEs) · write queries that include window functions · write correlated queries · implement error handling.

This chapter covers the query patterns that appear again and again inside other questions: a CTE that ranks rows, a window function that keeps every row, a correlated `EXISTS`, and a safe `TRY…CATCH`. It ends with **SARGability**, the rule behind many "why is this query scanning?" questions.

6.1 Common table expressions (CTEs)

A **CTE** is a named, temporary result set. You write it with `WITH name AS (query)` right before one statement, and then use it in that statement like a table. It exists only for that one statement.

```
WITH CategorySales AS (
    SELECT p.ProductCategoryID, SUM(sod.LineTotal) AS CategoryRevenue
    FROM SalesLT.Product AS p
    JOIN SalesLT.SalesOrderDetail AS sod ON p.ProductID = sod.ProductID
    GROUP BY p.ProductCategoryID
),
RankedCategories AS (                       -- a second CTE can use the first
    SELECT ProductCategoryID, CategoryRevenue,
           RANK() OVER (ORDER BY CategoryRevenue DESC) AS RevenueRank
    FROM CategorySales
)
SELECT * FROM RankedCategories WHERE RevenueRank <= 5;
```

Rules worth knowing:

- Several CTEs go in **one** `WITH`, separated by commas. A later CTE can use an earlier one, not the other way round.
- The statement **before** a CTE must end with a semicolon (the same rule as `THROW`).
- A CTE is **not materialized** (not stored): *"Each outer reference to the named result set requires the defined query to be re-executed."* If you use it many times, a temporary table may be faster.
- `ORDER BY` is not allowed inside a CTE (unless with `TOP` or `OFFSET/FETCH`).
- The outer query can only see the CTE's **columns**. In a question where a CTE builds `PhoneRaw`, the outer `WHERE` must use `p.PhoneRaw`, not the original JSON column.

Recursive CTEs walk hierarchies (employee → manager) or generate sequences. They have an **anchor** query (the starting rows), then `UNION ALL`, then a **recursive** query that joins back to the CTE.

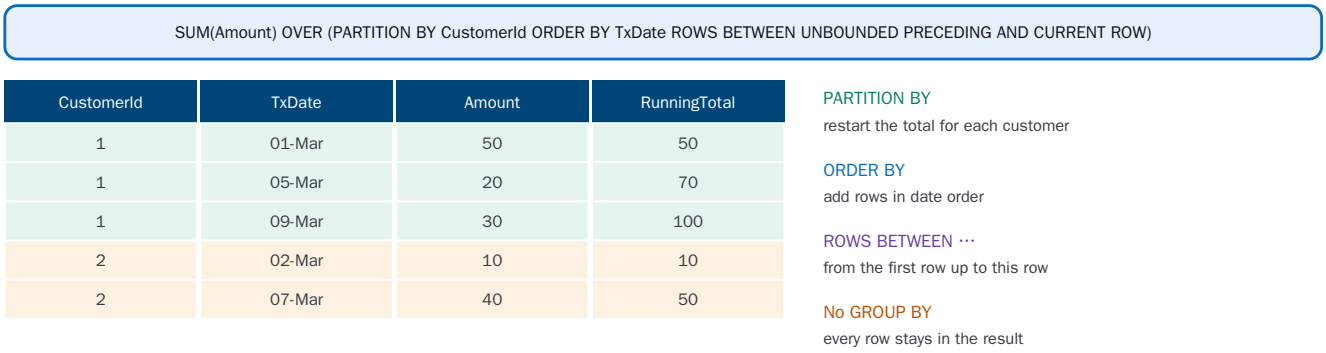
```
WITH EmployeeHierarchy AS (
    SELECT EmployeeID, ManagerID, 0 AS Level
    FROM HumanResources.Employee
    WHERE ManagerID IS NULL                        -- anchor: the top boss
    UNION ALL
    SELECT e.EmployeeID, e.ManagerID, eh.Level + 1
    FROM HumanResources.Employee AS e
    INNER JOIN EmployeeHierarchy AS eh ON e.ManagerID = eh.EmployeeID   -- recursive part
)
SELECT * FROM EmployeeHierarchy
OPTION (MAXRECURSION 200);                         -- hint goes on the OUTER statement
```

COMMON EXAM TRAP

`UNION ALL` is the **only** operator allowed between the anchor and the recursive member. The default recursion limit is **100**; `OPTION (MAXRECURSION 0)` removes the limit and the maximum value is 32,767. The recursive part cannot use `LEFT JOIN`, `TOP`, `GROUP BY`, `DISTINCT`, or subqueries.

6.2 Window functions

A **window function** calculates over a group of related rows (the *window*) but still returns **every** row. `GROUP BY` squashes rows together; a window function does not.



A running total per customer. The OVER clause has three parts: PARTITION BY (groups), ORDER BY (order inside a group), and the frame (which rows count).

The `OVER()` clause:

Part	What it does	If you leave it out
`PARTITION BY`	Splits rows into groups; the calculation restarts for each group	All rows are one group
`ORDER BY`	Sorts rows inside each group	For aggregates: the whole partition is the window
`ROWS` / `RANGE` frame	Which rows around the current row count	With `ORDER BY`, the default is `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`

Ranking functions — all need `ORDER BY`, none accept a frame, all return `bigint`:

Function	Ties	Example values
`ROW_NUMBER()`	Always unique numbers	1, 2, 3, 4
`RANK()`	Ties share a number, then a gap	1, 1, 3, 4
`DENSE_RANK()`	Ties share a number, no gap	1, 1, 2, 3
`NTILE(n)`	Splits rows into n buckets	1, 1, 2, 2

First row per group — rank inside a CTE, filter outside (you can't filter on a window function in the same `WHERE`):

```
WITH TicketRanks AS (
    SELECT CAST(t.CreatedAt AS date) AS TicketDate,
           JSON_VALUE(t.Payload, '$.customer.id') AS CustomerId,
           t.TicketId, t.CreatedAt,
           ROW_NUMBER() OVER (PARTITION BY CAST(t.CreatedAt AS date),
                                           JSON_VALUE(t.Payload, '$.customer.id')
                              ORDER BY t.CreatedAt ASC) AS rn
    FROM dbo.SupportTickets AS t
    WHERE t.CreatedAt >= DATEADD(day, -7, SYSUTCDATETIME())
)
SELECT TicketDate, CustomerId, TicketId, CreatedAt
FROM TicketRanks
WHERE rn = 1;                    -- exactly one row per customer per day
```

Running total — an aggregate with `OVER`:

```
SELECT CustomerId, TransactionDate, Amount,
       SUM(Amount) OVER (PARTITION BY CustomerId
                         ORDER BY TransactionDate, TransactionId      -- tie-breaker
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS RunningTotal
FROM dbo.Transactions;
```

Other useful window functions: `LAG(col, 1, 0)` and `LEAD(col, 1, 0)` read the previous or next row (here with 0 as the default at the edges); `AVG(col) OVER (ORDER BY d ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)` is a 3-row moving average; `FIRST_VALUE` / `LAST_VALUE` return the first or last value in the frame.

COMMON EXAM TRAP

Four classic window-function traps:

1. `DENSE_RANK` **or** `RANK` **for "exactly one row per group"** — ties get the same number, so two rows can both be 1. Use `ROW_NUMBER`.
2. `RANGE` **vs** `ROWS`**.** Without a frame, a running total uses `RANGE`, so rows with the same `ORDER BY` value are added together in one jump. Use `ROWS` plus a unique tie-breaker.
3. `LAST_VALUE` **without a frame** returns the *current* row's value, because the default frame stops at the current row. Add `ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING`.
4. `GROUP BY` **or** `HAVING` **inside** `OVER()` is invalid. `OVER` takes only `PARTITION BY`, `ORDER BY`, and a frame. Only `ROWS` (not `RANGE`) accepts numeric offsets like `2 PRECEDING`.

TIP

For speed, index the window: key columns = the `PARTITION BY` columns, then the `ORDER BY` columns, and `INCLUDE` the aggregated column — for example `CREATE INDEX IX ON Sales.SalesOrderHeader (CustomerID, OrderDate) INCLUDE (TotalDue);`.

6.3 Correlated subqueries and EXISTS

A subquery is **correlated** when it uses a column from the outer query. Think of it as running once for each outer row: "for *this* product, compare with the average price of *its* category".

```
-- Patients who had at least one procedure in the last 30 days
SELECT p.Name
FROM dbo.Patients AS p
WHERE EXISTS (SELECT 1
              FROM dbo.Procedures AS pr
              WHERE p.PatientId = pr.PatientId                      -- the correlation
                AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME()));

-- Products priced above their own category's average
SELECT p1.ProductID, p1.ListPrice
FROM SalesLT.Product AS p1
WHERE p1.ListPrice > (SELECT AVG(p2.ListPrice)
                      FROM SalesLT.Product AS p2
                      WHERE p2.ProductCategoryID = p1.ProductCategoryID);
```

Facts that decide questions:

- `EXISTS` *"Returns TRUE if a subquery contains any rows."* The select list doesn't matter (`SELECT 1`, `SELECT *`, even `SELECT NULL` all work). It can stop at the first match.
- Each patient is returned **once**, no matter how many procedures match — no `DISTINCT` needed.
- The correlation predicate (`p.PatientId = pr.PatientId`) belongs in the subquery's `WHERE`, not in a `HAVING` (`HAVING` filters groups after aggregation).
- `IN (correlated subquery)` returns the same rows, but `EXISTS` is the idiomatic choice for a correlated test.

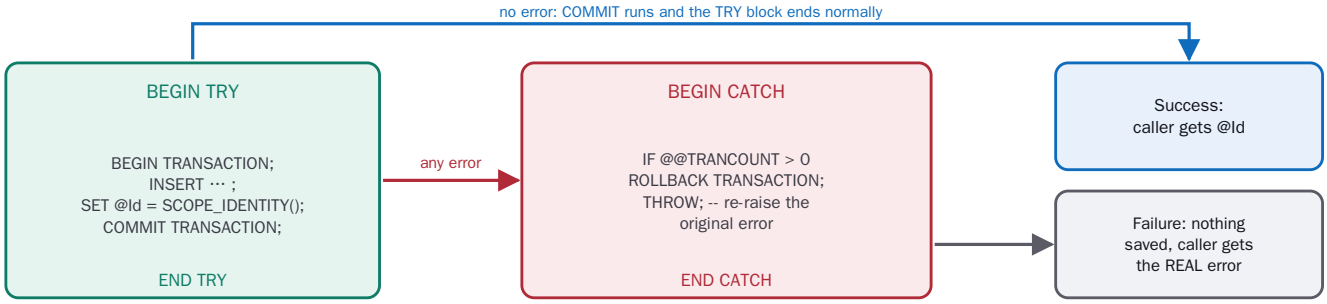
COMMON EXAM TRAP

`NOT IN` with NULLs. If the subquery can return a `NULL`, `NOT IN` returns **no rows at all**, because comparing with NULL is UNKNOWN. Use `NOT EXISTS` for "rows with no match".

KNOW THE DIFFERENCE

Correlated subquery vs JOIN vs window function. Use `EXISTS` / `NOT EXISTS` to test for related rows. Use a `JOIN` when you need columns from both tables. Use a window function for running totals, rankings, and previous/next row — Microsoft's training calls window functions *"More efficient than correlated subqueries for these patterns."*

6.4 Error handling with TRY…CATCH



The standard pattern: do the work in TRY; on any error, roll back only if a transaction is open, then re-raise the original error with THROW.

```
CREATE OR ALTER PROCEDURE dbo.usp_CreateOrder
    @CustomerID int, @OrderId int OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    SET XACT_ABORT ON;                    -- recommended: any run-time error dooms the
transaction
    BEGIN TRY
        BEGIN TRANSACTION;
        INSERT INTO dbo.Orders (CustomerID) VALUES (@CustomerID);
        SET @OrderId = SCOPE_IDENTITY();   -- the new ID from THIS scope
        COMMIT TRANSACTION;
    END TRY
    BEGIN CATCH
        IF @@TRANCOUNT > 0
            ROLLBACK TRANSACTION;          -- the semicolon here matters
        THROW;                             -- re-raise the original error to the caller
    END CATCH;
END;
```

What each piece does, in plain English:

Piece	Meaning
`BEGIN TRY … END TRY`	Run these statements; if one fails, jump to CATCH
`BEGIN CATCH … END CATCH`	Error handler. `ERROR_NUMBER()`, `ERROR_MESSAGE()`, etc. work only in here
`@@TRANCOUNT`	How many transactions are open. `> 0` means "there is something to roll back"
`XACT_STATE()`	`1` = open and committable, `0` = none, `-1` = doomed (can only roll back)
`THROW;`	With no arguments: re-raise the caught error, unchanged. **Only valid inside CATCH**
`THROW 50001, N'msg', 1;`	Raise a new error (number ≥ 50000, state 0–255, severity is always 16)
`SET XACT_ABORT ON`	Any run-time error rolls back the whole transaction
`SCOPE_IDENTITY()`	Last identity value inserted **in the same scope** (procedure, trigger, function, or batch)

COMMON EXAM TRAP

Error-handling details the exam checks:

- **Unguarded `ROLLBACK`.** If no transaction is open, `ROLLBACK TRANSACTION` fails with error 3903 (*"The ROLLBACK TRANSACTION request has no corresponding BEGIN TRANSACTION"*), hiding the real error. Always guard it with `IF @@TRANCOUNT > 0`.
- **Semicolon before THROW.** *"The statement before the THROW statement must be followed by the semicolon (;) statement terminator."*
- **Swallowed errors.** *"Errors trapped by a CATCH block aren't returned to the calling application"* unless you re-raise them. A CATCH block without `THROW` hides failures from the app.
- **`@@IDENTITY` vs `SCOPE_IDENTITY()`.** `@@IDENTITY` is not limited to your scope, so it returns a trigger's identity value if an insert trigger writes to another table.
- **`RAISERROR` vs `THROW`.** *"New applications should use THROW instead of RAISERROR."* `RAISERROR` does not honour `SET XACT_ABORT`.
- **What TRY…CATCH can't catch** in the same batch: compile (syntax) errors and object-name-resolution errors, plus warnings (severity 10 or lower). It also can't be used inside a user-defined function.

6.5 Writing SARGable predicates

A predicate is **SARGable** (*Search ARGument-able*) when *"SQL Server engine can use an index seek to speed up the execution of the query."* The rule:

> *"Applying any function or computation on the column(s) in the search predicate generally makes the query non-sargable."*

Non-SARGable (scans)	SARGable rewrite (seeks)
`WHERE CAST(CreateDate AS date) = @d`	`WHERE CreateDate >= @d AND CreateDate < DATEADD(day, 1, @d)`
`WHERE YEAR(OrderDate) = 2025`	`WHERE OrderDate >= '20250101' AND OrderDate < '20260101'`
`WHERE CONVERT(char(10), CreateDate, 121) = …`	Same half-open range on the bare column
`WHERE LEFT(ProductNumber, 3) = 'HN-'`	`WHERE ProductNumber LIKE 'HN-%'`
`WHERE UnitPrice * 0.10 > 300`	`WHERE UnitPrice > 300 / 0.10`
`WHERE CreateDate LIKE '2025-01-15%'`	Half-open range (`LIKE` converts every date to text first)

EXAMPLE

Why "half-open"? A range that includes the start and excludes the end — `>= midnight AND < next midnight` — catches every time on that day, even 23:59:59.9999999, whatever the column's precision. `BETWEEN '2025-01-15' AND '2025-01-15'` only catches exactly midnight, and `BETWEEN … '23:59:59'` misses fractional seconds.

Other things that block seeks: `ISNULL()` / `COALESCE()` around the column, and **implicit conversions** when the parameter and column types don't match (*"A data type mismatch forces the engine to convert values at runtime, which can turn an index seek into a scan."*). If you truly can't rewrite a query, an **indexed computed column** on the expression restores seeks.

SCENARIO PATTERN

"Modify the predicate so SQL Server can use an index seek on CreateDate for a single day" → `WHERE CreateDate >= @OrderDate AND CreateDate < DATEADD(day, 1, @OrderDate)`. Every option that wraps `CreateDate` in `CAST`, `CONVERT`, `LEFT`, or `LIKE` is wrong.

6.6 Chapter 6 self-check

1. Where does `OPTION (MAXRECURSION 500)` go in a recursive CTE query?
2. You need the top-selling product per category, and ties must not produce two rows. Which ranking function?
3. Why does `SUM(x) OVER (ORDER BY OrderDate)` sometimes jump by more than one row's value?
4. Rewrite `SELECT e.EmployeeId FROM dbo.Emp AS e WHERE e.EmployeeId NOT IN (SELECT ManagerId FROM dbo.Emp)` so it still works when `ManagerId` contains NULLs.

5. What two things must happen in the CATCH block so that the transaction is undone and the caller sees the original error?
6. Is `WHERE ISNULL(Discount, 0) = 0` SARGable?

Answers: 1) On the outer statement, after the final `SELECT`. 2) `ROW_NUMBER()`. 3) With no frame the default is `RANGE`, which adds all rows with the same `OrderDate` together — use `ROWS` and a tie-breaker. 4) `WHERE NOT EXISTS (SELECT 1 FROM dbo.Emp m WHERE m.ManagerId = e.EmployeeId)`. 5) `IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;` then `THROW;`. 6) No — the column is wrapped in a function; use `WHERE Discount = 0 OR Discount IS NULL`.

6.7 Learn more

- Training: Write advanced T-SQL code — units on CTEs, window functions, correlated subqueries, and TRY…CATCH
- Reference: WITH common_table_expression · OVER clause · ROW_NUMBER · EXISTS · Subqueries
- Reference: TRY…CATCH · THROW · SET XACT_ABORT · XACT_STATE · SCOPE_IDENTITY
- Troubleshooting: SARGability in the high-CPU guide

Chapter 7 — Advanced T-SQL, part 2: JSON, regular expressions, fuzzy matching, and graphs

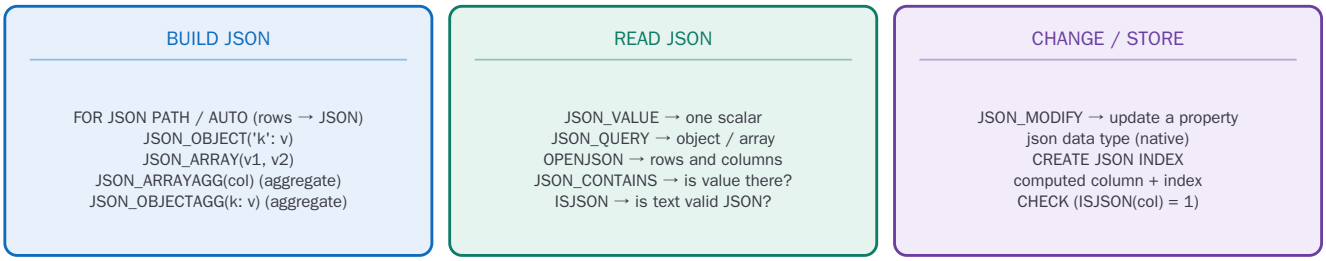
EXAM FOCUS

Write advanced T-SQL code — write queries that include JSON functions, such as JSON_OBJECT, JSON_ARRAY, JSON_ARRAYAGG, JSON_CONTAINS, OPENJSON, and JSON_VALUE · write queries that include regular expressions, such as REGEXP_LIKE, REGEXP_REPLACE, REGEXP_SUBSTR, REGEXP_INSTR, REGEXP_COUNT, REGEXP_MATCHES, and REGEXP_SPLIT_TO_TABLE · write queries that include fuzzy string matching functions, such as EDIT_DISTANCE, EDIT_DISTANCE_SIMILARITY, and JARO_WINKLER_DISTANCE · write graph queries that use the MATCH operator.

7.1 JSON functions

JSON (JavaScript Object Notation) is text that describes data with keys and values, for example `{"customer": {"id": 42, "email": "a@contoso.com"}, "items": [{"product": "Bike", "qty": 1}]}`. Curly braces `{ }` are an **object**; square brackets `[ ]` are an **array**; a single number or string is a **scalar**.

A **path** says where to look: `$` is the whole document, `$.customer.id` is a property inside an object, and `$.items[0].product` is the first array element (arrays start at 0). Each path has a mode: **lax** (the default) returns `NULL` when something is wrong; **strict** raises an error.



Every JSON function fits one of three jobs: build JSON, read JSON, or change/store JSON.

Reading JSON — the three functions you must not mix up:

Function	Returns	Use it for	Wrong-type behaviour (lax mode)
`JSON_VALUE(doc, '$.path')`	One **scalar**, `nvarchar(4000)` by default	A string, number, or true/false	Object or array → `NULL`
`JSON_QUERY(doc, '$.path')`	An **object or array**, as JSON text (`nvarchar(max)`)	A nested object, or an array such as an embedding	Scalar → `NULL`
`OPENJSON(doc [, '$.path']) [WITH (…)]`	A **table** (rows and columns)	Shredding arrays into rows, long values	— (it's table-valued: use in `FROM` / `APPLY`)

```
SELECT JSON_VALUE(@json, '$.customer.email') AS Email,       -- scalar
       JSON_QUERY(@json, '$.items')          AS ItemsArray;  -- array, still JSON

-- Shred an array into typed rows
SELECT o.OrderID, i.ProductName, i.Quantity
FROM dbo.Orders AS o
CROSS APPLY OPENJSON(o.OrderDetails)
     WITH (ProductName nvarchar(100) '$.product',
           Quantity    int           '$.qty') AS i;
```

COMMON EXAM TRAP

- A wrong-function choice usually **fails silently**: `JSON_VALUE` on an object, or `JSON_QUERY` on a scalar, just returns `NULL` in lax mode.
- `JSON_VALUE` returns `NULL` (lax) for values longer than 4,000 characters. Use `OPENJSON … WITH (col nvarchar(max) '$.x')` for long text, such as a long model answer.
- `OPENJSON` is **table-valued**. It can't sit in a `SELECT` list; it goes in `FROM` or `CROSS APPLY` / `OUTER APPLY` (`OUTER APPLY` keeps rows whose JSON is empty).
- `JSON_QUERY` takes no third argument.

Testing JSON:

- `ISJSON(text)` → 1 if the text is valid JSON (object or array), 0 if not, never an error. Use it in a `CHECK` constraint. An optional second argument (`VALUE`, `ARRAY`, `OBJECT`, `SCALAR`) tests a specific type.
- `JSON_PATH_EXISTS(doc, '$.path')` → 1 if the path exists.
- `JSON_CONTAINS(doc, value, '$.path')` → 1 if the value is found at the path (SQL Server 2025, Azure SQL, SQL database in Fabric). The search value is a normal SQL value, and an array path needs a wildcard:

```
DECLARE @j json = '{"tags": ["sql", "ai"], "flags": [1, 3, false]}';
SELECT JSON_CONTAINS(@j, 'sql', '$.tags[*]');   -- 1
```

Changing JSON: `JSON_MODIFY(doc, '$.path', newValue)` updates a property and returns the new JSON. In lax mode a `NULL` new value **deletes** the key, and `'append $.skills'` adds to an array. For the native `json` type, `UPDATE t SET doc.modify('$.a', 14859)` changes the document in place.

Building JSON from values:

```
SELECT JSON_OBJECT('id': 42, 'name': 'Bike');               -- {"id":42,"name":"Bike"}
SELECT JSON_ARRAY('a', 1, NULL, 2);                         -- ["a",1,2]   (NULLs dropped
by default)

-- One JSON array per category, built from many rows (SQL Server 2025 / Azure SQL / Fabric)
SELECT pc.Name,
       JSON_ARRAYAGG(JSON_OBJECT('id': p.ProductID, 'name': p.Name) ORDER BY p.ProductID) AS
Products
FROM SalesLT.ProductCategory AS pc
JOIN SalesLT.Product AS p ON pc.ProductCategoryID = p.ProductCategoryID
GROUP BY pc.ProductCategoryID, pc.Name;
```

> **PLAIN ENGLISH**
>
> `JSON_OBJECT` and `JSON_ARRAY` build JSON from the values in **one row**. `JSON_OBJECTAGG` and `JSON_ARRAYAGG` are **aggregates**: they collect values from **many rows** (with `GROUP BY`) into one object or array. Null handling differs: `JSON_OBJECT` / `JSON_OBJECTAGG` keep nulls by default (`NULL ON NULL`); `JSON_ARRAY` / `JSON_ARRAYAGG` drop them by default (`ABSENT ON NULL`).

Turning a whole result set into JSON — `FOR JSON`:

```
-- One JSON document per order, each with a nested "lines" array
SELECT o.OrderHeaderId AS orderHeaderId,
       o.OrderNumber   AS orderNumber,
       (SELECT l.LineNumber AS lineNumber, l.ProductId AS productId, l.Qty AS qty
        FROM Sales.OrderLines AS l
        WHERE l.OrderHeaderId = o.OrderHeaderId
        FOR JSON PATH) AS lines                       -- nested array
FROM Sales.Orders AS o
FOR JSON PATH;                                        -- outer array of orders
```

Option	Effect
`FOR JSON PATH`	You control the shape with column aliases (`AS 'product.id'` makes nesting)
`FOR JSON AUTO`	Shape follows the tables in the query; **needs a `FROM` clause**
`WITHOUT_ARRAY_WRAPPER`	Removes the outer `[ ]` — use with a **single-row** result to get one object
`INCLUDE_NULL_VALUES`	Keep properties whose value is NULL (they are left out by default)
`ROOT('name')`	Wrap the output in a named root property

TIP

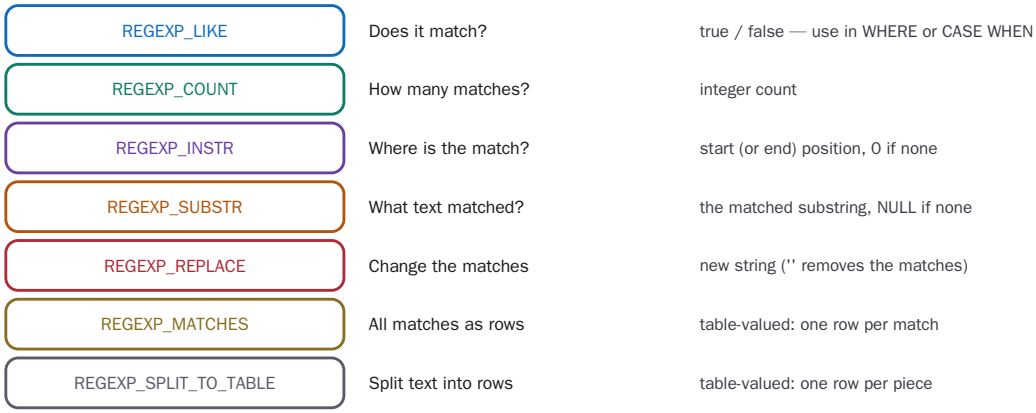
If a column already contains JSON text and you include it in `FOR JSON`, wrap it as `JSON_QUERY(col)` so it is embedded as JSON instead of being escaped as a string.

SCENARIO PATTERN

"Return one JSON document per order with a nested array of order lines" → `FOR JSON PATH` on the outer query **and** `FOR JSON PATH` in a correlated subquery aliased `lines`. `JSON_QUERY`, `JSON_MODIFY`, and `OPENJSON` don't build JSON from rows. "Extract a single scalar value from the JSON text" → `JSON_VALUE`.

7.2 Regular expressions

`LIKE` only understands `%` and `_`. A **regular expression** (regex) describes richer patterns. SQL Server 2025, Azure SQL Database, Azure SQL Managed Instance (2025 or Always-up-to-date policy), and SQL database in Fabric have seven regex functions, based on the RE2 regex library.



Seven functions, seven questions. Learn the question each one answers.

Regex basics you need for the exam:

Pattern	Meaning	Example
`\d` / `[0-9]`	One digit	`\d{3}` = three digits
`\D` / `[^0-9]`	One NON-digit (`^` inside `[ ]` means "not")	`[^0-9]` matches `+`, `-`, space
`.`	Any character	`.*` = anything, any length
`+` `*` `?`	One or more · zero or more · zero or one	`\s+` = one or more spaces
`{n}` `{n,m}`	Exactly n · between n and m	`\d{3,4}`
`^` `$`	Start · end of the string	`^A.*Y$`
`( )`	Group (capture)	`@(.+)$` captures the domain
`\`	Escape a special character	`\+` = a literal plus sign

```
-- Does it match? (Boolean - use directly in WHERE / CASE WHEN / CHECK)
SELECT * FROM dbo.Employees WHERE REGEXP_LIKE(FirstName, '^A.*Y$', 'i');

-- Remove email sub-addressing: user1+promo@contoso.com -> user1@contoso.com
SELECT REGEXP_REPLACE(JSON_VALUE(Payload, '$.customer_email'), '\+.*@', '@') FROM
dbo.Customers;

-- Keep only the digits (default occurrence 0 = replace EVERY match)
SELECT REGEXP_REPLACE(Phone, '[^0-9]', '') FROM dbo.Customers;

-- First phone number in a message (NULL if none), its digits, and a status
SELECT MessageID,
       REGEXP_SUBSTR(MessageRaw, '\d{3}[)\-\s]*\d{3}[\-\s]*\d{4}') AS RawNumber,
       REGEXP_REPLACE(REGEXP_SUBSTR(MessageRaw, '\d{3}[)\-\s]*\d{3}[\-\s]*\d{4}'), '\D', '')
AS DigitsOnly,
       CASE WHEN REGEXP_LIKE(MessageRaw, '\d{3}[)\-\s]*\d{3}[\-\s]*\d{4}') THEN 'Valid' ELSE
'Missing' END AS PhoneStatus
FROM dbo.CustomerMessages;

-- Capture group 1 = the email domain
SELECT REGEXP_SUBSTR(Email, '@(.+)$', 1, 1, 'i', 1) AS Domain FROM dbo.Customers;

-- Table-valued functions
SELECT * FROM REGEXP_MATCHES('Learning #AzureSQL #AzureSQLDB', '#([A-Za-z0-9_]+)');
SELECT p.ProductID, t.value FROM dbo.Products AS p CROSS APPLY REGEXP_SPLIT_TO_TABLE(p.Tags,
',\s*') AS t;
```

What each function returns when nothing matches (a favourite exam detail):

Function	No match returns	Default occurrence
`REGEXP_LIKE`	false	—
`REGEXP_SUBSTR`	`NULL`	1 (first match)
`REGEXP_INSTR`	`0`	1
`REGEXP_COUNT`	`0`	—
`REGEXP_REPLACE`	the original string	**0 = all matches**
`REGEXP_MATCHES`	no rows	—
`REGEXP_SPLIT_TO_TABLE`	the whole string as one row	—

Flags (last optional argument): `c` = case-sensitive (**the default**), `i` = case-insensitive, `m` = multi-line, `s` = `.` also matches newlines.

COMMON EXAM TRAP

- **Escape metacharacters.** `+ * ? ( ) |` and `.` are special. `'\+.*$'` replaced with `''` removes the tag **and the domain** (`user1`); `'\+.*@'` replaced with `'@'` removes just the tag.
- **`REGEXP_COUNT(...) = 1`** means "exactly one match", and **`REGEXP_INSTR(...) = 1`** means "the match starts at position 1". Neither means "a match exists" — that is `REGEXP_LIKE`.
- **`REGEXP_SUBSTR(x, '[0-9]+')`** returns only the **first** run of digits, not all digits. To strip everything else, use `REGEXP_REPLACE(x, '[^0-9]', '')`.
- **Compatibility level.** `REGEXP_LIKE`, `REGEXP_MATCHES`, and `REGEXP_SPLIT_TO_TABLE` need database compatibility level **170**; the other scalar functions work at any level.
- **Large inputs.** `varchar(max)` / `nvarchar(max)` inputs are supported up to **2 MB**. Older documentation (before May 2026) said `REGEXP_REPLACE`, `REGEXP_SUBSTR`, `REGEXP_MATCHES`, and `REGEXP_SPLIT_TO_TABLE` did not accept LOB types at all. So if an exam item asks how to use `REGEXP_SUBSTR` on an `nvarchar(max)` column whose data you need is near the start, **`CAST(col AS nvarchar(4000))`** first is the expected answer — it also keeps the input under the limit.
- `STRING_SIMILARITY` is not a T-SQL function.
- `REGEXP_LIKE` is SARGable (can use an index seek) only when the pattern starts with `^`.
- Backreferences in `REGEXP_REPLACE` are written `\1` … `\9` in the reference documentation (the training's `$1` style isn't documented).
- Exam items may print `CASE WHEN REGEXP_LIKE(…) = 1`. Still choose `REGEXP_LIKE` — `REGEXP_COUNT` and `REGEXP_INSTR` don't mean "a match exists".

7.3 Fuzzy string matching

Fuzzy matching scores how similar two **whole strings** are — useful for typos and spelling variants ("Colour" vs "Color", "Jon Smyth" vs "John Smith").

Function	Returns	Direction	Example: `'Colour'`, `'Color'`
`EDIT_DISTANCE(s1, s2 [, max])`	int: number of insertions, deletions, substitutions, and transpositions	**Lower** = more similar (0 = identical)	1
`EDIT_DISTANCE_SIMILARITY(s1, s2)`	int **0–100**	**Higher** = more similar (100 = identical)	83
`JARO_WINKLER_DISTANCE(s1, s2)`	float	**Lower** = more similar (0 = identical)	≈ 0.033
`JARO_WINKLER_SIMILARITY(s1, s2)`	int **0–100**	**Higher** = more similar	≈ 97

```
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;   -- SQL Server 2025 only

SELECT f.FeedbackId,
       JSON_VALUE(f.FeedbackJson, '$.text') AS FeedbackText,
       EDIT_DISTANCE_SIMILARITY(JSON_VALUE(f.FeedbackJson, '$.text'),
@KnownIssueDescription) AS SimilarityScore
FROM dbo.CustomerFeedback AS f
WHERE CONTAINS(f.FeedbackJson, @Keyword)      -- full-text keyword filter, not fuzzy
matching
ORDER BY SimilarityScore DESC;
```

PLAIN ENGLISH

`EDIT_DISTANCE_SIMILARITY` is calculated as `(1 − edit_distance ÷ length of the longer string) × 100`. **Jaro-Winkler** is tuned for short strings like names and gives extra credit when the beginnings match.

COMMON EXAM TRAP

- These functions compare **entire strings**. `EDIT_DISTANCE(long paragraph, 'brake') < 3` is almost never true, so it is the wrong tool for "the text **contains** a keyword". With a full-text index, use `CONTAINS`; otherwise `LIKE` or `REGEXP_LIKE`.
- The reference implements Damerau-Levenshtein (optimal string alignment), so swapping two adjacent letters costs **1** edit (the training's "database" → "databaes" = 2 example conflicts with this; trust the reference).
- **Distance vs similarity**: `*_DISTANCE` is lower-is-better; `*_SIMILARITY` is higher-is-better. (The training unit describes `JARO_WINKLER_DISTANCE` as if 1 meant identical; the reference page shows identical strings return 0. Trust the reference.)
- They are **preview** features: on SQL Server 2025 turn on `PREVIEW_FEATURES`. Inputs can't be `varchar(max)` / `nvarchar(max)`, and non-binary `SQL_*` collations aren't supported.
- They are CPU-heavy and scan. Narrow the rows first with a SARGable filter (`WHERE LastName LIKE 'Sm%'`), then fuzzy-match the small set.

7.4 Graph queries with MATCH

SQL Graph stores **things** in node tables and **relationships** in edge tables (see 3.4 for creating them). You query relationships with `MATCH` in the `WHERE` clause, using arrow "ASCII art":

```
-- People exactly two "Knows" hops away from a start person
SELECT p3.PersonID, p3.DisplayName
FROM dbo.Person AS p1, dbo.Knows AS k1, dbo.Person AS p2,     -- comma-separated FROM list
     dbo.Knows AS k2, dbo.Person AS p3
WHERE MATCH(p1-(k1)->p2-(k2)->p3)                              -- one MATCH, two hops
  AND p1.PersonID = @StartPersonId;
```

The rules behind every graph question:

- **Node aliases can repeat; edge aliases can't.** *"An edge name can't be repeated inside MATCH."* Two hops need two edge aliases (`k1` , `k2`).
- **Every edge needs an explicit direction** (`->` or `<-`). `p3<-(k2)-p2<-(k1)-p1` is the same path written right to left.
- **No `JOIN` or `APPLY` with MATCH.** List graph tables comma-separated in `FROM` ; a `JOIN … ON 1 = 1` gives error 13920 (*"JOIN and APPLY are not supported with MATCH clauses"*).
- `MATCH` can be combined with other conditions using **AND** only — not `OR` or `NOT` .
- For a variable number of hops or the "shortest route", use `SHORTEST_PATH` (SQL Server 2019+) inside `MATCH` , with `FOR PATH` on the participating tables. `+` means one or more hops; `{1,n}` means 1 to n hops, e.g. `MATCH(SHORTEST_PATH(Person1(-(fo)->Person2){1,3}))` .

> **PLAIN ENGLISH**
>
> Behind the scenes, every node row gets a hidden `$node_id` , and every edge row stores `$from_id` and `$to_id` pointing to the two nodes it connects. You insert an edge by selecting the `$node_id` values of the two nodes.

7.5 Chapter 7 self-check

1. `SELECT JSON_VALUE(@j, '$.items')` returns NULL although `items` exists. Why, and what should you use?
2. Which option turns a single-row `FOR JSON PATH` result into one object instead of an array?
3. Which regex function answers "does this text contain a postcode?", and what type does it return?
4. `REGEXP_SUBSTR(Notes, '\d{5}')` finds nothing. What does it return? What would `REGEXP_INSTR` return?
5. You need a 0–100 similarity score for misspelled surnames. Name two functions that return one.
6. Why does `FROM Person p1 JOIN Knows k ON 1=1 … WHERE MATCH(...)` fail?

Answers: 1) `items` is an array; `JSON_VALUE` only returns scalars — use `JSON_QUERY` . 2) `WITHOUT_ARRAY_WRAPPER` . 3) `REGEXP_LIKE` , Boolean (true/false). 4) `NULL` ; `0` . 5) `EDIT_DISTANCE_SIMILARITY` and `JARO_WINKLER_SIMILARITY` . 6) `JOIN` / `APPLY` can't be used with tables in a `MATCH` clause (error 13920); use a comma-separated `FROM` .

7.6 Learn more

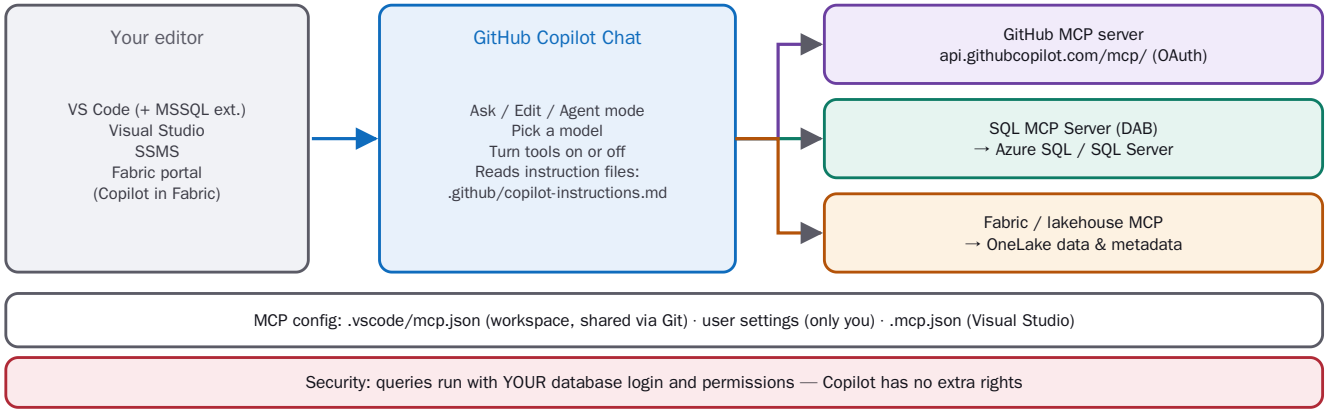
- Training: Process JSON data · Match patterns with regular expressions · Fuzzy string functions · Graph queries
- JSON: JSON_VALUE · JSON_QUERY · OPENJSON · JSON_CONTAINS · JSON_ARRAYAGG · FOR JSON · WITHOUT_ARRAY_WRAPPER
- Regex: Regular expressions overview · REGEXP_LIKE · REGEXP_REPLACE · REGEXP_SUBSTR
- Fuzzy matching: overview · EDIT_DISTANCE · EDIT_DISTANCE_SIMILARITY · JARO_WINKLER_DISTANCE
- Graph: SQL Graph overview · MATCH · SHORTEST_PATH

Chapter 8 — AI-assisted development: GitHub Copilot, Copilot in Fabric, and MCP

> **EXAM FOCUS**
>
> **Design and implement SQL solutions by using AI-assisted tools** — interpret security impact of using AI-assisted tools · enable GitHub Copilot and Copilot in Fabric · configure model and Model Context Protocol (MCP) tool options in a GitHub Copilot or Copilot in Fabric chat session · create and configure GitHub Copilot instruction files · connect to MCP server endpoints, including Microsoft SQL Server and Fabric lakehouse.

This is a short sub-area of the outline but it produces many quick, factual questions: *which file*, *which setting*, *which scope*, *whose permissions*. Learn the file names and the one big security rule, and these become easy points.

8.1 The tools



Your editor hosts GitHub Copilot Chat. Instruction files shape what it writes; MCP servers give it live tools and data. Whatever it runs, it runs as you.

Tool	Where it lives	What it gives you
GitHub Copilot in SSMS	SQL Server Management Studio 22 (installed with the *AI Assistance* workload)	Chat with Ask and Agent modes, code completions, slash commands (`/explain`, `/fix`, `/optimize`, `/doc`), MCP in Agent mode
GitHub Copilot + MSSQL extension	Visual Studio Code	The schema-aware `@mssql` chat participant, agent-mode tools (`list_tables`, `show_schema`, `run_query`, …), custom instructions
GitHub Copilot in Visual Studio	Visual Studio 2022/2026	Chat, agent mode, MCP servers via `.mcp.json`
Copilot in Fabric	The Fabric portal, including the SQL database query editor	Natural language to SQL, code completion, **Fix** and **Explain** quick actions, a chat pane

PLAIN ENGLISH

Model Context Protocol (MCP) is an open standard that lets an AI assistant call outside **tools** and read outside **data**. The AI app is the *host*, it uses an MCP *client*, and each data source or tool is an MCP *server*. Microsoft's training puts it this way: *"Instead of the AI assistant only seeing the code in your editor, MCP enables it to query your actual database schema, sample data, and metadata in real time."*

Ask mode answers questions. **Agent mode** can plan steps and call tools (including MCP tools) for you, asking your approval for each one.

TIP

In VS Code, the MSSQL extension's inline completions (grey "ghost text" in a `.sql` file) do **not** see your connected database schema. For schema-aware SQL in VS Code, use the `@mssql` chat participant or agent mode.

8.2 The security impact of AI-assisted tools

The one rule to remember: the AI runs as you.

> *"All queries that GitHub Copilot in SSMS generates and executes, in both Ask mode and Agent mode, run under the user or login you used to connect to the database."*
>
> *"Copilot has no separate permissions and no elevated access. If your user or login can't read a table, Copilot can't read it either."*

The VS Code MSSQL extension works the same way: *"All actions use the same connection context and credentials as the MSSQL extension. Agent mode doesn't introduce another authentication or permission changes."* So **SQL permissions** — roles, `GRANT` / `DENY`, row-level security, masking — are the real boundary. Not Azure RBAC, not GitHub organisation settings, not instruction files.

Statement you may see	True or false
"Copilot runs queries in an isolated read-only sandbox"	**False.** It runs against the connected database under your login
"Copilot filters out rows you aren't allowed to see on the client"	**False.** Enforcement happens in the database engine
"Copilot has its own RLS policies"	**False.** RLS is evaluated against *your* session
"Agent-mode approval prompts are a security boundary"	**False.** *"The actual security boundary is SQL Server's permission enforcement. Apply the principle of least-privilege."*
"A DBA can make SSMS Copilot run as a lower-privileged user"	**True.** A database-level extended property named `CONSTITUTION.md` with `agentExecuteAsUser: <user>` makes SSMS use `EXECUTE AS`; the signed-in user needs `IMPERSONATE` on that user

What gets sent to the model. Your prompt, the code around it, and **schema** details are sent to the AI service. Table *contents* are not sent unless you paste them or ask for them. Good habits from the training:

- *"Never paste credentials into prompts."* Use environment variables and secret stores.
- Don't paste rows containing **PII** (personally identifiable information, such as names and emails), and don't paste query result sets. Ask Copilot to write the query from the **schema only**.
- Don't give Copilot a connection string — it is a secret.
- For MCP, use dedicated accounts with read-only access when the AI only needs schema.
- **Review generated code** before running or committing it: over-broad `GRANT`s, dynamic SQL without parameters, and `SELECT *`.

> **SCENARIO PATTERN**
> "Policy forbids sharing PII, secrets, and result sets with any AI service; generate a procedure that joins Orders and Customers" → **ask Copilot using schema details only**. Pasting sample rows, giving a result set, or providing the connection string all break the policy.

8.3 Enabling GitHub Copilot and Copilot in Fabric

GitHub Copilot in SSMS. You need SSMS 22 and a GitHub account with Copilot access (Copilot Free works, with monthly limits). Open the **Visual Studio Installer**, choose **Modify** on SSMS, tick the **AI Assistance** workload, and select **Modify**. Then sign in from the Copilot badge and open **View > GitHub Copilot Chat**. Agent mode and MCP need SSMS 22.7 or later. Administrators can turn Copilot or Agent mode off with group policy (ADMX) settings.

GitHub Copilot in VS Code. Sign in to GitHub from the Copilot icon in the Status Bar (older guidance: install the *GitHub Copilot* and *GitHub Copilot Chat* extensions). Install the **SQL Server (mssql)** extension, connect to a database, then right-click it and choose **Chat with this database** (or type `@mssql` in chat).

Copilot in Fabric. Copilot needs a **paid Fabric capacity (F2 or higher)** or Power BI Premium (P1 or higher); it is **not** supported on trial capacities. It is **on by default** for tenants with paid capacities; admins control it (scope it to security groups, turn it off, or delegate it to capacity admins) with **tenant settings** in the Fabric admin portal, chiefly *"Users can use Copilot and other features powered by Azure OpenAI"*. If your capacity is outside the US or EU data boundary, an admin must also allow *"Data sent to Azure OpenAI can be processed outside your capacity's geographic region, compliance boundary, or national cloud instance."* Settings can be scoped to security groups or delegated to capacity admins. In a SQL database, select **Copilot** on the ribbon to open the chat pane (modes: *Read-only* or *Read and write with approval*); press Tab to accept a code completion; use **Fix** and **Explain** on a query.

> **PLAIN ENGLISH**
>
> Copilot in Fabric works on items that live **in Fabric** (and a Fabric data agent can expose them to other tools through MCP, see 8.6). If the data sits in an Azure SQL Database, it first has to be made available in Fabric — for example with Fabric mirroring.

8.4 Choosing models and MCP tools in a chat session

- **Model picker.** Every client has a model picker in the chat input (VS Code also has **Auto**). Use more capable models for complex design and tuning, faster ones for routine completions — *"Start with defaults."* In Copilot Business/Enterprise, admins control which models are allowed by policy.
- **MCP needs Agent mode.** *"You must be in Agent mode to use MCP. Ask mode doesn't support MCP."* (SSMS — the training says the same for VS Code.)
- **Tools are off until you enable them.** In SSMS and Visual Studio, *"MCP tools are disabled by default after you add a server. You must manually enable each tool in the Tools list."* In VS Code you choose tools with **Configure Tools** in the chat input.
- **Approvals.** Each tool call asks for approval: allow once, for this session, for this workspace/solution, or always.
- **SSMS Agent mode** is **read-only** by default and ships a built-in local MCP server (`sql-tools`). Even when set to read-write, writes still fail if your login lacks permission. Cancelling a request doesn't roll back queries you already approved.

8.5 GitHub Copilot instruction files

Instruction files are Markdown files that Copilot adds to every relevant request automatically, so you don't repeat your team's rules ("schema-qualify every object", "use TRY…CATCH", "no `SELECT *`") in each prompt.

File	Where	Who it applies to
Repository-wide instructions	`.github/copilot-instructions.md` (in the `.github` folder at the repository root)	Everyone working in the repo — committed and reviewed like code
Path-specific instructions	`.github/instructions/NAME.instructions.md` with front matter `applyTo: "**/*.sql"`	Only matching files
Agent instructions	`AGENTS.md` (repository root)	Agents that read the cross-tool format
Prompt files	`.github/prompts/NAME.prompt.md`	Reusable task templates you run on demand (`/NAME` in chat; the training unit says `#`)
Personal instructions	Your user profile (VS Code: `~/.copilot/copilot-instructions.md`)	Only you
Organization instructions	GitHub organisation settings (Copilot Business/Enterprise)	Every repo in the organisation
SSMS database instructions	Extended property `AGENTS.md` on a database object; `CONSTITUTION.md` at database level	Every SSMS Copilot user of that database

```
---
applyTo: "**/*.sql"
---
# T-SQL conventions
- Always schema-qualify objects: dbo.TableName
- Never use SELECT * - always name columns
- Wrap data changes in TRY...CATCH with IF @@TRANCOUNT > 0 ROLLBACK; THROW;
- Primary keys: PK_TableName; foreign keys: FK_Child_Parent
```

Two settings to know:

- `github.copilot.chat.codeGeneration.useInstructionFiles` (default **on**) — *"Automatically add custom instructions from .github/copilot-instructions.md to chat requests."* Turning it **off in your User settings** stops Copilot using the repository file **for you only**: *"This applies to your own use of Copilot Chat and does not affect other users."*
- Old **settings-based** instructions (instructions typed into `settings.json`) are gone: *"Settings-based code generation and test generation instructions are deprecated as of VS Code 1.102. Use file-based instructions instead."*

> PLAIN ENGLISH
>
> VS Code has two settings scopes. **User** settings apply to every project *you* open, on your machine. **Workspace** settings live in the project's `.vscode` folder and are usually committed, so they reach your teammates.

Precedence. All applicable instructions are sent together. When they conflict, GitHub documents this order for Copilot Chat on GitHub.com, highest first:

1. **Personal** instructions
2. **Repository** instructions — path-specific, then repository-wide (`.github/copilot-instructions.md`), then agent instructions (`AGENTS.md`)
3. **Organization** instructions

(VS Code and Visual Studio simply combine the files without a formal order, so the practical advice is: don't write contradictory instructions.)

COMMON EXAM TRAP

- **Disable repo instructions just for yourself** → clear *Code Generation: Use Instruction Files* in your **User** settings. Deleting `.github/copilot-instructions.md` or changing **workspace** settings affects everyone; a `--debug` flag changes nothing.
- **Team coding standards** → `.github/copilot-instructions.md`. Not `.vscode/settings.json` (settings-based instructions are deprecated) and not a file in your user profile (not shared).
- **Which wins?** → **personal** > repository > organization. Organization instructions are the *lowest*.
- Path-specific files must end in `.instructions.md` and live under `.github/instructions`.

8.6 Connecting to MCP servers

Where each client reads MCP configuration:

Client	File	Notes
VS Code	`.vscode/mcp.json` (workspace — shared via Git) or your user-profile `mcp.json` (only you)	Created with **MCP: Add Server**. Newer VS Code versions can also use a workspace-root `.mcp.json` and label the older files as legacy, but they still work
Visual Studio	`%USERPROFILE%\.mcp.json` (you, every solution) · `<SOLUTIONDIR>\.vs\mcp.json` (you, one solution) · `<SOLUTIONDIR>\.mcp.json` (*"track in source control"*) · also reads `.vscode\mcp.json`	OAuth through the **Authentication Required** CodeLens in the file
SSMS 22.7+	`%USERPROFILE%\.mcp.json`, or the Tools picker / MCP registry	Agent mode only

PLAIN ENGLISH

OAuth means you sign in and approve access in a browser; the tool receives a short-lived token, so no secret is stored in a file. A **CodeLens** is a small clickable link that the editor shows above a line. A **stdio** server is a local program that the editor starts and talks to directly; an **HTTP** server is a URL. **Front matter** is the block of settings between `---` lines at the top of a Markdown file.

The GitHub MCP server (remote, OAuth). URL: `https://api.githubcopilot.com/mcp/`. *"The remote GitHub MCP server uses one-click OAuth authentication by default, but you can also manually configure it to use a personal access token (PAT)."*

- **VS Code, scoped to the repository:** Command Palette → **MCP: Add Server** → **HTTP (HTTP or Server-Sent Events)** → `https://api.githubcopilot.com/mcp/` → save to the **Workspace** (writes `.vscode/mcp.json`). VS Code then asks you to sign in with GitHub. No secret is written to the file.
- **Visual Studio:** create `<SOLUTIONDIR>\.mcp.json`:

```
{ "servers": { "github": { "url": "https://api.githubcopilot.com/mcp/" } } }
```

then select **Authentication Required** in the CodeLens above it to sign in with OAuth.

> **COMMON EXAM TRAP**
>
> A **personal access token (PAT)** pasted into an `Authorization` header is *not* OAuth, and it puts a secret in a file that may be committed: *"Do not commit tokens, passwords, or populated authentication headers."* If a server really needs a secret in VS Code, use an input variable (`"Bearer ${input:api-token}"` with `"password": true`) so VS Code prompts for it and stores it securely. `.github\mcp.json` and `.vs\settings.json` are not MCP configuration locations.

SQL Server and Azure SQL.

- The simplest path needs no extra server: the MSSQL extension's `@mssql` participant and agent tools (VS Code), or the built-in `sql-tools` MCP server in SSMS Agent mode, use your existing connection.
- **SQL MCP Server** is part of **Data API builder** (DAB, see Chapter 13). It exposes the entities in `dab-config.json` as MCP tools (`describe_entities`, `read_records`, `create_record`, `update_record`, `delete_record`, `execute_entity`, `aggregate_records`) over HTTP (for example `https://<host>/mcp`) or stdio. It works with **data (DML), not schema**, it does *not* run free-form natural-language-to-SQL, and every tool call obeys DAB's role permissions.
- **Azure MCP Server** gives agents tools over Azure resources, authenticating with Microsoft Entra ID.

Fabric lakehouse. Publish a **Fabric data agent** over the lakehouse (or a warehouse, semantic model, KQL database, or mirrored database). *"When you publish a Fabric data agent, it becomes an MCP server that exposes a single MCP tool."* Copy the **MCP server URL** from the agent's **Settings** → **Model Context Protocol** tab and add it to VS Code with **MCP: Add Server** → **HTTP**; you sign in with Microsoft Entra ID. It needs a paid F2+ (or P1+) capacity, the tenant settings that allow cross-geo processing and storing for AI, and it only works after the agent is published.

> **SCENARIO PATTERN**
>
> "Developers use VS Code and GitHub Copilot and must retrieve **live** metadata from the databases" → **use an MCP server**. A `.dacpac` loaded into the context window, a schema pasted into an instruction file, or a SQL project in the repo are all **static copies** that go stale.

8.7 Chapter 8 self-check

1. Copilot in SSMS runs a query that returns customer emails. What decided that it could read them?
2. Name the file for team-wide Copilot coding standards, and its folder.
3. You want Copilot Chat to ignore the repository instructions on your machine only. What do you change?
4. Personal, organization, and repository instructions conflict. Which wins?
5. Which file makes a GitHub MCP server configuration part of a Visual Studio solution in source control?
6. Why do newly added MCP tools do nothing in SSMS until you act?
7. How do you let an AI assistant query a Fabric lakehouse through MCP?

Answers: 1) The developer's own login and its SQL permissions. 2) `copilot-instructions.md` in the `.github` folder at the repository root. 3) Turn off *Code Generation: Use Instruction Files* (`github.copilot.chat.codeGeneration.useInstructionFiles`) in your User settings. 4) Personal. 5) `<SOLUTIONDIR>\.mcp.json`. 6) MCP needs Agent mode, and new tools are disabled until you enable them in the Tools list. 7) Publish a Fabric data agent and add its MCP server URL as an HTTP server.

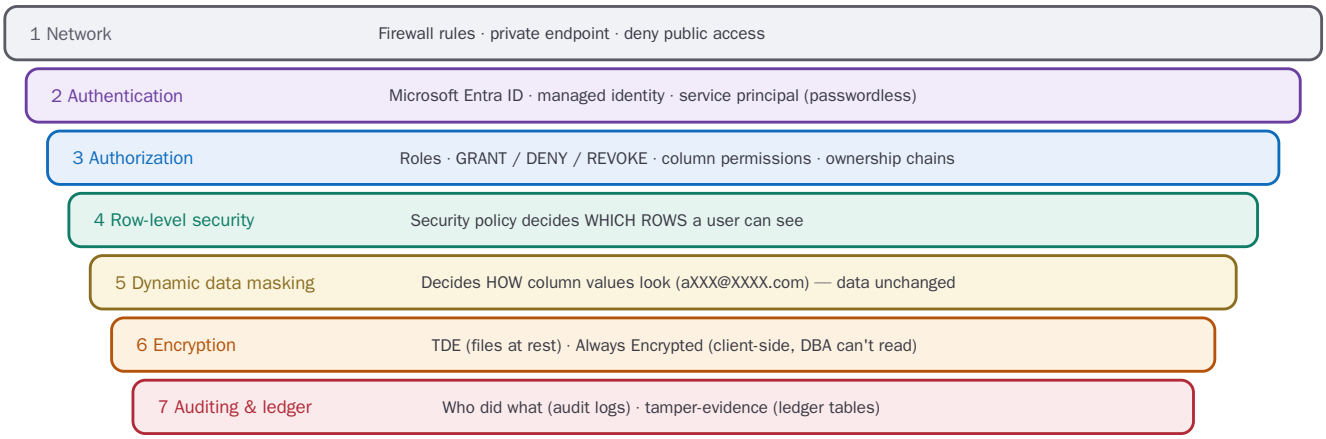
8.8 Learn more

- Training: Implement SQL solutions by using AI-assisted tools — especially security impact, instruction files, and MCP endpoints
- SSMS: GitHub Copilot in SSMS · Execution context · Agent mode · MCP servers in SSMS
- VS Code: GitHub Copilot with the MSSQL extension · Agent mode · Custom instructions · MCP servers in VS Code
- Visual Studio: Use MCP servers · GitHub: Custom instructions · Set up the GitHub MCP server
- Fabric: Enable Copilot in Fabric · Copilot for SQL database · Fabric data agent as an MCP server · SQL MCP Server

---

Chapter 9 — Data security: encryption, masking, row-level security, and permissions

> **EXAM FOCUS**
>
> **Implement data security and compliance** — design and implement data encryption, including Always Encrypted and column-level encryption · design and implement Dynamic Data Masking · design and implement Row-Level Security (RLS) · design and implement object-level permissions.

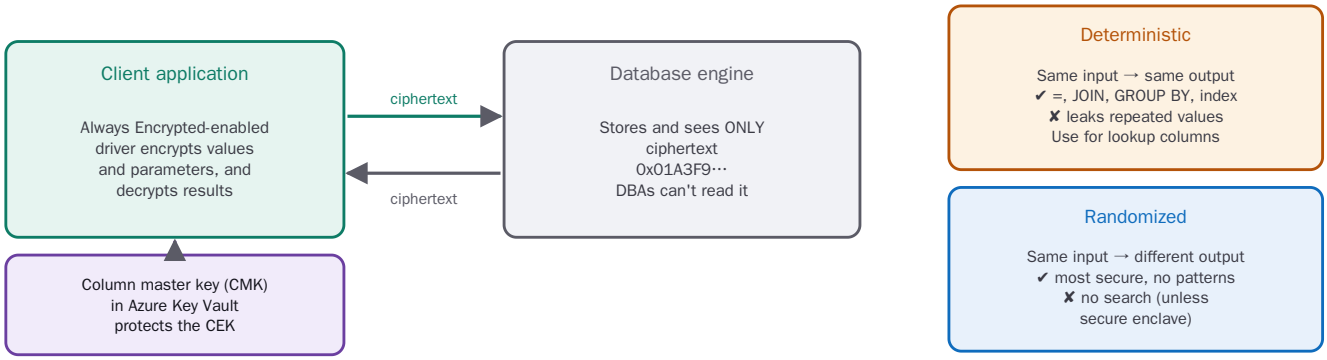


Defense in depth: each layer answers a different question. Exam scenarios usually need two layers at once — for example RLS for rows plus masking for columns.

9.1 Encryption: TDE, Always Encrypted, and column-level encryption

	TDE (Transparent Data Encryption)	Always Encrypted	Column-level (cell-level) encryption
What is encrypted	The database **files**, log, and backups on disk	Chosen **columns**	Chosen values, stored as `varbinary`
Who holds the keys	The server (service-managed or your key in Key Vault)	The **client** — keys never reach the engine	A symmetric key **inside** the database
Who can read plaintext	Anyone who can log in and has `SELECT`	Only clients with access to the column master key	Whoever can open the symmetric key
Protects against	Stolen disks or backups	DBAs, cloud operators, anyone on the server	Users without key permission
App changes	None	An Always Encrypted-enabled driver, parameterized queries	`ENCRYPTBYKEY` / `DECRYPTBYKEY` in T-SQL
Default	**On** for new Azure SQL databases	Off	Off

How Always Encrypted works. *"On the client, the Always Encrypted-enabled driver encrypts sensitive data before sending it to the Database Engine and automatically rewrites queries to maintain application semantics."* The engine stores and sees only ciphertext — in the column **and** in query parameters — and *"The Database Engine doesn't store or use any keys in plaintext."*



The client driver encrypts and decrypts; the engine only ever sees ciphertext. The column master key (often in Azure Key Vault) protects the column encryption key.

There are two keys: a **column encryption key (CEK)** encrypts the data; a **column master key (CMK)** — kept *"in a trusted key store outside of the database system, such as Azure Key Vault, Windows certificate store, or a hardware security module"* — encrypts the CEK.

Deterministic vs randomized — the most tested choice:

	Deterministic	Randomized
Same plaintext gives	The **same** ciphertext every time	A **different** ciphertext every time
Server-side operations allowed	Equality only: `=`, `IN`, joins, `GROUP BY`, `DISTINCT`, indexing — *"point lookups, equality joins, grouping, and indexing"*	**None** without an enclave
Weakness	*"unauthorized users might guess information about encrypted values by examining patterns"* — worst on low-cardinality columns (True/False, region)	No search, grouping, or joins
Use for	Columns you search by equality (national ID, account number)	Everything else (salary, notes) — **most secure**

Secure enclaves are a protected memory area inside the engine. With *Always Encrypted with secure enclaves*, the engine can do richer work on **randomized** columns — range comparisons, `LIKE`, sorting, and encrypting existing data in place. An enclave doesn't change the encryption type itself.

```
CREATE TABLE HR.Employees (
    EmployeeID int PRIMARY KEY,
    NationalID char(11) COLLATE Latin1_General_BIN2          -- strings need a _BIN2
collation
        ENCRYPTED WITH (ENCRYPTION_TYPE = DETERMINISTIC,        -- point lookups by
NationalID
                        ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256', COLUMN_ENCRYPTION_KEY =
CEK_HR),
    Salary money
        ENCRYPTED WITH (ENCRYPTION_TYPE = RANDOMIZED,           -- never searched: strongest
                        ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256', COLUMN_ENCRYPTION_KEY =
CEK_HR)
);
```

Practical limits: applications must pass **parameters** (not literals) for encrypted columns; keys are provisioned with tools (SSMS, PowerShell), not T-SQL; Always Encrypted can't be combined with dynamic data masking on the same column; and **SQL database in Fabric can't create Always Encrypted tables**.

Column-level encryption uses keys inside the database:

```
OPEN SYMMETRIC KEY CreditCards_Key DECRYPTION BY CERTIFICATE Sales09;
UPDATE Sales.CreditCard SET CardNumber_Encrypted = ENCRYPTBYKEY(KEY_GUID('CreditCards_Key'),
CardNumber);
SELECT CONVERT(nvarchar(25), DECRYPTBYKEY(CardNumber_Encrypted)) FROM Sales.CreditCard;
CLOSE SYMMETRIC KEY CreditCards_Key;
```

SCENARIO PATTERN

- "Neither the stored values **nor the query inputs** reveal information, and users can't infer **repetitions**" → **Always Encrypted, randomized**.
- "Point lookups by NationalIDNumber; InvestigationNotes is only returned; highest security" → **deterministic for NationalIDNumber, randomized for InvestigationNotes**. (Randomized on both only works with secure enclaves, which the question doesn't mention.)
- "Encrypted at rest, column master key in Key Vault, DBAs must not see it" → **Always Encrypted**.
- "Credit card numbers must NOT be stored as plain text" → Always Encrypted (TDE still shows plaintext to queries; masking stores plaintext).

9.2 Dynamic data masking (DDM)

DDM changes what users **see** in query results; *"the data in the database isn't changed."* Users with `SELECT` but without `UNMASK` see the mask.

Mask function	Result	Example
`default()`	Full mask by type: strings → `XXXX`, numbers → `0`, dates → `1900-01-01`	Phone `555-123-4567` → `XXXX`
`email()`	First letter, then `XXX@XXXX.com`	`alice@contoso.com` → `aXXX@XXXX.com`
`partial(prefix, "padding", suffix)`	Shows the first *prefix* and last *suffix* characters	`partial(0,"XXXX-XXXX-XXXX-",4)` → `XXXX-XXXX-XXXX-1234`
`random(start, end)`	A random number in the range	`random(1, 12)`
`datetime("Y")` (and M, D, h, m, s)	Masks part of a date/time	Year hidden

```
CREATE TABLE dbo.CustomerProfiles (
    CustomerId   int IDENTITY PRIMARY KEY,
    EmailAddress varchar(100) MASKED WITH (FUNCTION = 'email()')   NOT NULL,
    PhoneNumber  varchar(20)  MASKED WITH (FUNCTION = 'default()') NULL
);
ALTER TABLE dbo.Customers ALTER COLUMN CardNumber
    ADD MASKED WITH (FUNCTION = 'partial(0,"XXXX-XXXX-XXXX-",4)');

GRANT UNMASK TO ComplianceOfficer;                              -- whole database
GRANT UNMASK ON dbo.Customers(CardNumber) TO BillingLead;      -- one column (SQL Server
2022+, Azure SQL, Fabric)
```

COMMON EXAM TRAP

- **Admins always see real data.** *"Administrative users and roles can always view unmasked data via the CONTROL permission"* — `db_owner`, `sysadmin`, the server admin, and the Microsoft Entra admin.
- **DDM is not a security boundary for ad hoc queries.** A user can still filter (`WHERE Salary > 99999 AND Salary < 100001`) and infer values. Microsoft says DDM *"shouldn't be used alone to fully secure sensitive data"*; it is *"complementary to other SQL Server security features (auditing, encryption, row level security, etc.)."*
- **DDM doesn't encrypt.** "Not stored as plain text" → Always Encrypted, not DDM.
- **DDM and RLS stack.** RLS decides *which rows*; DDM decides *how the columns of those rows look*. Masking still applies to the rows RLS returns.
- In Fabric, DDM is configured with T-SQL only (no portal pane).

9.3 Row-level security (RLS)

RLS adds an invisible `WHERE` clause to every query on a table. You write an **inline table-valued function** (the *predicate*) that returns a row when the current user may see a row, and attach it with **`CREATE SECURITY POLICY`**.

- **Filter predicates** *"silently filter the rows available to read operations (SELECT, UPDATE, and DELETE)."*
- **Block predicates** *"explicitly block write operations (AFTER INSERT, AFTER UPDATE, BEFORE UPDATE, BEFORE DELETE) that violate the predicate."* A filter predicate alone still lets a user **insert** rows for another tenant — add `BLOCK … AFTER INSERT`.

```
CREATE SCHEMA Security;
GO
CREATE FUNCTION Security.fn_TenantPredicate (@TenantId int)
RETURNS TABLE
WITH SCHEMABINDING                                      -- required for the predicate
function
AS
RETURN SELECT 1 AS fn_result
       WHERE @TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS int);
GO
CREATE SECURITY POLICY Security.TenantPolicy
    ADD FILTER PREDICATE Security.fn_TenantPredicate(TenantId) ON dbo.Tickets,
    ADD BLOCK  PREDICATE Security.fn_TenantPredicate(TenantId) ON dbo.Tickets AFTER INSERT
WITH (STATE = ON);                                      -- always specify STATE = ON
GO
-- The application sets the tenant once per connection
EXEC sp_set_session_context @key = N'TenantId', @value = 42, @read_only = 1;
```

Two ways to know "who is asking": `USER_NAME()` / `SUSER_SNAME()` when every user signs in to the database, or `SESSION_CONTEXT(N'…')` when a middle-tier app shares one connection identity. For "analysts see the fleets they are assigned to (many fleets each)", the predicate joins to a user-to-fleet **mapping table**.

> **COMMON EXAM TRAP**
>
> - **RLS applies to everyone, including `dbo` and `db_owner`.** *"If high privileged users, such as sysadmin or db_owner, need to see all rows to troubleshoot or validate data, the security policy must be written to allow that."* Using an application doesn't bypass it: *"The database system applies the access restrictions every time that data access is attempted from any tier."*
> - RLS filters rows; it doesn't hide columns or encrypt anything. "See only the domain of the email" is DDM, not RLS.
> - Permissions (`GRANT` / `DENY`) can't target individual rows — that is what RLS is for.

9.4 Object-level permissions

Statement	Effect
`GRANT`	Allows an action
`DENY`	Explicitly forbids it — **DENY overrides any GRANT** (except for object owners and `sysadmin`)
`REVOKE`	Removes an earlier GRANT **or** DENY entry. It does not forbid anything by itself

> *"Permissions are cumulative, with the user receiving all the permissions granted to the user, login, and any group memberships; however any permission denial overrides all grants."*

```
GRANT EXECUTE ON OBJECT::Sales.usp_GetCustomerByCustomerId TO AppSupport;  -- procedures use
EXECUTE
GRANT SELECT  ON OBJECT::Sales.Customer TO AppSupport;                     -- query the
table
DENY  SELECT  ON OBJECT::Sales.Customer(TaxID) TO AppSupport;              -- …but never
TaxID
GRANT SELECT  ON SCHEMA::Reporting TO ReportReaders;                       -- all current
AND future objects
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];                         -- fixed role:
read all tables
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];                         -- fixed role:
write all tables
```

With the table-level `GRANT` plus the column-level `DENY` above, `AppSupport` can select every column except `TaxID` (and `SELECT *` fails). Using `REVOKE SELECT ON …(TaxID)` **instead of** the DENY would block nothing: REVOKE only removes an existing entry, there is no column-level entry to remove, and the table-level GRANT still covers `TaxID`. (Running REVOKE *after* the DENY would remove the DENY and expose `TaxID` again.)

One documented quirk: *"A table-level DENY does not take precedence over a column-level GRANT"* (kept for backward compatibility). The opposite pattern — table GRANT plus column DENY, as above — **does** block the column.

Ownership chaining. When a procedure and the tables it uses have the **same owner**, SQL Server checks only the caller's `EXECUTE` permission on the procedure, not their permissions on the tables. So you can grant `EXECUTE` on procedures and give users no direct table access at all.

> **TIP**
> Grant permissions to **roles**, not to individual users, and group objects into **schemas** so one schema-level grant covers new objects too. Least privilege: `db_datareader` + `db_datawriter` for an app that reads and writes, not `db_owner` or `CONTROL`.

9.5 Chapter 9 self-check

1. Which encryption protects data from a DBA who can query the table?
2. Which Always Encrypted type supports `WHERE AccountNumber = @acct`?
3. A support role must see only the domain part of customer emails. Which feature?
4. A tenant isolation policy uses only a filter predicate. What can a user still do?
5. A role has `GRANT SELECT` on a table and `DENY SELECT` on one column. What happens with `SELECT *`?
6. Does RLS filter rows for members of `db_owner`?

Answers: 1) Always Encrypted. 2) Deterministic. 3) Dynamic data masking. (The built-in `email()` mask shows `aXXX@XXXX.com`, not the real domain; a custom `partial()` mask can expose trailing characters.

Either way the answer is "configure DDM".) 4) Insert rows for another tenant — add a block predicate `AFTER INSERT`. 5) It fails with a permission error because it touches the denied column. 6) Yes, unless the predicate explicitly allows them.

9.6 Learn more

- Training: Implement data security and compliance with SQL — encryption, masking, RLS, permissions
- Always Encrypted · Always Encrypted cryptography · Secure enclaves · TDE in Azure SQL · Encrypt a column of data
- Dynamic data masking · Row-level security · CREATE SECURITY POLICY · Getting started with permissions

---

Chapter 10 — Secure access, auditing, and securing model and API endpoints

> **EXAM FOCUS**
> **Implement data security and compliance** — implement secure database access, including passwordless · implement auditing · secure model endpoints, including Managed Identity · secure GraphQL, REST, and MCP endpoints.

10.1 Passwordless access with Microsoft Entra ID

Passwordless means the application never stores a password, key, or secret. Instead it gets a token from **Microsoft Entra ID** (formerly Azure Active Directory).

Identity	What it is	Use it when
System-assigned managed identity	An identity Azure creates for **one** Azure resource; *"tied to the lifecycle of that Azure resource"*	One Azure app (App Service, Functions, Container Apps, VM) needs database access
User-assigned managed identity	A standalone identity resource that *"can be used by multiple resources"*	Several apps need the same permissions, or the identity must survive redeployments
Service principal	An app registration in Entra ID (best with a certificate credential)	The app runs **outside Azure** — on-premises or in another cloud
Microsoft Entra users and groups	People, ideally with MFA	Developers, DBAs, analysts

> *"Apps hosted outside of Azure, such as on-premises or in a third-party data center, should use an application service principal through Microsoft Entra ID to authenticate to Azure services."*

(The exception: a server onboarded to **Azure Arc** can use a managed identity.)

Setting up an app with a managed identity (the exam pattern):

1. Set a **Microsoft Entra admin** on the logical server.
2. Turn on the app's managed identity (for example `az webapp identity assign …`).
3. Connected **as an Entra identity** with `ALTER ANY USER`, create a contained database user from the external provider, and give it only what it needs:

```
CREATE USER [OrderApi-Id] FROM EXTERNAL PROVIDER;      -- display name of the identity
ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id];
ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];
```

1. Change the connection string — no password:

Identity	Connection string fragment
System-assigned	`Authentication=Active Directory Managed Identity;` (or `Active Directory Default`)
User-assigned	`Authentication=Active Directory Managed Identity;User Id=<client-id>;`
Service principal	`Authentication=Active Directory Service Principal;User Id=<app client id>;…`

COMMON EXAM TRAP

- `CREATE LOGIN … FROM EXTERNAL PROVIDER` creates a **server** login (in `master`), not a database user — role memberships like `db_datareader` need a database **user**.
- `CREATE USER … WITH PASSWORD` is SQL authentication — the opposite of passwordless.
- Azure SQL Database has no `sysadmin` role, and Azure RBAC roles (SQL DB Contributor) manage the *resource*, they don't grant data access.
- For a **user-assigned** identity, the user name is the identity's name (`OrderApi-Id`), not the web app's name; the connection string uses its **client ID**.
- A **shared access key** (as used by Storage or Event Hubs) is a secret, and Azure SQL doesn't accept it for sign-in.
- Keeping a SQL password in Key Vault, or rotating it weekly, is still password-based.

SQL database in Fabric uses Microsoft Entra authentication only — no SQL logins. To connect, a user needs at least the **Read** item permission (workspace roles Admin, Member, and Contributor give full access; Viewer gives read). T-SQL `GRANT` / `DENY` /RLS add finer control, and a `DENY` in the database always wins.

10.2 Network access

> *"By default, the logical server denies all connections."*

Option	What it does	Notes
Server/database IP firewall rules	Allow specific public IP ranges	Database-level rules are checked first
"Allow Azure services" = firewall rule `0.0.0.0` to `0.0.0.0` (named `AllowAllWindowsAzureIps`)	Lets **any** Azure-hosted client reach the public endpoint	*"allows connections from all Azure services, including services running in other customers' subscriptions"*
Virtual network rules (service endpoints)	Allow a specific subnet	Traffic still uses the public endpoint
Private endpoint (Private Link)	Gives the server *"a private IP address within a specific VNet and subnet"*	Works for VNet-integrated App Service and Functions too
Deny public network access	Only private endpoints can connect	*"When you add a private endpoint connection, public routing to your logical server isn't blocked by default"* — you must also deny public access

SCENARIO PATTERN

- "sales-api (App Service with VNet integration) must use **passwordless** authentication and **all database traffic must stay in the subscription**, minimal admin effort" → **managed identity + Microsoft Entra authentication**, and **a private endpoint with public network access disabled**.
- "The DAB container app can't connect to the logical server; the login is correct; don't change the container or the config file" → add the firewall rule `0.0.0.0` – `0.0.0.0` (allow Azure services). `127.0.0.1` is loopback; `10.0.0.0/8` is private address space, not where public-endpoint traffic comes from.

10.3 Auditing and ledger

Auditing records *who did what, and when*.

Platform	Feature	Where the logs go
Azure SQL Database	**Azure SQL auditing** (server-level policy for all databases, or database-level)	Azure Storage, Log Analytics, and/or Event Hubs
SQL Server	**SQL Server Audit**: a server audit + server/database audit specifications	A file, the Windows Security log, or the Application log
Azure SQL MI	The same SQL Server Audit objects	Azure Blob Storage (`TO URL`) or Log Analytics / Event Hubs (`TO EXTERNAL_MONITOR`) — not files or Windows logs
SQL database in Fabric	Fabric SQL auditing	Immutable files in OneLake, read with `sys.fn_get_audit_file_v2`

The default Azure SQL auditing policy captures *"all the queries and stored procedures executed against the database, as well as successful and failed logins."* For long retention that a DBA can't tamper with, audit to **Blob Storage with an immutable (WORM — write once, read many) policy**, in a separate storage account. Use **Log Analytics** for KQL (Kusto Query Language) queries and alerts (`AzureDiagnostics | where Category == "SQLSecurityAuditEvents"`), and **Event Hubs** to stream to a SIEM (a security information and event management system). Note that failed Microsoft Entra sign-ins don't appear in the SQL audit log — check the Entra sign-in logs.

```
-- SQL Server (on Managed Instance use TO URL or TO EXTERNAL_MONITOR instead of TO FILE)
CREATE SERVER AUDIT SecurityAudit TO FILE (FILEPATH = 'C:\AuditLogs\');
ALTER SERVER AUDIT SecurityAudit WITH (STATE = ON);
CREATE DATABASE AUDIT SPECIFICATION SensitiveDataAudit FOR SERVER AUDIT SecurityAudit
    ADD (SELECT, INSERT, UPDATE, DELETE ON dbo.SensitiveData BY public)
    WITH (STATE = ON);
```

Ledger (3.3) is different: it doesn't record *access*; it *proves the data wasn't altered*, by chaining SHA-256 hashes of every change and storing **digests** outside the database (for example immutable Blob Storage or Azure Confidential Ledger).

> **KNOW THE DIFFERENCE**
>
> **Auditing vs ledger.** "Who read or changed this data, and when?" → **auditing**. "Prove to auditors, cryptographically, that the history wasn't tampered with — even by a DBA" → **ledger tables**.

10.4 Securing model endpoints

When T-SQL calls an AI model with `sp_invoke_external_rest_endpoint` (16.2), the credentials come from a **database scoped credential**. The credential's **name is the endpoint URL** it applies to.

`IDENTITY =`	How authentication is sent	Security
`'Managed Identity'`	The server's managed identity gets an Entra token for the resource named in `SECRET`	**Best — no key stored anywhere**
`'HTTPEndpointHeaders'`	A key in a request header, e.g. `{"api-key":"…"}`	A stored, long-lived secret
`'HTTPEndpointQueryString'`	A key in the URL query string, e.g. `{"code":"…"}`	Weakest — URLs end up in logs
`'Shared Access Signature'`	A SAS token for storage	Delegated, time-limited

```
-- Most secure: managed identity, no API key
CREATE DATABASE SCOPED CREDENTIAL [https://my-openai.openai.azure.com]
    WITH IDENTITY = 'Managed Identity',
         SECRET   = '{"resourceid":"https://cognitiveservices.azure.com"}';

-- Who may call endpoints, and use this credential
GRANT EXECUTE ANY EXTERNAL ENDPOINT TO AIFeatureUsers;
GRANT REFERENCES ON DATABASE SCOPED CREDENTIAL::[https://my-openai.openai.azure.com] TO
AIFeatureUsers;
```

Then give the database server's managed identity the **Cognitive Services OpenAI User** role on the Azure OpenAI resource. Rules: the `SECRET` for a managed identity is **not a key** — it is a small JSON naming the Entra resource; the credential name must match the protocol and host (FQDN) of the URL you call and must not contain a query string. On **SQL Server 2025** you must first enable the procedure (`sp_configure 'external rest endpoint enabled', 1`) and, for managed identities, `sp_configure 'allow server scoped db credentials', 1` (the server is Arc-enabled).

> **COMMON EXAM TRAP**
>
> `IDENTITY = 'Managed Identity'` with an `api-key` secret mixes two models; `IDENTITY = 'AzureOpenAI'` isn't a valid value. A wrapper procedure (`GRANT EXECUTE ON dbo.GetEmbedding TO AIFeatureUsers`) is a good way to control who can spend model tokens.

10.5 Securing GraphQL, REST, and MCP endpoints

Data API builder (DAB) (Chapter 13) secures its REST, GraphQL, and MCP endpoints in two steps:

1. **Authentication** — who is calling. Providers include **EntraId**, **AppService** (App Service's built-in "Easy Auth" sign-in), **Custom** (any OpenID Connect issuer of JWTs — signed JSON Web Tokens), and **Simulator** (testing only). Providers other than Unauthenticated, Simulator, and StaticWebApps need a token `audience` and `issuer`. In DAB 2.0 the default is **Unauthenticated**: DAB doesn't validate tokens and every request runs as `anonymous` — fine behind a trusted front end, wrong if clients call DAB directly.

2. **Authorization** — what they can do. *"Any incoming request, authenticated or not, is assigned to a role."* The system roles are `anonymous` (no token) and `authenticated` (valid token); custom roles come from the token's roles claim plus the `X-MS-API-ROLE` header. *"No role has default permissions"* — each entity lists actions per role (`create`, `read`, `update`, `delete`, `execute`), optionally with `fields` include/exclude and a **database policy** such as `"@item.OwnerId eq @claims.userId"` (database policies work for read, update, and delete — not for create or execute).

```
"Book": {
  "source": "dbo.books",
  "permissions": [
    { "role": "anonymous",     "actions": [ { "action": "read", "policy": { "database":
"@item.IsPublic eq true" } } ] },
    { "role": "authenticated", "actions": [ { "action": "read", "fields": { "include":
["*"], "exclude": ["CostPrice"] } } ] },
    { "role": "admin",         "actions": [ "*" ] }
  ]
}
```

Hardening: GraphQL introspection is **on** by default in DAB and query depth is **unlimited** by default — set `"allow-introspection": false` and a `"depth-limit"` in `runtime.graphql` for public APIs. The **SQL MCP Server** uses the same entities and role permissions, offers only data (DML) tools, and never runs free-form SQL.

Fabric API for GraphQL is a Fabric item:

- Callers authenticate with **Microsoft Entra ID**.
- **Calling** the API needs the **Execute** permission — shown as *Run Queries and Mutations* when you share it: *"The authenticated credential (user principal, service principal, or managed identity) calling the API needs Execute permissions for the GraphQL API."*
- **Connectivity:** with **single sign-on (SSO)**, the caller's own identity reaches the data source, so the caller *also* needs read/write permission there (or the Contributor workspace role). With **saved credentials**, one stored credential is used and callers need no data-source access (saved credentials are required for an Azure SQL Database source).
- **Editing** the API (its schema, field names, mappings) is an **Edit** permission (*"Recipient can edit the item or its content"*), separate from running queries.
- GraphQL **introspection** (letting clients download the schema) is **off** by default in Fabric API for GraphQL and only workspace admins can turn it on — the opposite of DAB's default.

> **SCENARIO PATTERN**
> A group has *View and Edit GraphQL item* but **not** *Run Queries and Mutations*, the Viewer workspace role, and the API uses SSO → they **can't** read or modify data through the API (no Execute), but they **can** change the API's field mappings (Edit).

10.6 Chapter 10 self-check

1. An Azure Functions app and an App Service app need identical database permissions. Which identity type reduces the number of database users?
2. An on-premises service must connect without a password. Which identity?
3. Write the T-SQL that lets managed identity `OrderApi-Id` read and write data.
4. What does the firewall rule `0.0.0.0`–`0.0.0.0` allow, and why is it risky?
5. You added a private endpoint. Is public access blocked?
6. Which `IDENTITY` value gives the most secure database scoped credential for Azure OpenAI, and what goes in `SECRET`?
7. In DAB, a request has no token. Which role is it?

Answers: 1) A user-assigned managed identity. 2) A service principal. 3) `CREATE USER [OrderApi-Id] FROM EXTERNAL PROVIDER; ALTER ROLE db_datareader ADD MEMBER [OrderApi-Id]; ALTER ROLE db_datawriter ADD MEMBER [OrderApi-Id];` 4) Connections from any Azure service — including other customers' subscriptions. 5) No — you must also deny public network access. 6) `'Managed Identity'`; `'{"resourceid":"https://cognitiveservices.azure.com"}'`. 7) `anonymous`.

10.7 Learn more

- Training: Configure secure access to AI services · Secure data API endpoints · Implement auditing
- Identity: Microsoft Entra authentication for Azure SQL · Configure Entra authentication · Managed identities · Connect App Service with a managed identity · Apps hosted on-premises
- Network: Network access controls · IP firewall rules · Private Link
- Auditing: Azure SQL auditing · SQL Server Audit · Endpoints: sp_invoke_external_rest_endpoint · DAB authorization · Fabric API for GraphQL permissions · Share Fabric items

---

Chapter 11 — Optimize database performance

EXAM FOCUS

Optimize database performance — recommend database configurations · preserve data integrity and consistency by using transaction isolation levels and concurrency controls · evaluate query performance by using query execution plans, dynamic management views (DMVs), Query Store, and Query Performance Insight · identify and resolve query performance issues, including blocking and deadlocks.

11.1 Recommending database configurations

Pick the right container first. Azure SQL Database (vCore model) tiers:

	General Purpose	Business Critical	Hyperscale
Storage	Remote, about 5–10 ms latency	Local SSD, lowest latency	Distributed page servers + SSD cache
Max size	4 TB	4 TB	128 TB
Readable replica	No	**Yes — a free read-only replica**	Up to 4 HA + 30 named replicas
In-Memory OLTP	No	**Yes**	A subset
Best for	Most workloads (default)	Low latency, heavy I/O, fastest failover	Very large databases, fast scaling

Serverless compute (General Purpose, Hyperscale) scales automatically and can auto-pause when idle (General Purpose); **provisioned** compute is a fixed size.

Then the database settings:

Setting	What it does	Default (new database)
Compatibility level	Turns on newer optimizer features (150 = scalar UDF inlining, batch mode on rowstore; 160 = parameter-sensitive plans; 170 = latest)	**170** in Azure SQL Database and Fabric. Existing databases are **never upgraded automatically**
MAXDOP (database scoped)	Maximum parallel workers per query	**8** in Azure SQL Database and Fabric (avoid 0)
Automatic tuning	`FORCE_LAST_GOOD_PLAN` reverts plan regressions automatically; `CREATE_INDEX` / `DROP_INDEX`	Azure: FORCE_LAST_GOOD_PLAN **on**, index options off (Fabric turns CREATE INDEX on)
Query Store	Records plans and runtime stats over time	**On** in Azure SQL, MI, SQL Server 2022+, Fabric
RCSI (`READ_COMMITTED_SNAPSHOT`)	Readers see the last committed version instead of waiting for locks	**On** in Azure SQL Database and Fabric; off in SQL Server and MI
Accelerated database recovery (ADR)	Fast recovery and instant rollback	Always on in Azure SQL and Fabric
Optimized locking	Fewer, shorter-lived locks	Always on in Azure SQL Database, Fabric, and MI (latest policies); opt-in on SQL Server 2025
`OPTIMIZE_FOR_AD_HOC_WORKLOADS`	Caches a small stub for single-use ad hoc queries	Off

```
ALTER DATABASE SCOPED CONFIGURATION SET MAXDOP = 8;
ALTER DATABASE CURRENT SET COMPATIBILITY_LEVEL = 170;
ALTER DATABASE CURRENT SET AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON);
```

> **TIP**
> Safe way to raise the compatibility level: keep the old level, make sure Query Store is capturing a baseline, raise the level, then force the old plan for any query that regresses (11.5).

11.2 Transactions, isolation levels, and concurrency

Isolation levels decide how much one transaction is protected from others' changes while it reads. **Pessimistic** levels use locks (readers and writers wait for each other); **optimistic** levels use **row versions** so readers read an older committed copy instead of waiting. Writers always lock: *"A transaction always gets an exclusive lock on any data it modifies, and holds that lock until the transaction completes, regardless of the isolation level."*

Isolation level	Dirty read	Non-repeatable read	Phantom	How it works
READ UNCOMMITTED	✘ possible	✘ possible	✘ possible	No shared locks — reads uncommitted data
READ COMMITTED (locking)	✔ prevented	✘ possible	✘ possible	Shared locks released after each read
RCSI (default in Azure SQL DB & Fabric)	✔ prevented	✘ possible	✘ possible	Reads the last committed version; no read locks
REPEATABLE READ	✔ prevented	✔ prevented	✘ possible	Keeps shared locks on rows READ until commit
SNAPSHOT	✔ prevented	✔ prevented	✔ prevented	One point-in-time view; writers not blocked
SERIALIZABLE	✔ prevented	✔ prevented	✔ prevented	Range locks — also blocks INSERTs in the range

Lock-based levels lower down block more. Pick the LEAST strict level that still meets the requirement.

Read across: what each level prevents and how. Pick the least strict level that meets the requirement — stricter locking levels block more.

PLAIN ENGLISH

- **Dirty read** — you read a change another transaction hasn't committed yet (and may roll back).
- **Non-repeatable read** — you read a row twice in one transaction and get different values, because someone changed it in between.
- **Phantom** — you run the same range query twice and new rows appear, because someone inserted them.

Requirement	Isolation level
"Records accessed must **NOT be changed** by other transactions while the procedure runs"	**REPEATABLE READ** — *"no other transactions can modify data that was read by the current transaction until the current transaction completes"* (shared locks held to the end)
Also: no **new rows** may appear in the ranges read (no phantoms)	**SERIALIZABLE** — adds **range locks**; the most restrictive
Several reads must see **one consistent point in time**, without blocking writers	**SNAPSHOT** — needs `ALLOW_SNAPSHOT_ISOLATION ON` (on by default in Azure SQL Database). Conflicting updates fail with error 3960 and must be retried
Readers must not block writers (default behaviour)	**READ COMMITTED with RCSI** — already the default in Azure SQL Database and Fabric
Rough numbers fast, correctness not important	READ UNCOMMITTED (same as `NOLOCK`)

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
BEGIN TRANSACTION;
    -- rows read here can't be changed by others until COMMIT
COMMIT TRANSACTION;

ALTER DATABASE SalesDB SET READ_COMMITTED_SNAPSHOT ON;   -- RCSI is a DATABASE option
ALTER DATABASE SalesDB SET ALLOW_SNAPSHOT_ISOLATION ON;  -- enables SET … SNAPSHOT
```

COMMON EXAM TRAP

- **SNAPSHOT doesn't stop writers.** *"SNAPSHOT transactions reading data don't block other transactions from writing data."* If the requirement is "records must not be changed while it runs", SNAPSHOT is wrong.
- **RCSI is not a `SET TRANSACTION ISOLATION LEVEL` value** — it is the database option `READ_COMMITTED_SNAPSHOT`. Under RCSI, *"SELECT statements … don't acquire shared (S) locks on the data being read, and therefore don't block transactions that are modifying data."* So in a **new Azure SQL Database or Fabric SQL database**, an open `UPDATE` does **not** block a plain `SELECT` (it still blocks other writers).
- **REPEATABLE READ still allows phantoms**; only SERIALIZABLE and SNAPSHOT prevent them. Choosing SERIALIZABLE when REPEATABLE READ is enough adds blocking for nothing.

11.3 Reading execution plans

An **execution plan** is the recipe SQL chose to run a query. Get the **estimated** plan without running the query (`SET SHOWPLAN_XML ON`) or the **actual** plan with runtime row counts (`SET STATISTICS XML ON`, or *Include Actual Execution Plan* in SSMS). Data flows from the table-access operators on the right towards the result on the left (the DP-800 unit phrases it as "read left to right, top to bottom" — either way, start where the tables are read). Look for:

You see	It usually means	Typical fix
Index/table scan on a selective query	No useful index, or a non-SARGable predicate	Add an index; rewrite the predicate (6.5)
Key Lookup	*"the engine finds rows through a nonclustered index but needs extra columns from the clustered index"*	Add the columns to the index with `INCLUDE`
Sort before `TOP`	The index order doesn't match `ORDER BY`	Make the `ORDER BY` column part of the index key, in that direction
Estimated rows ≠ actual rows	Stale or missing statistics, parameter sensitivity	`UPDATE STATISTICS`; Query Store hints
Warnings: implicit conversion, spill to tempdb, missing index	Type mismatch, low memory grant	Match types; fix estimates

SCENARIO PATTERN

A plan shows an **Index Seek on** `IX_Orders_OrderDate`, a **Key Lookup** for `CustomerId`, `Status`, `TotalAmount`, and a **Sort before Top (50)** for `ORDER BY OrderDate DESC`. Fix: `CREATE INDEX … ON dbo.Orders (CustomerId, OrderDate DESC) INCLUDE (Status, TotalAmount);`. Adding `CustomerId` as an *included* column to the date index does not give an ordered seek. The bottleneck is a **suboptimal plan**, not locking.

11.4 Dynamic management views (DMVs)

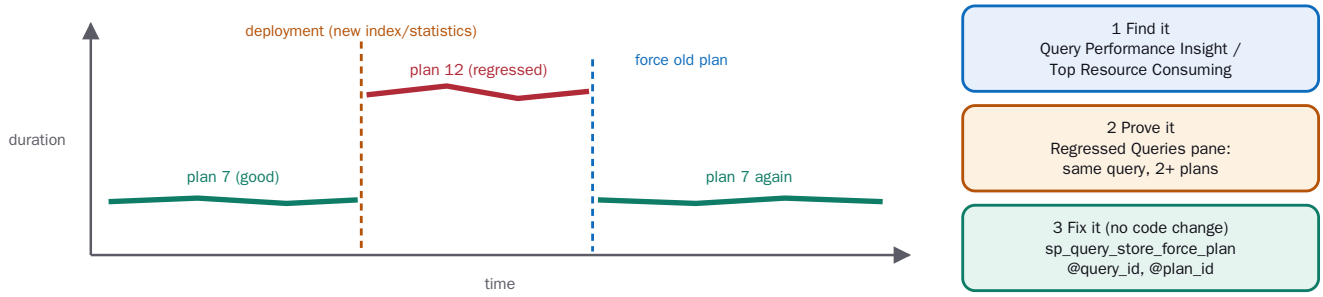
DMV	Shows	Remember
`sys.dm_exec_requests`	Requests **running now**: status, `wait_type`, `blocking_session_id`, CPU	A **sleeping** session has **no row** here
`sys.dm_exec_sessions`	**Every** connection, active or not: login, host, status, `open_transaction_count`	Use it to find an idle head blocker
`sys.dm_exec_sql_text(sql_handle)`	Text of a running batch	NULL for an idle session
`sys.dm_exec_input_buffer(session_id, NULL)`	The **last command** a session sent	Works for sleeping sessions
`sys.dm_exec_query_stats`	Aggregate stats for plans **in the plan cache**	Lost when a plan leaves the cache — not Query Store
`sys.dm_tran_locks`, `sys.dm_os_waiting_tasks`	Lock requests and waits	One row per lock
`sys.dm_db_missing_index_details`	Missing-index suggestions	Equality columns, inequality columns, included columns
`sys.dm_db_resource_stats`	Azure SQL Database CPU / I/O %, every 15 s for about an hour	

In Azure SQL Database, reading most DMVs needs `VIEW DATABASE STATE`. To see which session **holds** the locks in a blocking chain, join `sys.dm_tran_locks.request_session_id` to `sys.dm_exec_sessions` — joining to `sys.dm_exec_requests` misses an idle lock holder.

11.5 Query Store and Query Performance Insight

Query Store is a built-in "flight recorder". It has *"a plan store for persisting the execution plan information"*, a **runtime stats store**, and a **wait stats store**, all kept across restarts. Its views include `sys.query_store_query`, `sys.query_store_plan`, `sys.query_store_runtime_stats`, and `sys.query_store_runtime_stats_interval`. SSMS shows reports such as **Regressed Queries**, **Top Resource Consuming Queries**, **Queries With High Variation**, and **Queries With Forced Plans**.

Query Performance Insight (QPI) is the Azure SQL Database portal view built **on Query Store**: *"It helps identify the top resource consuming and long-running queries in your workload."* By default it shows the top five CPU-consuming queries; switch the metric to **duration** on the *Custom* tab to find long-running queries.



A plan choice regression: after a deployment the optimizer picked a worse plan. Find it, prove it, and force the old plan — no code change.

Fixing a plan regression without changing code:

```
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 7;    -- 7 = the older, faster
plan
EXEC sp_query_store_unforce_plan @query_id = 42, @plan_id = 7;  -- undo later
```

> *"Whenever you notice a plan choice regression has occurred, you should find a previous good plan and force it to be used instead of the current one."* Plan forcing *"doesn't require any change in user applications."*

Option	When
`sp_query_store_force_plan`	A known-good older plan exists — fastest, direct fix
`sys.sp_query_store_set_hints @query_id, N'OPTION (…)'`	You need a **hint** (RECOMPILE, MAXDOP, MAX_GRANT_PERCENT) on a query you can't edit
`AUTOMATIC_TUNING (FORCE_LAST_GOOD_PLAN = ON)`	Let the engine detect, force, validate, and revert automatically (on by default in Azure) — but it waits to confirm
`DBCC FREEPROCCACHE`	**Not a fix**: it evicts every plan, causes a recompile storm, and the new compile will probably pick the same bad plan

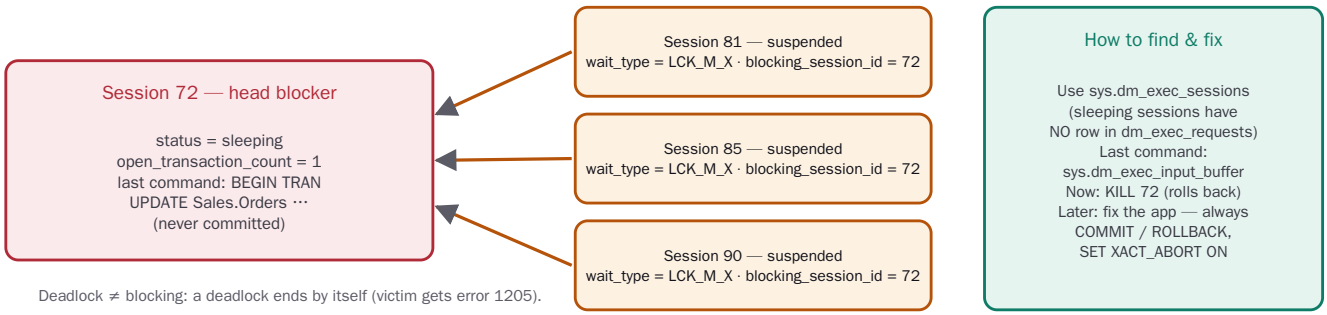
Top CPU queries in the last two hours (the documented Azure SQL query):

```
SELECT TOP 15 q.query_hash,
       SUM(rs.count_executions * rs.avg_cpu_time / 1000.0) AS total_cpu_ms,   -- avg ×
executions = total
       COUNT(DISTINCT p.plan_id) AS number_of_distinct_plans
FROM sys.query_store_query_text AS qt
JOIN sys.query_store_query AS q          ON qt.query_text_id = q.query_text_id
JOIN sys.query_store_plan AS p           ON q.query_id = p.query_id
JOIN sys.query_store_runtime_stats AS rs ON rs.plan_id = p.plan_id
JOIN sys.query_store_runtime_stats_interval AS rsi
                                         ON rsi.runtime_stats_interval_id =
rs.runtime_stats_interval_id
WHERE rsi.start_time >= DATEADD(HOUR, -2, GETUTCDATE())                       -- intervals
are UTC
GROUP BY q.query_hash
ORDER BY total_cpu_ms DESC;
```

Query Store keeps an **average** per plan per interval (`avg_cpu_time`, in microseconds), so multiplying by `count_executions` gives the total; `last_cpu_time` is only the latest execution. Many distinct `query_id`s for one hash suggests the app isn't parameterizing its queries.

> **SCENARIO PATTERN**
>
> "Identify whether the latency increase was caused by a plan regression and restore the previous plan without code changes" → **Query Performance Insight** to find long-running queries → **Regressed Queries** pane to confirm the query has several plans over time → **plan forcing in Query Store**. Azure Monitor and Log Analytics show metrics and logs; they can't force a plan.

11.6 Blocking and deadlocks



The classic: an application started a transaction, updated rows, and never committed. The session sleeps while holding exclusive locks; everyone who needs those rows waits.

Blocking means one session waits for a lock another session holds. Find the **head blocker** — a session that blocks others but isn't blocked itself — and see what it is doing.

Clue	Meaning
Waiters show `wait_type = LCK_M_X` and the same `blocking_session_id`	They want exclusive locks the head blocker holds
Head blocker `status = sleeping` and `open_transaction_count = 1`	An **explicit transaction was started but never committed or rolled back**
Input buffer shows `BEGIN TRANSACTION UPDATE …` with no `COMMIT`	Confirms it
It clears only when you `KILL` the session	KILL rolls the transaction back and releases the locks

> *"This type of blocking can often be identified by a session ID that is sleeping or awaiting a command, yet whose transaction nesting level (@@TRANCOUNT, open_transaction_count …) is greater than zero."*

The blocking-chain query (from Microsoft's Azure SQL blocking guide) — note the join types:

```
WITH cteBL (session_id, blocking_these) AS
(SELECT s.session_id, blocking_these = x.blocking_these
 FROM sys.dm_exec_sessions AS s
 CROSS APPLY (SELECT ISNULL(CONVERT(varchar(6), er.session_id), '') + ', '
              FROM sys.dm_exec_requests AS er
              WHERE er.blocking_session_id = ISNULL(s.session_id, 0)
                AND er.blocking_session_id <> 0
              FOR XML PATH('')) AS x (blocking_these))
SELECT s.session_id, blocked_by = r.blocking_session_id, bl.blocking_these,
       batch_text = t.text, input_buffer = ib.event_info
FROM sys.dm_exec_sessions AS s
LEFT OUTER JOIN sys.dm_exec_requests AS r ON r.session_id = s.session_id   -- keeps idle
sessions
INNER JOIN cteBL AS bl ON s.session_id = bl.session_id
OUTER APPLY sys.dm_exec_sql_text(r.sql_handle) AS t                         -- NULL for idle
sessions
OUTER APPLY sys.dm_exec_input_buffer(s.session_id, NULL) AS ib            -- last command,
even when idle
WHERE bl.blocking_these IS NOT NULL OR r.blocking_session_id > 0;
```

Why these choices: *"While sys.dm_exec_requests lists only requests currently executing, any connection (active or not) is listed in sys.dm_exec_sessions."* An `INNER JOIN` to requests, or `CROSS APPLY` to `sql_text`, would drop the sleeping head blocker — the one you most need. The input buffer must use `s.session_id` (always present), not `r.session_id`.

Fixing blocking: short term, `KILL` an orphaned session; long term, fix the application — commit or roll back on every code path, add `IF @@TRANCOUNT > 0 ROLLBACK` to error handlers, use `SET XACT_ABORT ON`, keep transactions short, and never wait for user input inside a transaction. Index the blocking query so it touches fewer rows.

Deadlocks are a *circle* of waits (A waits for B, B waits for A). The engine detects them and picks a **victim** — *"the transaction that is the least expensive to roll back"* (adjust with `SET DEADLOCK_PRIORITY`) — which gets

error 1205. *"Deadlocks are resolved almost immediately, whereas blocking can, in theory, persist indefinitely."* Applications should **retry** after 1205. To capture deadlock graphs in Azure SQL Database or Fabric, create an Extended Events session for `sqlserver.database_xml_deadlock_report` (SQL Server and MI capture them in the built-in `system_health` session).

> **KNOW THE DIFFERENCE**
>
> **Blocking vs deadlock vs lock escalation.** *Blocking* waits until the holder finishes (maybe forever). A *deadlock* is resolved automatically by killing one victim. *Lock escalation* turns many row/page locks into one table lock during a statement (about 5,000 locks); it can widen blocking but doesn't explain a sleeping session with an open transaction.

11.7 Chapter 11 self-check

1. Which Azure SQL Database tier gives low-latency local storage and a free read-only replica?
2. A procedure must make sure no row it reads changes until it commits, with the least blocking. Which isolation level?
3. In a new Fabric SQL database, does an open `UPDATE` block a plain `SELECT` of the same row?
4. Why does `sys.dm_exec_requests` not show the head blocker in the classic open-transaction scenario?
5. Two plans exist in Query Store and the newer one is slower. What do you run?
6. Which portal tool finds long-running queries in Azure SQL Database, and what does it depend on?
7. What does error 1205 mean, and what should the application do?

Answers: 1) Business Critical. 2) REPEATABLE READ. 3) No — RCSI is on by default, so readers read the last committed version. 4) It is sleeping (no active request); only `sys.dm_exec_sessions` lists it. 5) `sp_query_store_force_plan @query_id, @plan_id` with the older plan. 6) Query Performance Insight; Query Store. 7) Its transaction was chosen as a deadlock victim and rolled back — retry it.

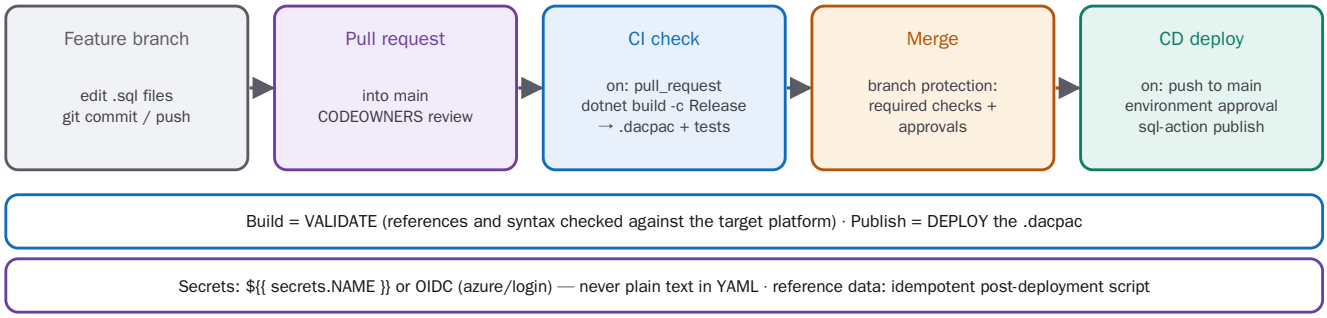
11.8 Learn more

- Training: Optimize database performance — configurations, isolation levels, plans and DMVs, Query Store, blocking and deadlocks
- vCore service tiers · Automatic tuning · SET TRANSACTION ISOLATION LEVEL · Locking and row versioning guide · Optimized locking
- Monitor with Query Store · Query Store usage scenarios · Query Store hints · Query Performance Insight · Diagnose high CPU
- Understand and resolve blocking · Deadlocks guide · Analyze deadlocks in Azure SQL

Chapter 12 — CI/CD with SQL Database Projects

EXAM FOCUS

Implement CI/CD by using SQL Database Projects — design and implement a testing strategy, including unit tests and integration tests · create and manage reference/static data in source control · create, build, and validate database models by using SQL Database Projects, including SDK-style models · configure source control for SQL Database Projects · manage branching, pull requests, and conflict resolution · implement secrets management · detect schema drift by using SQL Database Projects · update an SQL database project and deploy changes · design and implement controls for deployment pipelines, including branching policies, triggers in approvals, authentication tables, and code owners.



The life of a schema change: branch, pull request, automatic validation build, reviewed merge, then an approved deployment of the built .dacpac.

12.1 SQL database projects in one page

A **SQL database project** is your database written as files. Each table, view, procedure, and function is a `.sql` file containing a declarative `CREATE` statement that describes the **end state** you want. You never write `ALTER` scripts by hand.

Step	Command	Result
Create	`dotnet new sqlproj -n MyDatabaseProject` (templates: `dotnet new install Microsoft.Build.Sql.Templates`)	A `.sqlproj` plus folders of `.sql` files
Build	`dotnet build MyDatabaseProject.sqlproj`	**Validates** the model and produces a **`.dacpac`** in `bin/Debug` (or `bin/Release` with `-c Release`)
Publish	`SqlPackage /Action:Publish` or `azure/sql-action`	Compares the `.dacpac` with the target database and runs only the needed `CREATE` / `ALTER` / `DROP` statements

> *"When a SQL project is built, the relationships between objects are validated. For example, a view definition can't contain a table or columns that don't exist in the SQL project."*

The **target platform** (`<DSP>` in the `.sqlproj`, for example `…SqlAzureV12DatabaseSchemaProvider` for Azure SQL Database or `…Sql170DatabaseSchemaProvider` for SQL Server 2025) is used *"during the build process to validate that the functions and T-SQL syntax exists in that version of SQL."* Errors fail the build; warnings (such as inconsistent casing) don't. Deployment is idempotent: *"Deploy the same .dacpac five times and the fifth run changes nothing."*

SDK-style projects (the `Microsoft.Build.Sql` SDK) are the modern format: cross-platform .NET 8+ builds, **NuGet package references**, and automatic inclusion of every `.sql` file in the folder. They are GA in VS Code (the original format is what Visual Studio's SSDT uses).

```
<Project DefaultTargets="Build">
  <Sdk Name="Microsoft.Build.Sql" Version="2.2.0" />
  <PropertyGroup>
    <Name>SalesDB</Name>
    <DSP>Microsoft.Data.Tools.Schema.Sql.SqlAzureV12DatabaseSchemaProvider</DSP>
  </PropertyGroup>
  <ItemGroup>
    <PackageReference Include="Microsoft.SqlServer.Dacpacs.Azure.Master" Version="170.0.1"
/>
  </ItemGroup>
</Project>
```

> **PLAIN ENGLISH**
> A `.dacpac` (data-tier application package) is a single file that describes the whole database schema — like a compiled blueprint. **SqlPackage** is the command-line tool that deploys, compares, scripts, and extracts `.dacpac` files.

12.2 Build validation, references, and system objects

> *"By default, SQL projects don't include system objects in the database model, which can lead to validation errors if your project contains references to system objects. To resolve these validation errors, you would include a database reference to the master.dacpac for the target platform of your project."*

Unresolved-reference build errors are **SQL71501 / SQL71502**. The fix is a reference to the **right platform's** system-objects package:

Package	System objects for
`Microsoft.SqlServer.Dacpacs.Azure.Master`	**Azure SQL Database**
`Microsoft.SqlServer.Dacpacs.Master`	SQL Server (versioned by release, e.g. 160.x = SQL Server 2022)
`Microsoft.SqlServer.Dacpacs.Msdb`	SQL Server msdb
`Microsoft.SqlServer.Dacpacs.DbFabric`	SQL database in Fabric

```
dotnet add Database.sqlproj package Microsoft.SqlServer.Dacpacs.Azure.Master
```

An **artifact reference** to the platform's `master.dacpac` file also fixes the error (in SDK-style and original projects), but Microsoft advises against it for new SDK-style work: *"Don't use direct references to a .dacpac artifact file for new development in SDK-style projects. Instead, use NuGet package references"* — the file path points into a Visual Studio install folder that a CI runner doesn't have.

COMMON EXAM TRAP

A Yes/No series on unresolved system objects in an Azure SQL Database project:

- Add `Microsoft.SqlServer.Dacpacs.Azure.Master` → **Yes** (the recommended fix).
- Add an artifact reference to the Azure SQL Database `master.dacpac` → **Yes** (works; just not the preferred method).
- Add `Microsoft.SqlServer.Dacpacs.Master` → **No** (that is SQL Server's master — the wrong platform).
- Run `dotnet build -bl -flp:v=diag` → **No** (binary and diagnostic **logging** only; it changes nothing in the model).

12.3 Source control and reference data

Commit the `.sqlproj`, the `.sql` files (organised by schema and object type), and the pre/post-deployment scripts. **Don't commit** `bin/`, `obj/`, or `*.dacpac` — *"The .dacpac gets recreated from the project on every CI/CD build."*

Reference (static) data — status codes, regions, settings — isn't part of the schema model. It goes in the **post-deployment script**:

> *"Pre/post-deployment scripts are included in the .dacpac but they aren't compiled into or validated with database object model … A post-deployment script is executed after the deployment plan completes."*

The script runs on **every** deployment, including every automatic re-run of a pipeline, so it must be **idempotent** (running it twice gives the same result as running it once). Plain `INSERT`s would create duplicates or fail with key violations; use `MERGE` (or `IF NOT EXISTS`) matched on the **business key**:

```
-- PostDeploy.sql
MERGE dbo.OrderStatuses AS target
USING (VALUES (1, N'Pending'), (2, N'Processing'), (3, N'Shipped')) AS source (StatusID,
StatusName)
   ON target.StatusID = source.StatusID
WHEN MATCHED AND target.StatusName <> source.StatusName THEN
     UPDATE SET StatusName = source.StatusName
WHEN NOT MATCHED BY TARGET THEN
     INSERT (StatusID, StatusName) VALUES (source.StatusID, source.StatusName);
```

A project can have only **one** pre-deployment and **one** post-deployment script; use SQLCMD `:r .\Scripts\seed-regions.sql` includes to split the work (and exclude included files from the build with `<Build Remove="…" />`). If the key column is an `IDENTITY`, match on a natural key such as a code — the identity value can't tell you whether a row already exists.

> **SCENARIO PATTERN**
> "Deploy reference data in the same CI/CD process; reruns must produce the same outcome and no duplicates" → **a post-deployment script with `IF NOT EXISTS` or `MERGE` logic**. Restoring a backup after each deployment destroys production data; GitHub secrets store credentials, not rows.

12.4 Branching, pull requests, and conflicts

The workflow: a short-lived **feature branch** for each change → a **pull request (PR)** into `main` → an automatic validation build → review → merge. *"Always keep main in a deployable state."*

```
git checkout feature/db1-add-staticdata      # be on the feature branch
git fetch origin                             # download new commits; your branches are
unchanged
git merge origin/main                        # bring main's latest changes into the feature
branch
gh pr create --title "Feature Update: DB1" --body "Adds static data" \
             --head feature/db1-add-staticdata --base main --repo contoso/DB1 --web
```

Command	Does
`git fetch origin`	Updates remote-tracking branches such as `origin/main`; *"local branches remain unchanged"*
`git merge origin/main`	Merges `origin/main` into the **current** branch
`git pull origin main`	`fetch` + `merge` in one step
`gh pr create`	Opens a PR: `--head` = the source branch, `--base` = the target branch (*"The branch into which you want your code merged"*), `--web` = finish in the browser
`gh pr merge`	Merges an **existing** PR (`--squash`, `--rebase`, `--auto`)
`gh pr ready`	Marks a draft PR *"ready for review"*

Merge conflicts: Git marks the clash with `<<<<<<<`, `=======`, and `>>>>>>>`. Edit the `.sql` file to keep both intended changes, remove the markers, then `git add` and `git commit`. Then **rebuild**: *"A clean text merge doesn't guarantee a valid schema"* — for example, one branch renames a column while another adds a procedure that uses the old name. Note that GitHub *"Workflows will not run on pull_request activity if the pull request has a merge conflict."*

12.5 Pipelines with GitHub Actions, and secrets

Triggers — when does a workflow run?

You want	Use
Validate every PR **before** it merges into `main`	`on: pull_request: branches: [ "main" ]`
Deploy **after** a merge into `main`	`on: push: branches: [ "main" ]` (a merge is a push)
A manual **Run workflow** button	`on: workflow_dispatch`
A nightly drift check	`on: schedule` (cron)
Only when certain files change	`paths:` / `paths-ignore:`

There is no `merge` event. For a PR, `github.ref` is `refs/pull/<n>/merge`; for a push to main it is `refs/heads/main`.

```yaml
name: Validate SQL Project
on:
  pull_request:
    branches: [ "main" ]          # runs BEFORE the merge; make it a required status check
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with: { dotnet-version: 8.0.x }
      - name: Build (validate) in Release
        run: dotnet build Database.sqlproj -c Release    # output:
bin/Release/Database.dacpac
```

COMMON EXAM TRAP

- `push` **validates too late** — it fires after commits land on `main`. Validating every PR before merge needs `pull_request`.
- `-c Release` **moves the output** to `bin/Release/…dacpac`. A later step that still points to `bin/Debug/…dacpac` fails on a fresh runner. `-c` *"Defines the build configuration. The default for most projects is Debug."* There is no "Production" configuration.
- `dotnet pack` makes a NuGet package; `dotnet publish` doesn't deploy a database — SQL projects deploy with SqlPackage or `azure/sql-action`.
- `needs:` **and** `if:`**.** `needs: build-and-deploy` runs a job only after that job succeeds; `if: github.ref == 'refs/heads/main'` is true on a push to main and false on a PR run.

Secrets management:

Approach	How	Notes
GitHub secrets	`${{ secrets.SQL_CONNECTION_STRING }}` (repository, environment, or organization level)	Masked in logs. **Environment** secrets are released only after the environment's required reviewers approve. `env:` values and `${{ vars.X }}` are **not** secret
OIDC / workload identity federation (Azure trusts GitHub's short-lived OpenID Connect token, so no secret is stored)	`permissions: id-token: write` **plus** an `azure/login` step with `client-id`, `tenant-id`, `subscription-id`, and a federated credential in Entra ID	No stored password at all. `id-token: write` by itself only *allows* the job to request a token — it doesn't sign in
Azure Key Vault	Sign in with OIDC, then read secrets with `az keyvault secret show` (or the Azure DevOps Key Vault task)	Central secret store

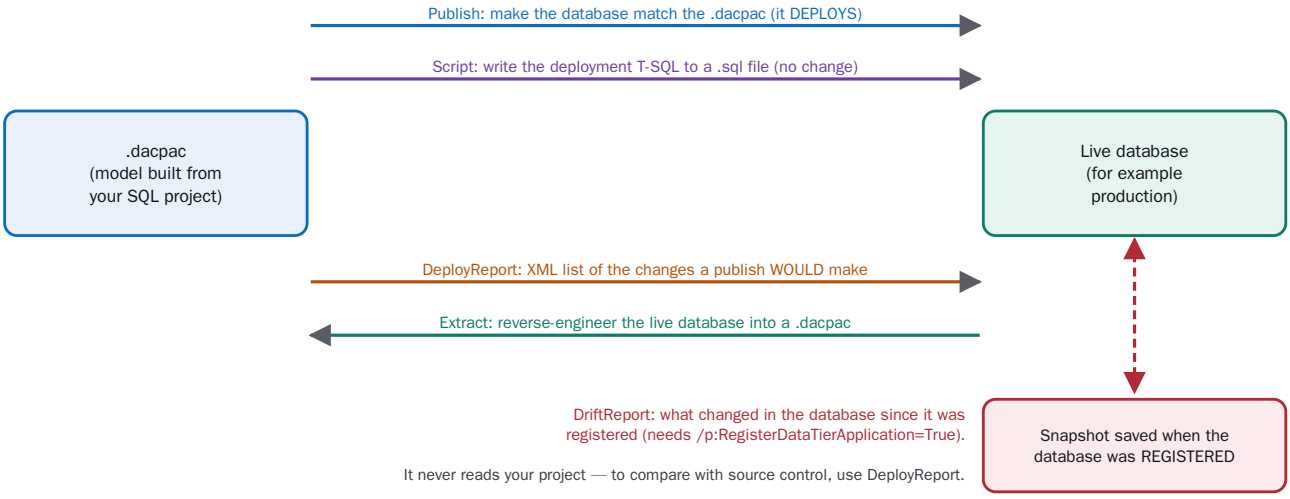
```
permissions:
  id-token: write        # allow requesting a GitHub OIDC token
  contents: read
steps:
  - uses: azure/login@v2
    with:
      client-id:       ${{ secrets.AZURE_CLIENT_ID }}
      tenant-id:       ${{ secrets.AZURE_TENANT_ID }}
      subscription-id: ${{ secrets.AZURE_SUBSCRIPTION_ID }}
```

SCENARIO PATTERN

A workflow has `permissions: id-token: write`, uses `connection-string: ${{ secrets.AZURE_SQL_CONNECTION_STRING }}` with `azure/sql-action`, has **no** `azure/login` step, and the server uses mixed authentication → it does **not** rely on a Microsoft Entra workload identity (the secret holds a SQL login). If its `on:` block includes `workflow_dispatch:`, it **can** be run manually without a commit.

12.6 Detecting schema drift

Drift is the gap between what the project defines and what actually exists in the live database — for example after a hotfix applied directly in production.



Know each SqlPackage action by its direction and output. DeployReport compares your build with a live database; DriftReport compares a live database with its own registration.

SqlPackage action	Compares	Output
DeployReport	Your `.dacpac` (the source-control model) **vs** the live database	*"an XML report of the changes that would be made by a publish action"* — every Create/Alter/Drop is a difference
DriftReport	The live database **vs** the snapshot taken when it was last **registered** (`/p:RegisterDataTierApplication=True`, off by default)	*"an XML report of the changes that have been made to the registered database since it was last registered"* — takes **no** `.dacpac`
Script	`.dacpac` vs live database	The T-SQL deployment script (for review or approval)
Extract	Live database → `.dacpac` or a folder of `.sql` files	Reverse-engineers the database (pair with `git status` to count drifted files)
Publish	`.dacpac` → live database	Deploys the changes

```
sqlpackage /Action:DeployReport /SourceFile:bin/Release/MyProject.dacpac \
           /TargetConnectionString:"{connection string}" /OutputPath:report.xml
```

Schema compare in VS Code or Visual Studio does the same interactively (database → project to find drift; project → database to preview a deployment); save the settings as an `.scmp` file in the repo. Useful publish properties: `BlockWhenDriftDetected` (default True, applies to registered databases), `BlockOnPossibleDataLoss` (default True), `DropObjectsNotInSource` (default False — set True only if the project is the single source of truth, because drifted objects get dropped).

> **COMMON EXAM TRAP**
>
> "Generate a report that shows whether the **production schema has diverged from the model in source control**" → **DeployReport**. **DriftReport** sounds right but never reads your project; it only works on a registered database and reports changes since the registration.

12.7 Deploying changes

```
- name: Deploy dacpac
  uses: azure/sql-action@v2
  with:
    connection-string: ${{ secrets.SQL_CONNECTION_STRING }}   # from repository secrets,
masked
    path: './bin/Release/Database.dacpac'
    action: 'publish'                                          # Publish | Script |
DeployReport | DriftReport
```

`azure/sql-action` runs SqlPackage for `.dacpac` and `.sqlproj` inputs (it builds an SDK-style `.sqlproj` for you; pass build options with the optional `build-arguments` input, for example `'-c Release'`) and go-sqlcmd for `.sql` scripts. If the server doesn't allow Azure services, add an `azure/login` step first so the

action can open a temporary firewall rule. SqlPackage itself (`dotnet tool install -g microsoft.sqlpackage`) is no longer preinstalled on `ubuntu-latest` runners. In Azure DevOps the equivalent task is `SqlAzureDacpacDeployment` (Windows agent) or SqlPackage directly.

> **COMMON EXAM TRAP**
>
> The `extract` action goes the wrong way (database → dacpac) and isn't a sql-action option; `env.SQL_CONNECTION_STRING` is plain text typed into the YAML, not a repository secret; passing a connection string to `dotnet build` deploys nothing.

12.8 Pipeline controls: branch policies, approvals, code owners

Control	Where	What it does
Branch protection / rulesets on `main`	Repository settings	Require a PR, required reviewers, **required status checks** (the validation build), code-owner review, conversation resolution; block force pushes
CODEOWNERS file	`.github/CODEOWNERS` (or root, or `docs/`)	Automatically requests reviews from owners of matching paths, e.g. `*.sql @contoso/db-team` (teams are `@org/team`). Enforced only when branch protection requires code-owner review
Environments	`environment: production` on the deploy job	**Required reviewers** (up to six; one approval releases the job), wait timers, allowed branches, and environment-only secrets
Authentication	OIDC or secrets (12.5)	No credentials in YAML; short-lived tokens
Azure DevOps equivalents	Branch policies	Minimum reviewers, **build validation**, automatically included reviewers, environment approvals and checks

> **KNOW THE DIFFERENCE**
>
> **Required reviewers on a branch vs on an environment.** Branch protection reviewers gate the **merge**; environment reviewers gate the **deployment job**. A complete pipeline uses both.

12.9 Testing strategy

Layer	Proves	How
Build validation	The schema is valid (references, syntax for the target platform)	`dotnet build` on every PR
Unit tests	One procedure or function returns the right result for known inputs	SQL Server unit tests (SSDT) in a C# test project: pre-test (setup), test, post-test (cleanup), with **test conditions** such as Row Count, Scalar Value, Empty ResultSet, Expected Schema
Integration tests	A multi-step scenario works on a deployed database	Run against a **dedicated test database**, reset to a known state each run

The training's pipeline order is build → deploy to a test database → run tests (`dotnet test` in GitHub Actions, `VSTest@2` in Azure DevOps); *"When a test fails, the pipeline stops and the change doesn't reach staging or production."* A **negative test** (the procedure should raise an error) uses the `[ExpectedSqlException(Severity = 16, …)]` attribute. Remove the default *Inconclusive* condition from new tests, and never test against production.

12.10 Chapter 12 self-check

1. What does `dotnet build` do to a SQL project, and what does it produce?
2. An Azure SQL Database project fails with SQL71501 on `sys.objects`. What do you add?
3. Why must a post-deployment seed script be idempotent?
4. Which two commands update a feature branch with the latest `main` (before creating a PR)?
5. Which trigger validates PRs before merge, and which gives a manual run button?
6. Which SqlPackage action shows how production differs from your project?
7. What does `permissions: id-token: write` do on its own?

Answers: 1) It validates the model (references and syntax for the target platform) and produces a `.dacpac`. 2) A package reference to `Microsoft.SqlServer.Dacpacs.Azure.Master`. 3) It runs on every deployment, including reruns, so it must not create duplicates. 4) `git fetch origin` and `git merge origin/main` (or `git pull origin main`). 5) `pull_request`; `workflow_dispatch`. 6) DeployReport. 7) It only allows the job to request an OIDC token; you still need `azure/login` to sign in.

12.11 Learn more

- Training: Implement CI/CD by using SQL database projects — build and validate, source control and reference data, branching and PRs, schema drift, pipelines, testing
- SQL projects: What are SQL database projects? · System objects · Package references · Pre/post-deployment scripts · Automation · Troubleshoot builds
- SqlPackage: Overview · Publish · DeployReport and DriftReport · Extract

- GitHub: Connect to Azure SQL with GitHub Actions · Azure/sql-action · OpenID Connect from GitHub Actions · Events that trigger workflows · About code owners · gh pr create

---

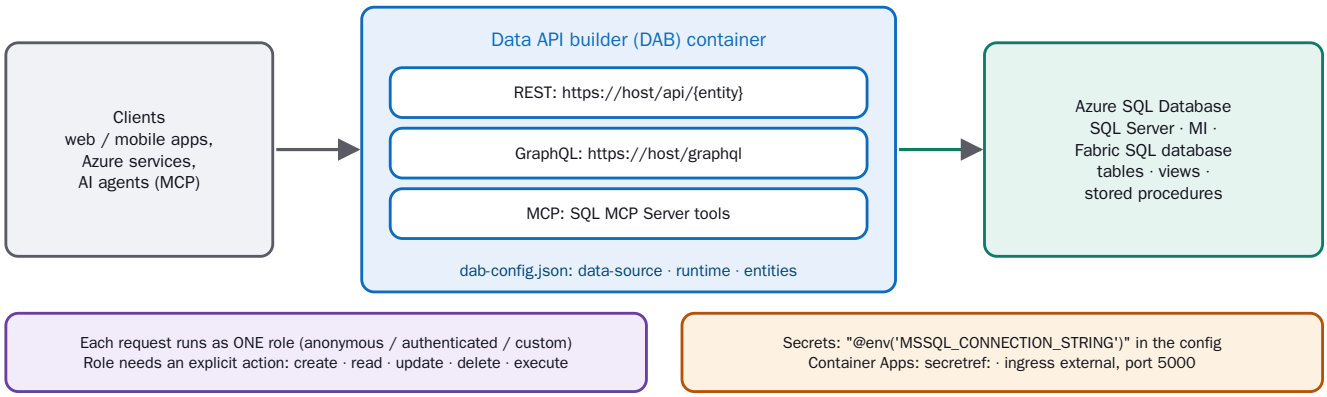
Chapter 13 — Integrate SQL solutions with Azure services

EXAM FOCUS

Integrate SQL solutions with Azure services — create configuration files for Data API builder (DAB) · configure entities for REST and GraphQL, including data caching, pagination, searching, and filtering · configure REST or GraphQL endpoints · expose database objects, stored procedures, and views, including GraphQL relationships · configure and implement DAB deployment · recommend Azure Monitor configurations, including Application Insights and Log Analytics · handle changes by using change event streaming (CES), change data capture (CDC), Change Tracking, Azure Functions with SQL trigger binding, or Azure Logic Apps.

13.1 Data API builder (DAB): what and why

Data API builder is a free, open-source engine that reads **one JSON file** and serves **REST**, **GraphQL**, and (since version 1.7) **MCP** endpoints over your database — no API code to write. The file says how to connect, which tables, views, and procedures to expose (the **entities**), and which **roles** may do what.



One container, one config file, three kinds of endpoint. Every request runs as exactly one role, and each role needs an explicit action on each entity.

13.2 The configuration file

```
dab init --database-type mssql \
         --connection-string "@env('MSSQL_CONNECTION_STRING')" \
         --host-mode Production --config dab-config.json
dab add Product --source dbo.Products --permissions "anonymous:read"
dab validate          # checks the file without starting; use it as a CI gate
dab start
```

```
{
  "$schema": "https://github.com/Azure/data-api-
builder/releases/latest/download/dab.draft.schema.json",
  "data-source": {
    "database-type": "mssql",
    "connection-string": "@env('MSSQL_CONNECTION_STRING')",
    "options": { "set-session-context": true }
  },
  "runtime": {
    "rest":    { "enabled": true, "path": "/api" },
    "graphql": { "enabled": true, "path": "/graphql", "allow-introspection": false },
    "mcp":     { "enabled": true, "path": "/mcp" },
    "host":    { "mode": "production",
                "authentication": { "provider": "EntraId",
                                    "jwt": { "audience": "api://<app-id>",
                                             "issuer":
"https://login.microsoftonline.com/<tenant-id>/v2.0" } } },
    "cache":   { "enabled": true, "ttl-seconds": 5 }
  },
  "entities": { }
}
```

Setting	Fact
`database-type: mssql`	Covers SQL Server, Azure SQL Database, Azure SQL MI, and SQL database in Fabric
`@env('NAME')`	*"At configuration load time, DAB scans string values for the @env('VAR_NAME') pattern and replaces the token with the value of the environment variable."* The quotes are required; `@akv('name')` reads from Key Vault. The connection string never sits in the file
`host.mode`	`production` (default) or `development` (adds Swagger UI and the Nitro GraphQL playground, more logging)
`set-session-context`	Sends the caller's token claims to SQL as `SESSION_CONTEXT`, so RLS and procedures can use them
`dab init`	Overwrites an existing file (no merge)

SCENARIO PATTERN

DAB runs in Azure Container Apps with secrets `MSSQL_CONNECTION_STRING` and `DAB_CONFIG_BASE64` → initialize with **`dab init --database-type mssql --connection-string "@env('MSSQL_CONNECTION_STRING')" --host-mode Production --config dab-config.json`**. Two layers: Container Apps maps a secret to an environment variable with `secretref:` (`MSSQL_CONNECTION_STRING=secretref:mssql-connection-string`), and the DAB file reads the variable with `@env()`. `secretref:` inside `dab init` is just a literal string, and `DAB_CONFIG_BASE64` holds the configuration, not a connection string.

13.3 Entities, permissions, caching, pagination, and filtering

```
"entities": {
  "Procedures": {
    "source": { "object": "dbo.Procedures", "type": "table" },
    "permissions": [ { "role": "anonymous", "actions": [ "read" ] } ]
  },
  "Transactions": {
    "source": { "object": "dbo.Transactions", "type": "table" },
    "cache": { "enabled": true, "ttl-seconds": 60 },
    "permissions": [ { "role": "authenticated", "actions": [ "read", "create" ] } ]
  },
  "UpdateProcedurePatient": {
    "source": { "object": "dbo.sp_UpdateProcedurePatient", "type": "stored-procedure" },
    "rest": { "methods": [ "post" ] },
    "permissions": [ { "role": "authenticated", "actions": [ "execute" ] } ]
  }
}
```

Actions:

Action	SQL operation	REST verb
`create`	**Insert** rows	`POST`
`read`	Select rows	`GET`
`update`	**Modify** existing rows	`PUT` / `PATCH`
`delete`	Delete rows	`DELETE`
`execute`	Run a **stored procedure** (its only action)	`POST` by default (or `GET`)
`*`	All actions for that object type	

Roles: *"Any incoming request, authenticated or not, is assigned to a role."* `anonymous` = no token; `authenticated` = a valid token; a custom role needs the token's roles claim **and** the `X-MS-API-ROLE` header. *"No role has default permissions"* — an entity with no permissions is inaccessible. DAB 2.0 adds role inheritance (named role → authenticated → anonymous).

> **COMMON EXAM TRAP**
> With the entities above: anonymous apps **can** read `Procedures`; authenticated apps can read and **insert** `Transactions` but **not update** them (`create` ≠ `update`); authenticated apps **can** execute the procedure.

Caching: turn it on globally (`runtime.cache.enabled`, default **false**, `ttl-seconds` default **5**) **and** per entity. The in-memory (level 1) cache applies to **REST** reads, works per route and parameters, and is invalidated when data changes through DAB.

Pagination: default page size **100** (`runtime.pagination.default-page-size`), maximum 100,000. REST uses `$first=N` and `$after=<token>` (the response includes a `nextLink`); GraphQL uses `first:` and

`after:` with `hasNextPage` / `endCursor`.

Filtering, sorting, and projection:

```
GET /api/Product?$filter=price gt 100 and category eq 'Bikes'&$orderby=price
desc&$select=id,name,price&$first=20
```

```
query { products(filter: { price: { gt: 100 }, name: { contains: "bike" } }, first: 20) {
items { id name price } } }
```

REST `$filter` uses `eq ne gt ge lt le and or not` (dates unquoted, ISO 8601). GraphQL filters use `eq neq gt gte lt lte contains notContains startsWith endsWith in isNull`. Without `$orderby`, DAB sorts by the primary key. For searching that operators can't express (for example full-text or vector search), expose a **view** or **stored procedure**.

13.4 REST and GraphQL endpoints, stored procedures, views, and relationships

Endpoint	Default	Facts
REST	`https://{host}/api/{entity}`	*"All path components and query parameters are case sensitive"* — use the **entity name** (`Todo`), not the table name (`dbo.todos`). Single item: `/api/Todo/id/42`. Change the prefix with `runtime.rest.path`
GraphQL	`https://{host}/graphql`	One endpoint; all requests are `POST`ed with the query in the body. Queries like `products` and `product_by_pk`, mutations like `createProduct`
MCP	`https://{host}/mcp`	SQL MCP Server tools (8.6)

> **COMMON EXAM TRAP**
>
> `/data-api/…` belonged to **Azure Static Web Apps database connections**, which were retired on 30 November 2025. A DAB container (Container Apps, App Service) serves REST under `/api`.

Views need a key: `"source": { "type": "view", "object": "dbo.vw_BookDetails", "key-fields": ["id"] }` (DAB 2.0 prefers `fields` with `primary-key: true`). Grant `read`, and create/update/delete only if the view is updatable.

Stored procedures use the `execute` action. REST supports `GET` and `POST` (default `POST`); in GraphQL they appear as `execute<Name>` (a mutation by default, or a query with `"graphql": { "operation": "query" }`). Only the **first result set** is returned, with no pagination, filtering, or relationships.

GraphQL relationships let one GraphQL query return a parent and its children:

```
"Book": {
  "source": "dbo.books",
  "relationships": {
    "series":  { "cardinality": "one",  "target.entity": "Series",
                 "source.fields": ["series_id"], "target.fields": ["id"] },
    "authors": { "cardinality": "many", "target.entity": "Author",
                 "linking.object": "dbo.books_authors",
                 "linking.source.fields": ["book_id"], "linking.target.fields":
["author_id"] }
  }
}
```

`one` = many-to-one (returns one object), `many` = one-to-many (returns a list); `linking.object` handles many-to-many through a junction table without exposing it. Relationships are **GraphQL only**, both entities must be in the same config file, view entities don't support relationships, and DAB can infer the fields when a foreign key exists.

13.5 Deploying DAB

DAB ships as the container image `mcr.microsoft.com/azure-databases/data-api-builder`, which listens on port **5000**. The usual home is **Azure Container Apps**:

```
az containerapp create --name dab-app --resource-group rg --environment env \
    --image "mcr.microsoft.com/azure-databases/data-api-builder:latest" \
    --ingress external --target-port 5000 --system-assigned
az containerapp secret set --name dab-app --resource-group rg \
    --secrets mssql-connection-string="$SQL_CONNECTION_STRING"
az containerapp update --name dab-app --resource-group rg \
    --set-env-vars MSSQL_CONNECTION_STRING=secretref:mssql-connection-string
az sql server firewall-rule create --resource-group rg --server sql01 \
    --name AllowAzure --start-ip-address 0.0.0.0 --end-ip-address 0.0.0.0
```

Requirement	Setting
Reachable from the public internet (for example integration tests)	Ingress **external**, target port **5000**. *"Internal"* ingress is reachable only inside the Container Apps environment
The SQL logical server must allow connections from Azure services	Firewall rule `0.0.0.0` – `0.0.0.0` (or private networking)
No password in the connection string	Managed identity: `Authentication=Active Directory Default;` and a database user for the identity
Scales with request volume, can scale to zero	Azure Container Apps
Health check	`/` returns `{"status":"healthy"}`; there is also `/health`

SCENARIO PATTERN

"`/health` returns healthy, but every query of the Products entity fails with a **connection** error; the SQL login is correct; don't change the container app or the DAB file" → on the logical server, **create a firewall rule with start and end IP** `0.0.0.0`. An auto-failover group, `DBCC CHECKDB`, or `FORCE_LAST_GOOD_PLAN` don't open the network path.

13.6 Recommending Azure Monitor configurations

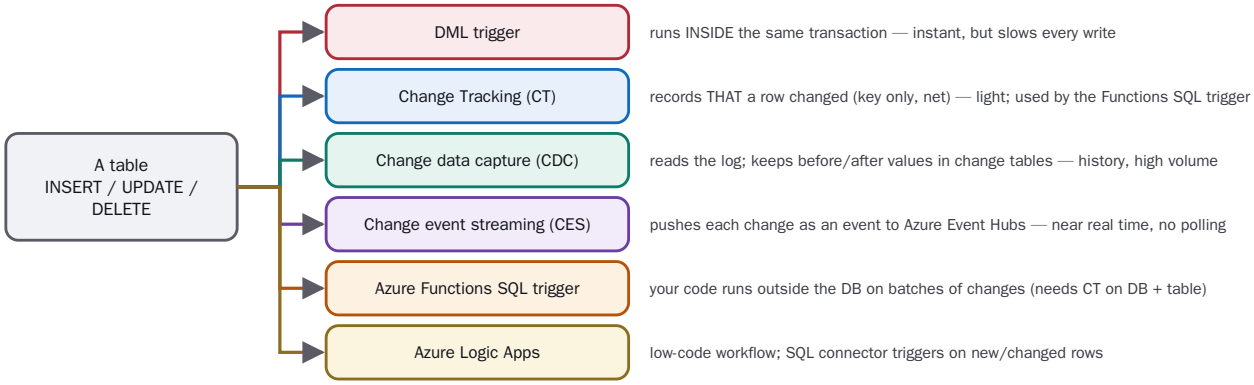
Need	Recommend
API request rates, latency, failures, SQL dependency timings, exceptions for DAB	**Application Insights** — `runtime.telemetry.application-insights` with the **connection string** (via `@env()`), set with `dab add-telemetry`
Vendor-neutral traces and metrics	**OpenTelemetry** export (`dab add-telemetry --otel-…`)
Central log storage and KQL queries across resources	A **Log Analytics** workspace — Container Apps environment logs, DAB's Log Analytics sink, and Azure SQL diagnostic settings
Azure SQL errors, deadlocks, blocks, timeouts, Query Store statistics	**Diagnostic settings** on the database → Log Analytics (the `AzureDiagnostics` table), Event Hubs, or Storage
Alert when p95 latency > 2 seconds	A log-based alert on an Application Insights KQL query using `percentile(duration, 95)`
Alert on CPU, DTU, or failed connections	A metric alert with an action group (email, SMS, webhook, Logic App, Function…)
Deep, estate-wide SQL monitoring dashboards	**Database watcher** (preview), which stores data in Azure Data Explorer or Fabric Real-Time Intelligence — not Log Analytics

```
requests
| summarize P95 = percentile(duration, 95) by bin(timestamp, 1m)
| where P95 > 2000
```

TIP

Use percentiles (p95, p99) for latency goals — an average hides slow outliers. Application Insights needs the **connection string**, not just the old instrumentation key.

13.7 Handling data changes



Six ways to react when rows change. They differ in whether they run inside the transaction, what they record, and whether consumers poll or are pushed.

Feature	What it records	How consumers get changes	Remember
Change Tracking (CT)	*That* a row changed — primary key + operation only (`I` / `U` / `D`), net changes	Poll with `CHANGETABLE(CHANGES …)` since a version	Lightweight and synchronous; **needs a primary key**; enable on the **database**, then on each **table**
Change data capture (CDC)	Full **before and after** values in change tables, read asynchronously from the log	Query `cdc.fn_cdc_get_all_changes_…` or `…_net_changes_…`	History for audit/ETL; SQL Server uses SQL Agent, Azure SQL Database uses a built-in scheduler (DTU tiers need S3+)
Change event streaming (CES) — preview	Each change as a CloudEvent (a standard JSON envelope for events) with current and previous values and the schema	**Pushed** to Azure Event Hubs (or Fabric Eventstream) — no polling	Full recovery model; **can't be enabled on a database with CDC**; data is streamed unmasked; doesn't send rows that existed before it was enabled
Azure Functions SQL trigger	—	Your function runs on **batches** of changes	Built on **Change Tracking**; settings `MaxBatchSize` (default **100**) and `PollingIntervalMs` (default **1000**) live in host.json / app settings
Azure Logic Apps SQL connector	—	Low-code workflow triggers ("When an item is created/modified")	Polling; the managed V2 triggers need an `IDENTITY` column (created) or a `ROWVERSION` column (modified); built-in Standard triggers need Change Tracking
DML trigger	—	Runs **inside** the transaction	Synchronous; adds cost to every write

The Azure Functions SQL trigger setup (an exam favourite):

> *"The Azure SQL trigger uses SQL change tracking functionality to monitor a SQL table for changes and trigger a function when a row is created, updated, or deleted."*

```
ALTER DATABASE [AdventureWorksDB]
SET CHANGE_TRACKING = ON (CHANGE_RETENTION = 2 DAYS, AUTO_CLEANUP = ON);   -- 1. database

ALTER TABLE [dbo].[Employee] ENABLE CHANGE_TRACKING;                       -- 2. table
```

The function's identity also needs `SELECT` and `VIEW CHANGE TRACKING` on the table and rights to create its lease tables in the `az_func` schema (`db_datareader`/`db_datawriter` aren't enough).

COMMON EXAM TRAP

- "Which **two database** configurations?" → change tracking at the **database** level **and** on the **table**. `Sql_Trigger_MaxBatchSize` and `Sql_Trigger_PollingIntervalMs` are **function app** settings — and their defaults (100 changes, 1 second) already meet "within five seconds" and "no more than 100 changes".
- The SQL trigger binding uses **change tracking**, not CDC, and it doesn't use DML or DDL triggers.
- DDL triggers fire on schema changes, never on row inserts or updates.

Enabling CDC (for example to drive embedding refreshes — 14.6):

```
EXEC sys.sp_cdc_enable_db;                                      -- must run first
EXEC sys.sp_cdc_enable_table
     @source_schema        = N'dbo',
     @source_name          = N'Articles',
     @role_name            = NULL,
     @captured_column_list = N'ArticleId, Title, Body',         -- must include the primary
key
     @supports_net_changes = 1;                                 -- needs a primary key or
@index_name

SELECT * FROM cdc.fn_cdc_get_net_changes_dbo_Articles(
         sys.fn_cdc_get_min_lsn('dbo_Articles'), sys.fn_cdc_get_max_lsn(), N'all');
```

(An **LSN** — log sequence number — is a position in the transaction log; the functions return the changes between two positions.)

Enabling change event streaming (preview; needs the full recovery model, and `PREVIEW_FEATURES` on SQL Server 2025):

```
CREATE MASTER KEY ENCRYPTION BY PASSWORD = '<strong password>';
CREATE DATABASE SCOPED CREDENTIAL EventHubCredential WITH IDENTITY = 'Managed Identity';
EXEC sys.sp_enable_event_stream;
EXEC sys.sp_create_event_stream_group
     @stream_group_name      = N'OrdersStream',
     @destination_type       = N'AzureEventHubs',
     @destination_location   = N'<namespace>.servicebus.windows.net:9093/<eventhub>',
     @destination_credential = EventHubCredential;               -- managed identity
preferred
EXEC sys.sp_add_object_to_event_stream_group N'OrdersStream', N'dbo.Orders';
```

More CES facts: no primary key is required; it streams only to Event Hubs **public** endpoints; it silently skips `json`, `vector`, and `xml` columns; and Microsoft Entra authentication isn't available for it in SQL database in Fabric.

Requirement	Choose
Know which rows changed since the last sync; current values are enough	Change Tracking
Run serverless code when rows change	Azure Functions SQL trigger (on Change Tracking)
Before/after history for audit or ETL; "only the net change per row"	CDC (with `@supports_net_changes = 1`)
Push changes in near real time to many consumers or analytics, no polling	CES (preview)
Low-code workflow when a row is created or modified	Logic Apps

13.8 Chapter 13 self-check

1. Write the REST URL for an entity named `Todo` (source `dbo.todos`) served by a DAB container at `app1.example.com`.
2. Which DAB action lets authenticated users insert rows but not change them?
3. What does `"connection-string": "@env('MSSQL_CONNECTION_STRING')"` do?
4. DAB in Container Apps must be reachable from the internet. Which ingress and port?
5. Which telemetry service do you configure in `runtime.telemetry` to track DAB request latency?
6. What must you enable in the database for an Azure Functions SQL trigger?
7. Which change feature pushes events to Event Hubs, and which feature can't be enabled alongside it?

Answers: 1) `https://app1.example.com/api/Todo`. 2) `create`. 3) Replaces the token with the value of the environment variable when the config loads, so no secret is stored in the file. 4) External ingress, target port 5000. 5) Application Insights. 6) Change tracking on the database and on the table. 7) Change event streaming (CES); CDC can't be enabled on the same database.

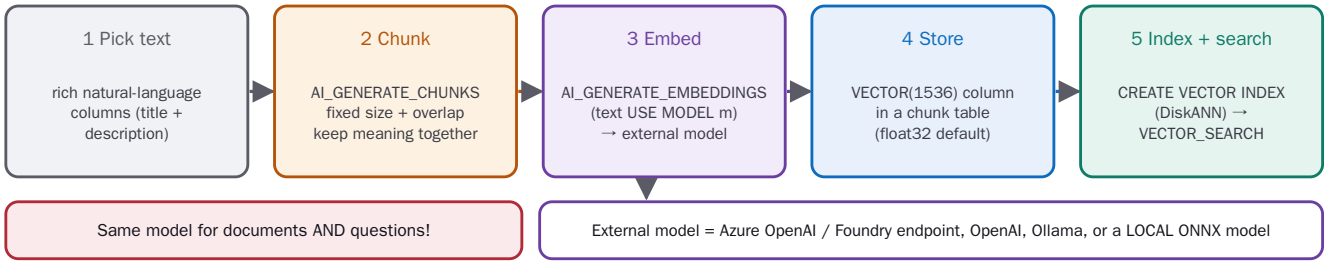
13.9 Learn more

- Training: Integrate SQL solutions with Azure services — configuration files, entities, procedures and views, deployment, Azure Monitor, event-driven patterns
- DAB: Overview · Configuration · Entities · @env function · Authorization · REST · GraphQL relationships · Deploy to Container Apps · Application Insights
- Azure: Container Apps ingress · Container Apps secrets · Azure SQL monitoring with Azure Monitor · Database watcher
- Changes: Azure SQL trigger for Functions · Enable change tracking · CDC in Azure SQL Database · sys.sp_cdc_enable_table · Change event streaming · Logic Apps SQL connector

---

Chapter 14 — Models and embeddings

> **EXAM FOCUS**
>
> **Design and implement models and embeddings** — evaluate external models, including multimodal, multilanguage, sizes, and structured output · create and manage external models · choose an embedding maintenance method, including table triggers, Change Tracking, Azure Functions with SQL trigger binding, Azure Logic Apps, CDC, and Microsoft Foundry · identify which columns to include in embeddings · design and implement chunks for embeddings · generate embeddings.



From text to searchable vectors: pick meaningful text, split it into chunks, embed each chunk with an external model, store the vectors, then index and search them.

14.1 Evaluating models

A RAG solution usually needs **two** models: an **embedding model** (turns text into vectors) and a **chat/generation model** (writes the answer). The training lists four things to evaluate:

Question	What the training says
Modality — text only, or images too?	*"Some models process only text, while others support other inputs such as images or structured data."* GPT-4o-family models are multimodal (text + images)
Languages	*"Multilingual support is important when applications serve users across regions or when stored content spans multiple languages."* The third-generation embedding models are much better than ada-002 on multilingual benchmarks
Size	*"Larger models typically provide stronger reasoning and more nuanced output, but they also consume more tokens and can introduce higher latency and cost. Smaller models might be more suitable for focused tasks such as embedding generation."* Bigger isn't automatically better
Structured output	*"Models that can produce structured output, such as JSON, are easier to integrate into SQL-based workflows."* Structured outputs (`response_format` with a JSON schema) enforce your schema; the older JSON mode only guarantees valid JSON

Also check **token limits** (*"A model might accept a maximum of 8,000 or 128,000 tokens in a single request. This limit constrains how much database content you can include as context"*) and cost (*"model providers*

typically charge based on tokens processed"). The **Microsoft Foundry** model catalog shows capabilities, benchmarks, versions, and lifecycle status.

Embedding model	Dimensions	Notes
`text-embedding-ada-002`	1,536	Older generation
`text-embedding-3-small`	1,536 (can be reduced with the `dimensions` parameter)	Better than ada-002, including multilingual
`text-embedding-3-large`	3,072 (can be reduced)	Most capable; 3,072 exceeds float32's 1,998 limit, so use `vector(3072, float16)` or fewer dimensions

14.2 Creating and managing external models

> *"An external model is a database object that stores metadata about an AI model endpoint. This metadata includes information such as the endpoint URL, authentication details, and model-specific configuration." "External models aren't models that run inside SQL Server." "Creating the external model doesn't invoke the model or generate output."*

```
-- 1. A credential (managed identity: no key stored)
CREATE DATABASE SCOPED CREDENTIAL [https://my-openai.openai.azure.com]
    WITH IDENTITY = 'Managed Identity', SECRET =
'{"resourceid":"https://cognitiveservices.azure.com"}';

-- 2. The external model
CREATE EXTERNAL MODEL MyEmbeddingModel
WITH (
    LOCATION   = 'https://my-openai.openai.azure.com/openai/deployments/text-embedding-3-
small/embeddings?api-version=2024-02-01',
    API_FORMAT = 'Azure OpenAI',          -- or 'OpenAI', 'Ollama', 'ONNX Runtime'
    MODEL_TYPE = EMBEDDINGS,              -- the only accepted value
    MODEL      = 'text-embedding-3-small',
    CREDENTIAL = [https://my-openai.openai.azure.com],
    PARAMETERS = '{"dimensions":1536}'    -- optional JSON added to every request
);

-- 3. Who may use it
GRANT EXECUTE ON EXTERNAL MODEL::MyEmbeddingModel TO AIApplicationUser;

-- Repoint it later; queries that say USE MODEL MyEmbeddingModel don't change
-- (the Azure OpenAI deployment is part of LOCATION — and a new model means re-embedding ALL
rows)
ALTER EXTERNAL MODEL MyEmbeddingModel SET (
    LOCATION = 'https://my-openai.openai.azure.com/openai/deployments/text-embedding-3-
large/embeddings?api-version=2024-02-01',
    MODEL    = 'text-embedding-3-large');
DROP EXTERNAL MODEL MyEmbeddingModel;     -- the credential is NOT dropped
```

Argument	Rule
`LOCATION`	The HTTPS inference URL (Azure OpenAI: `…/openai/deployments/{deployment}/embeddings?api-version=…`), or a **local folder** for ONNX
`API_FORMAT`	`Azure OpenAI`, `OpenAI`, `Ollama`, or `ONNX Runtime`
`MODEL_TYPE`	`EMBEDDINGS` only — chat models are called directly with `sp_invoke_external_rest_endpoint` (16.2)
Permissions	Create/alter needs `CREATE EXTERNAL MODEL` or `ALTER ANY EXTERNAL MODEL`; **using** it needs `EXECUTE ON EXTERNAL MODEL::name`; metadata is in `sys.external_models`

Offline SQL Server 2025 — a local ONNX model. When the instance has **no outbound internet**, the model must run on the server itself (Windows only, a preview feature, and SQL Server Machine Learning Services must be installed):

```
ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON;
EXECUTE sp_configure 'external AI runtimes enabled', 1;
RECONFIGURE WITH OVERRIDE;

CREATE EXTERNAL MODEL LocalEmbeddingModel
WITH (
    LOCATION          = 'C:\onnx_runtime\model\all-MiniLM-L6-v2-onnx',   -- folder with
model.onnx + tokenizer.json
    API_FORMAT        = 'ONNX Runtime',
    MODEL_TYPE        = EMBEDDINGS,
    MODEL             = 'allMiniLM',
    PARAMETERS        = '{"valid":"JSON"}',                              -- placeholder
required on SQL Server 2025
    LOCAL_RUNTIME_PATH = 'C:\onnx_runtime\'                              -- folder with
onnxruntime.dll
);
GRANT EXECUTE ON EXTERNAL MODEL::LocalEmbeddingModel TO AIApplicationUser;
SELECT AI_GENERATE_EMBEDDINGS(N'Test text' USE MODEL LocalEmbeddingModel);
```

SCENARIO PATTERN

- "SQL Server 2025 has **no outbound connectivity**; only `AIApplicationUser` may generate embeddings" → **create an external model with ONNX Runtime and local paths**, and **grant EXECUTE on that external model** to the user. A Foundry REST endpoint needs outbound HTTPS; `CONTROL` on the database is far too much; an audit specification grants nothing.
- "Switch from `text-embedding-ada-002` to `text-embedding-3-small`; security requires managed identities only" → **regenerate all embeddings** and **change the endpoint** (point the model at the new deployment, with a managed-identity credential). The `VECTOR(1536)` column can stay, primary keys are irrelevant, and a firewall rule controls inbound traffic, not outbound model calls.

14.3 Which columns to embed

> *"Embeddings work best when they represent text that carries semantic meaning, such as descriptions, titles, or other free-form text fields."*
>
> *"Columns that store identifiers, numeric values, or operational metadata usually don't add semantic value and should be excluded. Limiting embeddings to meaningful text reduces token usage and improves similarity results."*

Column type	Embed it?	Instead
Descriptions, titles, notes, reviews, incident narratives	**Yes** (alone or concatenated, e.g. `product_name + category + description`)	—
Short category labels	Sometimes, as part of a concatenated string	Often better as a keyword/ `WHERE` filter
IDs, prices, quantities, dates, status codes, audit columns	**No**	Normal columns used in `WHERE`, joins, and sorting — *hybrid* filtering
A location string or address	Usually no	Exact filters or geospatial logic

You can create one embedding per column or concatenate several columns into one string. A documented design tip is to store vectors in a **separate embeddings table**, which makes it easier to measure their size and rebuild them without touching the source data.

EXAMPLE

An ecommerce catalog wants natural-language search with filters on brand and price. Embed `product_name + category + description`. Keep `brand` and `price` as ordinary columns and filter with `WHERE brand = @b AND price BETWEEN @min AND @max` next to the vector search. Putting price in the embedding is **not** required — it changes several times a day (forcing re-embedding) and would only give fuzzy price matching. The new column `VECTOR(1536)` uses **float32** by default.

14.4 Chunking

Embedding models accept limited input (about 8,192 tokens for the Azure OpenAI embedding models), and one vector for a whole long document blurs many topics together. So you split long text into **chunks** and embed each chunk.

> *"Chunks that are too large may exceed token limits or dilute semantic focus. Chunks that are too small may lose important context. The goal is to preserve meaning while keeping chunks efficient to process."*

Strategy	How	Notes
Fixed size + overlap	A fixed number of characters/tokens; neighbouring chunks share some text	Simple and cheap; overlap keeps ideas that cross a boundary
Sentence / paragraph / heading boundaries	Split at natural breaks (variable size)	Keeps meaning together better — "chunking must preserve semantic coherence"
Semantic chunking	Use embeddings to group related content	Best quality, most effort and cost

Microsoft's search guidance suggests starting around **512 tokens (about 2,000 characters) with about 25% overlap**, and appending the document title to chunks from the middle so they keep context.

Chunking in T-SQL — `AI_GENERATE_CHUNKS` (a table-valued function; database compatibility level **170** or higher):

```
INSERT INTO dbo.ProcedureDocumentChunks (DocumentId, ChunkOrder, ChunkText, Embedding)
SELECT d.DocumentId, c.chunk_order, c.chunk,
       AI_GENERATE_EMBEDDINGS(c.chunk USE MODEL MyEmbeddingModel)
FROM dbo.ProcedureDocuments AS d
CROSS APPLY AI_GENERATE_CHUNKS(SOURCE = d.Content, CHUNK_TYPE = FIXED,
                               CHUNK_SIZE = 2000, OVERLAP = 10) AS c;
```

Argument / column	Meaning
`CHUNK_TYPE`	`FIXED` is the only value
`CHUNK_SIZE`	Size in **characters** (required)
`OVERLAP`	**Percentage** (0–50) of the previous chunk repeated in the next one; default 0
Output columns	`chunk`, `chunk_order`, `chunk_offset`, `chunk_length` (and `chunk_set_id` if enabled)

> **SCENARIO PATTERN**
> "Each document has **one** embedding of its whole `NVARCHAR(MAX)` content; RAG retrieval is slow and uses too many tokens" → **chunking**. Embeddings already exist; a small language model changes the generator, not the retrieval; JSON formatting adds tokens.

14.5 Generating embeddings

```
-- Bulk: fill or refresh a vector column
UPDATE dbo.documents
SET embedding = AI_GENERATE_EMBEDDINGS(description USE MODEL MyEmbeddingModel);

-- Query time: embed the user's question with the SAME model
DECLARE @qv vector(1536) = AI_GENERATE_EMBEDDINGS(N'heavy rain, rear-end collision' USE
MODEL MyEmbeddingModel);

-- Override a model parameter at run time
SELECT AI_GENERATE_EMBEDDINGS(description USE MODEL MyEmbeddingModel PARAMETERS
N'{"dimensions":768}')
FROM dbo.documents;
```

`AI_GENERATE_EMBEDDINGS(text USE MODEL name)` returns the vector as a JSON array, which converts automatically to a `vector` column. It needs the `external rest endpoint enabled` option on SQL Server 2025 and Managed Instance (on by default in Azure SQL Database and Fabric) and `EXECUTE` on the model.

Calling the embeddings endpoint yourself works too — remember that the embedding is an **array**:

```
EXECUTE sp_invoke_external_rest_endpoint
    @url = N'https://my-openai.openai.azure.com/openai/deployments/text-embedding-3-
small/embeddings?api-version=2024-08-01-preview',
    @method = 'POST', @credential = [https://my-openai.openai.azure.com],
    @payload = N'{"input":"hello world"}', @response = @response OUTPUT;

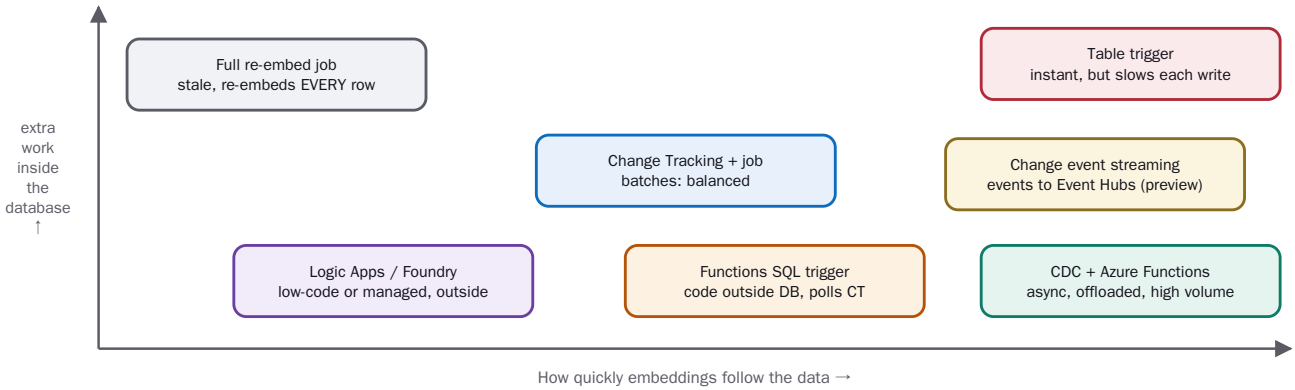
DECLARE @v vector(1536) = CAST(JSON_QUERY(@response, '$.result.data[0].embedding') AS
vector(1536));
```

COMMON EXAM TRAP

Never mix embedding models. *"You can't upgrade between embedding models. To move from text-embedding-ada-002 to text-embedding-3-large, you need to generate new embeddings."* Vectors from different models live in different "spaces", even when the dimension count is the same, and *"In a RAG solution, you embed the user query by using the same embedding model as your chunks."* So when the model changes, the **first** step is to regenerate the existing embeddings. Normalizing old vectors, converting the column to `nvarchar(max)`, or creating a vector index first doesn't help.

14.6 Keeping embeddings fresh: maintenance methods

> *"Generating embeddings is typically not a one-time operation. When source data changes, embeddings may need to be regenerated so they continue to reflect the current state of the data."*



The trade-off: how quickly embeddings follow the data versus how much extra work lands inside the database.

Learn the training unit's one-line description of each method — exam answers often hinge on these exact phrases:

Method	The training unit's description
Table triggers	*"a trigger can mark rows that need new embeddings or initiate regeneration immediately. This approach **reflects changes quickly but adds work to write operations**."*
Change Tracking	*"records that a row changed since a given point in time. A background process can … **process them in batches**. This approach **balances latency and performance**."*
Change data capture (CDC)	*"records detailed information about data changes, including **before and after values** … regenerate embeddings **asynchronously**. This approach is **suitable for high-volume workloads**."*
Azure Functions (SQL trigger binding)	*"allows embedding generation to **run outside the database engine** while still responding to data changes. This approach **offloads work from the database and can scale independently**."*
Azure Logic Apps	*"can **orchestrate** embedding updates as part of a workflow … periodically check for changed rows … This approach is **low-code** and integrates well with other Azure services."*
Change event streaming (CES)	*"streams DML changes directly into **Azure Event Hubs** in near real-time … **without adding work to database transactions** … **supports multiple consumers**."* (Preview)
Microsoft Foundry	*"can be used to **manage and evaluate the models** that generate embeddings … while **another process handles change detection** and database updates."*

Requirement in the question	Best answer
Embeddings updated **every time** content changes, **no nightly batch**, nothing said about CPU or write cost	**Table triggers**
Updated whenever data changes **and minimize CPU** on the database (high volume)	**CDC + an Azure Functions app** — CDC reads changes asynchronously from the log, and the function (for example on a timer) reads the CDC change functions and does the embedding work outside the database. (The Functions SQL trigger *binding* itself uses Change Tracking, not CDC.)
Refresh changed rows in batches, only need to know *which* rows changed	**Change Tracking** + a background job
An Azure Functions **SQL trigger binding** must fire on changes	**Change Tracking** on the database **and** the table (13.7)
Low-code workflow with other systems	**Logic Apps**
Many downstream consumers, near real time, no extra transaction work	**CES** to Event Hubs
Results don't show updates until the next day	The nightly batch is the problem — choose one of the above

COMMON EXAM TRAP

- A trigger that calls `AI_GENERATE_EMBEDDINGS` runs **inside** every write transaction — fast to reflect changes, but the opposite of "minimize CPU". An hourly job that re-embeds **all** rows wastes CPU and model cost on rows that didn't change.
- Switching `VECTOR_SEARCH` to `VECTOR_DISTANCE` changes how you *query*, not how fresh the embeddings are. Fixed-size chunking or a smaller model doesn't change *when* embeddings are refreshed.
- **CDC for embedding maintenance** — capture only what the embedding needs plus the key, and enable net changes (one final row per changed key):

```
EXEC sys.sp_cdc_enable_db;                                    -- database first
EXEC sys.sp_cdc_enable_table
     @source_schema = N'dbo', @source_name = N'Articles', @role_name = NULL,
     @captured_column_list = N'ArticleId, Title, Body',     -- the key must be included
     @supports_net_changes = 1;                               -- needs a primary key or
unique index
```

14.7 Chapter 14 self-check

1. What is an external model, and does creating one call the model?
2. Which permission lets a user call `AI_GENERATE_EMBEDDINGS` with a specific model?
3. Which three columns of a product table would you embed for natural-language search: `product_id`, `name`, `category`, `description`, `price`, `sku`?
4. What does `OVERLAP = 10` mean in `AI_GENERATE_CHUNKS`?
5. You change embedding models. What must you do before serving queries?
6. Which maintenance method "balances latency and performance", and which one "adds work to write operations"?
7. Which path and function extract the embedding from an `sp_invoke_external_rest_endpoint` response?

Answers: 1) A database object holding metadata about a model endpoint (URL, format, credential); no, creating it calls nothing. 2) `GRANT EXECUTE ON EXTERNAL MODEL::name`. 3) `name`, `category`, `description`. 4) 10% of the chunk size is repeated from the previous chunk. 5) Regenerate every stored embedding with the new model. 6) Change Tracking; table triggers. 7) `JSON_QUERY(@response, '$.result.data[0].embedding')` (it is an array).

14.8 Learn more

- Training: Design and implement models and embeddings with SQL — evaluate models, external models, design embeddings, generate and maintain embeddings
- Reference: CREATE EXTERNAL MODEL · ALTER EXTERNAL MODEL · AI_GENERATE_EMBEDDINGS · AI_GENERATE_CHUNKS · vector data type
- Models: Foundry models sold by Azure · Understand embeddings · Structured outputs
- Architecture: RAG chunking phase · RAG generate-embeddings phase · Chunk large documents
- Change detection: About change tracking · About change data capture · Change event streaming

---

Chapter 15 — Intelligent search: full-text, vector, and hybrid

EXAM FOCUS

Design and implement intelligent search — choose from full-text, semantic vector, and hybrid search · implement full-text search · design for vector data, including vector data type, vector indexes, and size · identify when to use vector-related types and functions for semantic searching, including VECTOR_NORMALIZE, VECTOR_DISTANCE, VECTORPROPERTY, and VECTOR_SEARCH · choose between using ANN and KNN for vector search · evaluate vector index types and metrics · implement vector search · implement hybrid search · implement reciprocal rank fusion (RRF) · evaluate performance of vector and hybrid search.

WATCH OUT

Vector search in SQL changed recently. The **latest vector index version** (Azure SQL Database, SQL database in Fabric, Azure SQL MI with the Always-up-to-date policy) supports full `INSERT` / `UPDATE` / `DELETE`, filters *during* the search, and requires the syntax `SELECT TOP (N) WITH APPROXIMATE … ORDER BY distance`. The older `TOP_N` parameter is **deprecated** — but the training module and many exam items still show it. This chapter teaches both and labels them.

15.1 Choosing full-text, vector, or hybrid search

Approach	Finds	Best for	Needs
Full-text search	Words and phrases (and their forms: ride, rides, riding, rode)	Exact names, product codes, error messages, technical terms — *"Use full-text search when users know the specific words they want."*	A full-text index on text columns
Vector (semantic) search	Similar **meaning**, even with no shared words	Natural-language questions, "more like this", different vocabulary — *"compares the mathematical representation of concepts"*	Embeddings in `vector` columns
Hybrid search	Both, merged into one ranked list	When you can't predict how users search, and for RAG retrieval	Both of the above

Trade-offs from the training: full-text gives fewer, very precise results; vector search finds more related results (better recall); hybrid *"typically improves recall compared to either approach alone, often with minimal precision loss"* but *"takes longer than either one alone."*

EXAMPLE

A user types **"XR-500 portable cooler"**. Full-text finds the exact model number "XR-500"; vector search finds "insulated travel fridge" items that mean the same as "portable cooler". Hybrid search returns both kinds, ranked together.

COMMON EXAM TRAP

"Semantic search" can mean three different things on the exam: **semantic vector search** (embeddings, this chapter), SQL Server's older **Statistical Semantic Search** (`SEMANTICKEYPHRASETABLE`, `SEMANTICSIMILARITYTABLE` — key phrases, SQL Server only, *not* embeddings), and Azure AI Search's **semantic ranker** (`queryType: "semantic"`, 15.9). Also, `FREETEXT` "matches meaning" only in the full-text sense (word forms and synonyms), not with embeddings.

15.2 Full-text search

Setup has three parts: a **full-text catalog** (a logical container), a **unique key index** on the table (unique, single-column, non-nullable — an integer is best), and a **full-text index** on the text columns that points to that key index.

```
CREATE FULLTEXT CATALOG ftCatalog AS DEFAULT;

CREATE FULLTEXT INDEX ON dbo.Products (Name, Description LANGUAGE 1033)
    KEY INDEX PK_Products                -- unique, single-column, non-null
    WITH STOPLIST = SYSTEM;              -- CHANGE_TRACKING AUTO is the default
```

Facts: only **one full-text index per table**; indexable types are `char`, `varchar`, `nchar`, `nvarchar`, `text`, `ntext`, `image`, `xml`, and `varbinary(max)` — **not** `vector` or numbers; **stopwords** (very common words like "the") aren't indexed; with `CHANGE_TRACKING AUTO` changes flow in automatically but *"might not be reflected immediately"*. Full-text is available in SQL Server, Azure SQL Database, and Azure SQL MI, and in **preview** in SQL database in Fabric.

Querying — two predicates and two rowset functions:

	`CONTAINS`	`FREETEXT`	`CONTAINSTABLE`	`FREETEXTTABLE`
Used in	`WHERE`	`WHERE`	`FROM` (join back on `[KEY]`)	`FROM` (join back on `[KEY]`)
Returns	true / false	true / false	rows with `KEY` and `RANK`	rows with `KEY` and `RANK`
Matching	Precise: words, phrases, prefixes, `NEAR`, `FORMSOF`, weights	Free text: word-broken, stemmed, thesaurus applied automatically	Same as CONTAINS	Same as FREETEXT (ranked with BM25 — a classic keyword-relevance formula based on term frequency, word rarity, and document length)

```
WHERE CONTAINS(Description, '"mountain bike"')                  -- exact phrase
WHERE CONTAINS(Description, '"light*"')                         -- prefix: needs the double
quotes
WHERE CONTAINS(Description, 'FORMSOF(INFLECTIONAL, ride)')      -- ride, rides, riding, rode
WHERE CONTAINS(Comments, 'NEAR((bike, control), 10, TRUE)')     -- within 10 words, in order
WHERE FREETEXT(Description, 'riding bikes in the rain')         -- plain-language search
text

-- Ranked results: join the rowset function back to the table on [KEY]
SELECT p.ProductID, p.Name, ft.[RANK]
FROM dbo.Products AS p
INNER JOIN CONTAINSTABLE(dbo.Products, Description, '"waterproof" AND "jacket"') AS ft
        ON p.ProductID = ft.[KEY]
ORDER BY ft.[RANK] DESC;
```

RANK is *"a value (from 0 through 1000) for each row indicating how well a row matched"* — **higher is better** — but it is only a *relative* order inside one query: *"The rank value doesn't hold any significance across queries."*

> **TIP**
> Use `CONTAINS` / `CONTAINSTABLE` for a precise search condition (a `@keywords` parameter, phrases, prefixes). Use `FREETEXT` / `FREETEXTTABLE` for raw user text. Only the `…TABLE` functions give you a `RANK` to combine with vector results.

15.3 Designing for vector data

```
CREATE TABLE dbo.ProductChunks (
    ChunkId     int IDENTITY PRIMARY KEY CLUSTERED,
    ProductID   int NOT NULL,
    Category    nvarchar(50) NOT NULL,
    ChunkText   nvarchar(max) NOT NULL,
    Embedding   vector(1536) NOT NULL                -- float32 by default
);

INSERT dbo.ProductChunks (ProductID, Category, ChunkText, Embedding)
VALUES (1, N'Footwear', N'Lightweight waterproof hiking boot…', '[0.012, -0.044, …]');   --
a JSON array casts to vector
```

Fact	Detail
Declaration	`VECTOR(dimensions [, base_type])` — dimensions must match the embedding model (1,536 for `text-embedding-3-small` and `ada-002`)
Base type	**float32 by default** (4 bytes per number); `float16` halves the size (preview on SQL Server 2025)
Maximum dimensions	1,998 for float32; 3,996 for float16
Size	About 4 bytes × dimensions + 8 bytes — a 1,536-dimension vector is about **6 KB** per row
Input / output	Stored in a compact binary format, shown as a JSON array; converts only to/from string and `json` types
Not allowed	B-tree or columnstore indexes on the column (it can be an `INCLUDE` column), PK/FK/UNIQUE/CHECK/DEFAULT constraints, math operators, memory-optimized tables, Always Encrypted
Availability	`vector` type and vector functions: **GA** on SQL Server 2025, Azure SQL Database, Azure SQL MI (2025 / Always-up-to-date), and SQL database in Fabric

COMMON EXAM TRAP

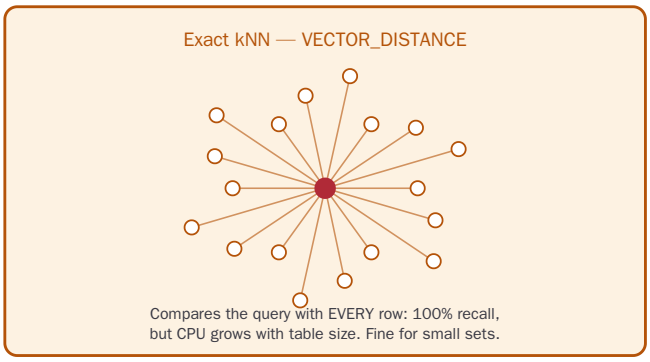
Storing embeddings as `varbinary` or `nvarchar(max)` "to save space" loses everything: the vector functions need `vector` inputs, and `CREATE VECTOR INDEX` needs a `vector` column (*"It must be of vector type."*).

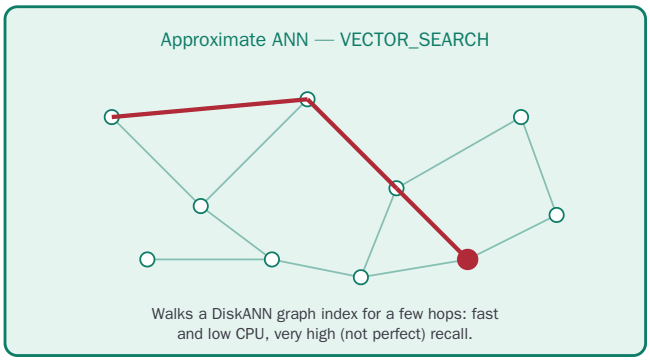
15.4 The vector functions

Function	Returns	Use it to
`VECTOR_DISTANCE('cosine', v1, v2)`	A float distance (smaller = more similar)	Exact search, thresholds, scoring a small candidate set. *"Vector distance is always exact and doesn't use any vector index, even if available."*
`VECTOR_SEARCH(TABLE = …, COLUMN = …, SIMILAR_TO = …, METRIC = …)`	A table: all source columns **plus** `distance`	**Approximate** nearest-neighbour search through a vector index
`VECTOR_NORM(v, 'norm2')`	A float — the vector's length	Check whether vectors are unit length
`VECTOR_NORMALIZE(v, 'norm2')`	A vector with the same direction and length 1	Make dot product and cosine agree; fix un-normalized models
`VECTORPROPERTY(v, 'Dimensions')` / `'BaseType'`	Metadata (an int or a name)	Debug dimension mismatches — **never** similarity

Norm types are `'norm1'`, `'norm2'` (the usual Euclidean length), and `'norminf'`. Example: `VECTOR_NORMALIZE('[3, 4, 0]', 'norm2')` → `[0.6, 0.8, 0.0]`, and `VECTOR_NORM` of the same vector → `5.0`.

15.5 kNN vs ANN, vector indexes, and metrics





Exact kNN compares the question with every row. Approximate ANN walks a DiskANN graph and checks only a few rows — much faster, with recall close to 1.

	kNN — exact	ANN — approximate
How	`VECTOR_DISTANCE` + `TOP` + `ORDER BY distance`	`CREATE VECTOR INDEX` + `VECTOR_SEARCH`
Accuracy	Perfect (recall = 1)	Very high, not perfect (recall close to 1)
Cost	*"a complete scan of the table and significant CPU usage"*	Low latency, low CPU
Use when	*"less than 50,000 vectors as a general recommendation"* — or your `WHERE` cuts the candidates to that	Hundreds of thousands to millions of vectors, speed matters
Platforms	GA everywhere	GA in Azure SQL Database, Fabric, MI (Always-up-to-date); **preview** on SQL Server 2025 (`PREVIEW_FEATURES = ON`)

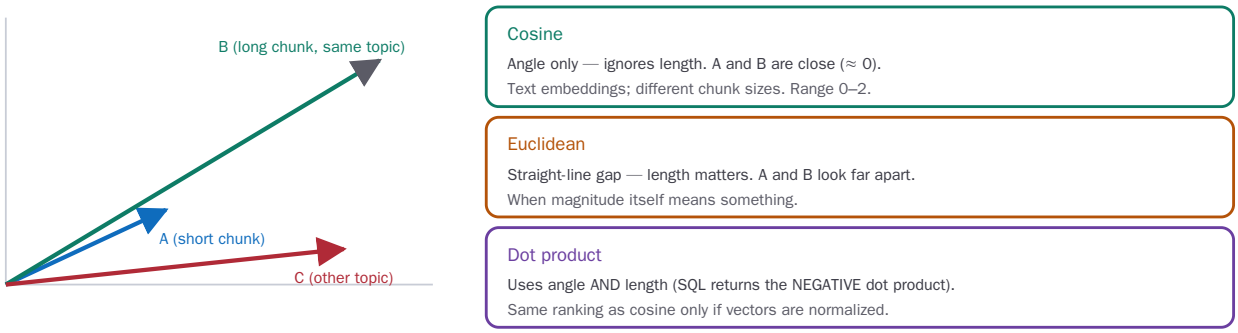
> **PLAIN ENGLISH**
>
> **Recall** measures how many of the *true* nearest neighbours (the exact kNN answer) the approximate search found. A recall of 1 means no approximation. Because embeddings are themselves approximations of meaning, *"using ANN doesn't significantly affect the results, provided the recall is close to 1."*

Creating a vector index:

```
CREATE VECTOR INDEX vix_ProductChunks_Embedding
ON dbo.ProductChunks (Embedding)                    -- must be a vector column
WITH (METRIC = 'cosine', TYPE = 'DiskANN');         -- DiskANN is the only (and default)
type
```

- **DiskANN** is a graph-based index that *"efficiently uses SSDs and minimal memory"* while keeping high throughput and low latency. A vector index returns **approximate** results.
- The latest index version needs **at least 100 rows** with non-NULL vectors; it can't be partitioned, the table can't be `TRUNCATE`d while it exists, and it isn't deployed by dacpac.
- **Older index versions** (and SQL Server 2025 preview) make the table **read-only** after the index is created (in Azure SQL Database and Fabric, `ALLOW_STALE_VECTOR_INDEX` allows DML if you accept stale results) and filter only *after* the search; the latest version allows full DML and filters *during* the search.
- The training module (and earlier index versions) require a **single-column `int` clustered primary key** on the table — design for it.
- `VECTOR_SEARCH` works only on base tables (not views or temporary tables) and can't be used inside a view. The table hint `WITH (FORCE_ANN_ONLY)` forces the index to be used.

The metric must match. *"An ANN (Approximate Nearest Neighbor) index is used only if a matching ANN index, with the same metric and on the same column, is found. If there are no compatible ANN indexes, a warning is raised and the kNN (k-nearest neighbor) algorithm is used."*



Cosine looks only at direction, so a short chunk and a long chunk about the same topic stay close. Euclidean and dot product are affected by length.

Metric	Range	Choose when
`cosine`	0 (same direction) to 2 (opposite)	Text embeddings (Azure OpenAI models are built for it); vectors of **different magnitudes**, for example from different chunk sizes
`dot`	Any value — SQL returns the **negative** dot product so smaller = more similar	Vectors are normalized (then it ranks exactly like cosine)
`euclidean`	0 to infinity	Magnitude itself carries meaning (often image embeddings)

For all three metrics, **smaller distance = more similar**, so you always sort **ascending**.

15.6 Writing vector search queries

Exact (kNN):

```
DECLARE @qv vector(1536) = AI_GENERATE_EMBEDDINGS(N'lightweight hiking boots' USE MODEL
MyEmbeddingModel);

SELECT TOP (10) c.ChunkId, c.ChunkText,
       VECTOR_DISTANCE('cosine', @qv, c.Embedding) AS distance
FROM dbo.ProductChunks AS c
ORDER BY distance;                                  -- ascending: closest first
```

Approximate (ANN) — current syntax (latest index version):

```
SELECT TOP (10) WITH APPROXIMATE
       t.ChunkId, t.ChunkText, s.distance
FROM VECTOR_SEARCH(
        TABLE      = dbo.ProductChunks AS t,
        COLUMN     = Embedding,
        SIMILAR_TO = @qv,                              -- a vector variable or column
        METRIC     = 'cosine'                          -- must match the index
     ) AS s
WHERE t.Category = N'Footwear'                        -- latest version filters DURING the
search
ORDER BY s.distance;                                   -- distance only, ascending only
```

Approximate (ANN) — older syntax (earlier index versions; `TOP_N` is deprecated and fails with error 42274 on a latest-version index):

```
SELECT TOP (10) t.ChunkId, t.ChunkText, s.distance
FROM VECTOR_SEARCH(
        TABLE = dbo.ProductChunks AS t, COLUMN = Embedding,
        SIMILAR_TO = @qv, METRIC = 'cosine', TOP_N = 10) AS s
ORDER BY s.distance;
```

Rules that decide questions:

- The result has *"An additional column named distance"*, and *"The distance column is the only valid ordering key for approximate vector search results"* — ascending only. `GROUP BY s.distance` or `PARTITION BY s.distance` don't rank anything.
- Refer to table columns through the **TABLE alias** (`t.ChunkText`) and to the distance through the **VECTOR_SEARCH alias** (`s.distance`).
- `WITH APPROXIMATE` only works with `VECTOR_SEARCH`. For window functions, `GROUP BY`, `UNION`, or extra sort columns, run the vector search in a **subquery** and do the rest outside.
- The question vector must come from the **same embedding model** as the stored vectors.
- On SQL Server 2025, turn on `PREVIEW_FEATURES` first (not needed in Azure SQL Database or Fabric).

SCENARIO PATTERN

- "Similarity search over a `vector(1536)` column with **minimum latency**" → a **vector index** on that column + `VECTOR_SEARCH`. A B-tree index isn't allowed, a full-text index works only on text, and a computed column still compares every row.
- "`VECTOR_DISTANCE('cosine', …)` is slow and CPU spikes at peak" → create a vector index and switch to **`VECTOR_SEARCH` with `METRIC = 'cosine'`**. `VECTOR_NORMALIZE`, a nonclustered index, `CONTAINSTABLE` only, or `VARBINARY(8000)` don't fix it.
- "ANN across two embedding tables built with **different chunk sizes**" → **`VECTOR_SEARCH`** with **cosine** distance.
- "Complete `CREATE VECTOR INDEX … ON dbo.Products ( ___ )`" → the **vector column** (`embedding`), never a text column like `product_name`.

15.7 Hybrid search and reciprocal rank fusion (RRF)

Hybrid search runs a keyword search and a vector search for the same question and merges the results. The hard part is merging, because the scores point in **opposite directions** on **different scales**: full-text `RANK` is 0–1000 (higher is better) while cosine distance is 0–2 (lower is better).

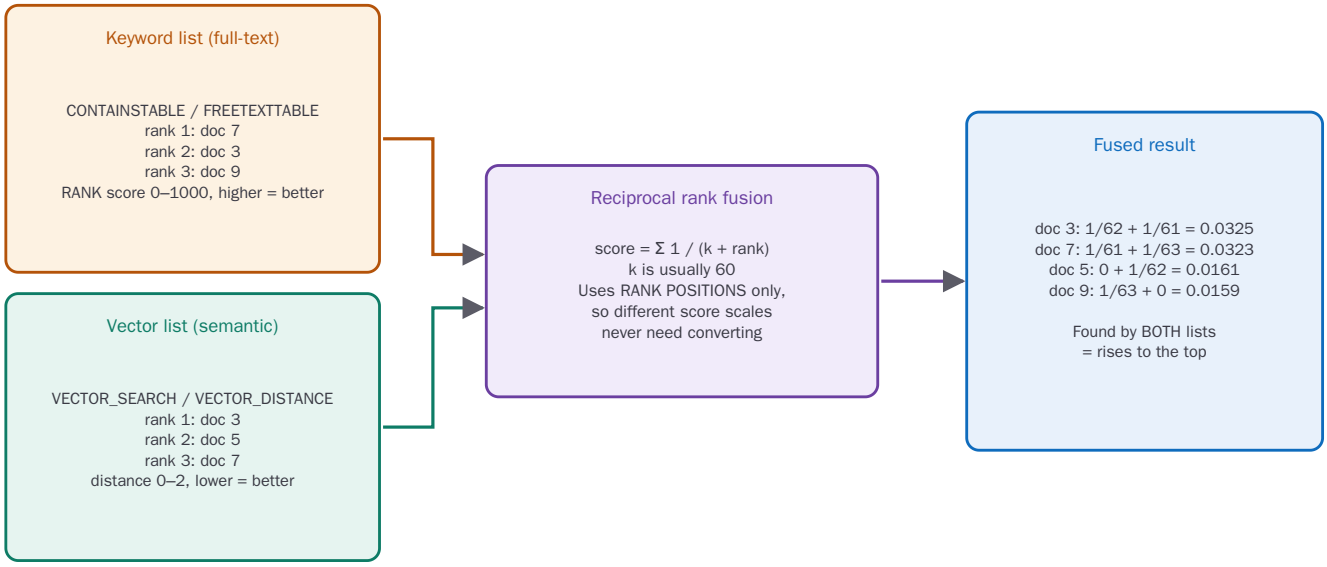
Option 1 — weighted score. Convert both to "lower is better" on a 0–1-ish scale and add them with weights:

```
-- 60% vector distance + 40% full-text rank, top 20 (lower combined score = better)
WITH candidates AS (
    SELECT s.store_id, s.description,
           VECTOR_DISTANCE('cosine', @qv, s.embedding) AS distance,
           ft.[RANK]                                   AS ft_rank
    FROM dbo.stores AS s
    INNER JOIN FREETEXTTABLE(dbo.stores, description, @searchText) AS ft
            ON s.store_id = ft.[KEY]
)
SELECT TOP (20) store_id, description, distance, ft_rank
FROM candidates
ORDER BY (distance * 0.6) + ((1.0 - ft_rank / 1000.0) * 0.4);
```

`1.0 - RANK/1000.0` rescales RANK to 0–1 and flips it so that both terms are lower-is-better. Divide by `1000.0` (decimal) — integer division would give 0. Adding raw `RANK` to a distance (`distance + RANK`, or `distance * 0.6 + RANK * 0.4`) is wrong: the best keyword matches would sort last, and RANK would swamp the distance.

Option 2 — Reciprocal rank fusion (RRF). Ignore the scores and use **positions**:

> *"RRF calculates a score based on each document's position (the rank) using the formula 1/(rank + k). The constant k is 60 … Documents appearing in multiple result sets have their scores summed."*



RRF only needs each document's position in each list. Documents found by both searches add up two scores and rise to the top.

```
DECLARE @k int = 60, @n int = 50;                    -- RRF constant, candidates per list

WITH keyword_list AS (                               -- positions 1..n by full-text rank
    SELECT ft.[KEY] AS ProductID,
           ROW_NUMBER() OVER (ORDER BY ft.[RANK] DESC) AS rnk
    FROM FREETEXTTABLE(dbo.Products, Description, @searchText, @n) AS ft
),
vector_list AS (                                     -- positions 1..n by distance
    SELECT v.ProductID, ROW_NUMBER() OVER (ORDER BY v.distance) AS rnk
    FROM (SELECT TOP (@n) WITH APPROXIMATE t.ProductID, s.distance
          FROM VECTOR_SEARCH(TABLE = dbo.Products AS t, COLUMN = DescriptionVector,
                             SIMILAR_TO = @qv, METRIC = 'cosine') AS s
          ORDER BY s.distance) AS v
)
SELECT TOP (10)
       COALESCE(kl.ProductID, vl.ProductID) AS ProductID,
       COALESCE(1.0 / (@k + kl.rnk), 0.0) + COALESCE(1.0 / (@k + vl.rnk), 0.0) AS rrf_score
FROM keyword_list AS kl
FULL OUTER JOIN vector_list AS vl ON kl.ProductID = vl.ProductID   -- keep items found by
only one list
ORDER BY rrf_score DESC;                                           -- higher RRF score =
better
```

Why RRF: *"Score normalization isn't needed. Full-text BM25 scores and cosine distances use completely different scales. RRF sidesteps these differences by using ranks instead."* A larger k smooths the gap between positions; a smaller k favours the top items. To weight one list more, multiply its term (for example `2.0 / (@k + vl.rnk)`). Filter *before* fusing — filtering after RRF can remove top results.

Option 3 — ANN candidates re-ranked by a keyword match. Take the nearest vectors, keep only those that also match a precise full-text condition, then rank:

```
DECLARE @qv vector(1536) = AI_GENERATE_EMBEDDINGS(@query_text USE MODEL Ada2Embeddings);

WITH ann AS (                                        -- older-syntax form, as exam items
show it
    SELECT t.product_id, t.product_name, s.distance
    FROM VECTOR_SEARCH(TABLE = dbo.Products AS t, COLUMN = embedding,
                       SIMILAR_TO = @qv, TOP_N = 20, METRIC = 'cosine') AS s
)
SELECT TOP (10) a.product_id, a.product_name, a.distance, fts.[RANK]
FROM ann AS a
JOIN CONTAINSTABLE(dbo.Products, description, @keywords) AS fts   -- INNER JOIN = must also
match
     ON a.product_id = fts.[KEY]
ORDER BY (a.distance * 0.6) + ((1.0 - fts.[RANK] / 1000.0) * 0.4);
```

KNOW THE DIFFERENCE

Exact `VECTOR_DISTANCE` vs `VECTOR_SEARCH` in hybrid queries. When a question says *"use approximate nearest neighbor (ANN) to retrieve the top 20 candidates"*, the answer is `VECTOR_SEARCH`. When it only asks for a weighted score over the keyword matches, with no ANN requirement, the documented pattern pairs `VECTOR_DISTANCE` with `FREETEXTTABLE` — `VECTOR_SEARCH … TOP_N` would cut the list to the N nearest vectors before the formula sees the keyword matches. For the keyword side, `CONTAINSTABLE` fits a precise `@keywords` condition and `FREETEXTTABLE` fits plain search text; both return `KEY` and `RANK`.

15.8 Evaluating vector and hybrid search

Measure	Question it answers
Recall (ANN)	What share of the exact kNN neighbours did the approximate search return?
Precision	*"Of the results returned, what percentage are relevant?"*
Recall (relevance)	*"Of all relevant documents in your database, what percentage did the search find?"*
MRR (mean reciprocal rank)	*"How high does the first relevant result appear?"*
Latency, CPU, index build time	What it costs

How to evaluate: build a **test set** of queries with known relevant documents, then run full-text only, vector only, and hybrid, and compare the metrics. Use **exact kNN** (`VECTOR_DISTANCE`) as the ground truth for ANN recall. If vector results are poor although an index exists, first check that the query's **metric and column** match the index (a mismatch makes the engine ignore the index and run an exact kNN scan — only a warning is raised). After re-embedding with a new model, recreate the index and use the same model for questions. Index the columns used in `WHERE` filters, and remove duplicate vectors before indexing.

15.9 Azure AI Search: hybrid queries with semantic ranking

Some exam items use **Azure AI Search**, a separate search service. A **hybrid query** there is one REST request with both a text query (`search`) and vector queries (`vectorQueries`); the service runs them in parallel and merges them with **RRF automatically**. The optional **semantic ranker** then re-ranks the top 50 merged results and can return **captions** and **answers**.

```
{
  "search": "historic hotel walk to restaurants and shopping",
  "vectorQueries": [
    { "kind": "vector", "vector": [0.0191, -0.0072, ...], "fields": "DescriptionVector",
"k": 50 }
  ],
  "queryType": "semantic",
  "semanticConfiguration": "hotels",
  "captions": "extractive",
  "answers": "extractive",
  "top": 10
}
```

Setting	Rule
`k`	*"Whenever you use semantic ranking with vectors, make sure k is set to 50. Semantic ranker uses up to 50 matches as input."* Fewer → "Semantic ranker returns fewer results"
`queryType`	`simple` (default), `full` (Lucene), or **`semantic`**. There is **no** `"hybrid"` value — hybrid just means `search` + `vectorQueries` together
`semanticConfiguration`	Required with `queryType: "semantic"` — the **name** of a semantic configuration defined in the index
`captions`, `answers`	`"extractive"` or `"none"` (default none). Both are verbatim text from your documents. `"generative"` is a value for `queryRewrites`, not for captions or answers

15.10 Chapter 15 self-check

1. A user searches for part number "BX-4471". Full-text or vector search?
2. Which full-text functions return a `RANK`, and in which direction is it better?
3. Your table has 30,000 vectors and needs perfect accuracy. kNN or ANN?
4. A vector index uses `METRIC = 'cosine'` but the query uses `'euclidean'`. What happens?
5. Name two things wrong with `ORDER BY s.distance DESC, t.title` in an approximate query.
6. What does RRF add up for each document?
7. In Azure AI Search, which `queryType` and `k` make captions work with a vector query?

Answers: 1) Full-text (exact codes). 2) `CONTAINSTABLE` and `FREETEXTTABLE`; higher is better (0–1000). 3) kNN (exact) with `VECTOR_DISTANCE` — under about 50,000 vectors. 4) The index isn't used; a warning is raised and an exact kNN scan runs. 5) Approximate results must be ordered by `distance` only, and ascending only. 6) `1 / (k + position)` from each list it appears in (k is usually 60). 7) `"semantic"` with a `semanticConfiguration`, and `k = 50`.

15.11 Learn more

- Training: Design and implement intelligent search with SQL — choose an approach, full-text, prepare for vector search, vector query patterns, hybrid search and ranking

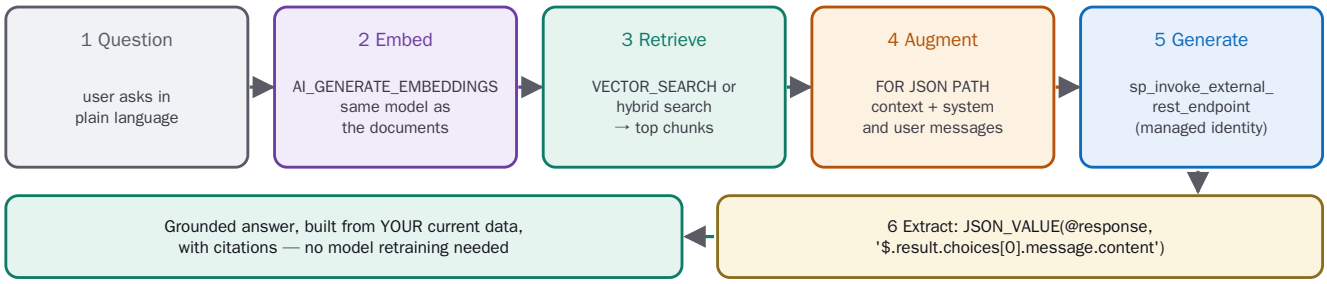
- Vectors: Vector search and vector index · vector data type · CREATE VECTOR INDEX · VECTOR_SEARCH · VECTOR_DISTANCE · VECTOR_NORMALIZE · VECTORPROPERTY · Vectors FAQ
- Full-text: Full-text search · CREATE FULLTEXT INDEX · CONTAINS · CONTAINSTABLE · FREETEXTTABLE
- Azure AI Search: Hybrid search · Create a hybrid query · RRF scoring · Semantic ranking queries

---

Chapter 16 — Retrieval-augmented generation (RAG) in T-SQL

EXAM FOCUS

Design and implement retrieval-augmented generation (RAG) — identify use cases for RAG · create a prompt by using the sp_invoke_external_rest_endpoint stored procedure · convert structured data to JSON for language model processing · send results to language model · extract language model responses.



The whole RAG loop inside the database: embed the question, retrieve the best rows, turn them into JSON, send them with the question to the model, and pull the answer out of the response.

16.1 What RAG is, and when to use it

> *"Large Language Models (LLM) don't know your organization. Your products, customers, policies. None of that exists in their training data."*

RAG fixes that per question: you **retrieve** relevant rows from your database, **augment** the prompt by adding them as context, and the model **generates** an answer *"grounded in your data rather than its training."*

Need	Choose	Why (from Microsoft's guidance)
Answers about **company-specific, frequently changing** data, with **citations**, without retraining	**RAG**	*"RAG works when data changes frequently … RAG retrieves current data every time. RAG also fits when you need to trace answers. You control retrieval, so you know which records informed each response."*
Sensitive data must stay under your control	**RAG**	*"Fine-tuning sends data to the model provider for training. RAG keeps data in your database and sends only the context needed per request."*
Change the model's tone, style, format, or task skill	**Fine-tuning**	*"Use fine-tuning when you need to change model behavior, style, or task performance, rather than add fresh knowledge."*
General knowledge, or text the user already pasted in	**Prompt only**	Nothing needs to be retrieved

PLAIN ENGLISH

Fine-tuning retrains a model so the knowledge is *"baked in permanently"* — it goes stale when your data changes. **RAG** looks the facts up fresh every time. Grounding *reduces* hallucination but doesn't remove it, so keep clear rules in the system message and return citations.

The retrieval step: *"In a RAG solution, you embed the user query by using the same embedding model as your chunks. Then, you search your database for relevant vectors to return the most semantically relevant chunks."* Hybrid search (15.7) is a good upgrade when users also search by exact titles or categories.

SCENARIO PATTERN

- "A chatbot calls the LLM directly; it answers general HR questions but gives outdated answers when policies change and can't find policies by title or category" → retrieve grounding data from the **policy knowledge table**, by **generating a query embedding and running a vector similarity search** (return title and category so answers can cite them). Not the employee tables, not PDF exports, not the training data, not fine-tuning, and not "call the LLM first and store the response".
- "Grounded in the latest data, no retraining, citations" → the best RAG use case is **answering user questions based on company-specific knowledge** — not summarizing pasted text, training a custom model, or writing slogans.

16.2 Calling a model: `sp_invoke_external_rest_endpoint`

> *"The sp_invoke_external_rest_endpoint stored procedure invokes an HTTPS REST endpoint provided as an input argument to the procedure."*

It is **the** way to call Azure OpenAI (or any HTTPS API) from T-SQL. It runs in Azure SQL Database, Azure SQL MI, SQL database in Fabric, and SQL Server 2025.

```
DECLARE @response nvarchar(max), @returnValue int;

EXECUTE @returnValue = sp_invoke_external_rest_endpoint
    @url        = N'https://my-openai.openai.azure.com/openai/deployments/gpt-
chat/chat/completions?api-version=2024-10-21',
    @method     = 'POST',                                   -- the default
    @payload    = @payload,                                 -- JSON body (16.4)
    @credential = [https://my-openai.openai.azure.com],     -- database scoped credential
(10.4)
    @timeout    = 30,                                       -- seconds, 1–230
    @retry_count = 3,                                       -- retries 408, 429, 500, 502,
503, 504
    @response   = @response OUTPUT;
```

Fact	Detail
Return value	**0** when the HTTP status is 2xx; otherwise **the HTTP status code itself** (429 = throttled, 401/403 = credential problem). An exception if the call can't be made at all
Response	A JSON envelope: `{"response": {"status": {"http": {"code":…, "description":…}}, "headers": {…}}, "result": { …the API's JSON… }}`
Permission	`GRANT EXECUTE ANY EXTERNAL ENDPOINT TO …;` plus `REFERENCES` on the credential
Enabling	**On** by default in Azure SQL Database and SQL database in Fabric; **off** in SQL Server 2025 and Managed Instance — `EXECUTE sp_configure 'external rest endpoint enabled', 1; RECONFIGURE;`
Endpoints	HTTPS only; in Azure SQL an allow-list of Azure endpoints (including `*.openai.azure.com`); use API Management for others

COMMON EXAM TRAP

`xp_cmdshell` (spawns a Windows command shell), `sp_OACreate` (creates an OLE object), and `CREATE ENDPOINT` (inbound listeners for T-SQL, Service Broker, or mirroring) are SQL Server-only features that have nothing to do with calling a REST API. `sp_addendpoint` doesn't exist.

16.3 Converting rows to JSON for the model

A model reads text, not tables. *"JSON solves this issue by preserving structure in a text format. Field names stay attached to values. Nested objects represent relationships."*

```
-- Many rows → a JSON array (the retrieved context)
DECLARE @context nvarchar(max) = (
    SELECT TOP (5) n.NoteText AS note, t.Title AS ticket, t.CreatedAt AS created
    FROM dbo.TicketNotes AS n
    JOIN dbo.Tickets AS t ON t.TicketId = n.TicketId
    ORDER BY VECTOR_DISTANCE('cosine', n.Embedding, @questionVector)
    FOR JSON PATH
);

-- One row → a single object
SELECT Name, Description, Size FROM Production.Product WHERE ProductID = @id
FOR JSON PATH, WITHOUT_ARRAY_WRAPPER;

-- Header with a nested array of lines
SELECT o.OrderNumber AS orderNumber,
       (SELECT l.ProductId AS productId, l.Qty AS qty
        FROM Sales.OrderLines AS l WHERE l.OrderId = o.OrderId
        FOR JSON PATH) AS lines
FROM Sales.Orders AS o
FOR JSON PATH;
```

Keep the context **lean**: internal IDs, audit timestamps, and warehouse codes *"just consume tokens and add noise."* `FOR JSON AUTO` is quick but needs a `FROM` clause; `FOR JSON PATH` gives you control of names and nesting; `INCLUDE_NULL_VALUES` keeps null fields (they are dropped by default).

16.4 Building the prompt and sending it

Chat models expect a `messages` array with **roles**: *"The system role defines how the assistant should behave. The user role contains the original question and the database context into which the model should ground its answer."*

```
DECLARE @payload nvarchar(max) = JSON_OBJECT(
    'messages': JSON_ARRAY(
        JSON_OBJECT('role': 'system',
                    'content': 'You are a support assistant. Answer only from the provided
notes. If the notes do not contain the answer, say so.'),
        JSON_OBJECT('role': 'user',
                    'content': 'Notes: ' + @context + CHAR(10) + 'Question: ' + @question)
    ),
    'max_tokens': 500,
    'temperature': 0.3          -- low = consistent, factual answers
);
```

Good grounding instructions *"set scope ('use only the provided product data'), encourage honesty ('if you don't have enough information, say so'), and can specify format."* Building the body with `JSON_OBJECT` / `JSON_ARRAY` escapes quotes and newlines for you.

The `FOR JSON` alternative: a one-row `SELECT` with no `FROM`, whose aliases describe the body, finished with `FOR JSON PATH, WITHOUT_ARRAY_WRAPPER` — because the endpoint expects **one JSON object**, and *"Use this option with a single-row result to generate a single JSON object as output instead of an array with a single element."* `FOR JSON AUTO` can't work without a `FROM` clause, and `FOR XML` produces XML, which the endpoint doesn't accept.

> **TIP**
>
> Reasoning-model deployments may reject `temperature` and `max_tokens` (they use `max_completion_tokens`). The training module and exam items use `max_tokens` and `temperature`, so answer the exam that way.

16.5 Extracting the response

```
IF @returnValue = 0
    SET @answer = JSON_VALUE(@response, '$.result.choices[0].message.content');
ELSE IF @returnValue = 429
    THROW 50429, 'The model is busy. Try again later.', 1;
ELSE IF @returnValue IN (401, 403)
    THROW 50401, 'Authentication failed. Check the database scoped credential.', 1;
```

You need	Function	Path
The answer text	**`JSON_VALUE`** (a scalar)	`$.result.choices[0].message.content`
An answer longer than 4,000 characters	`OPENJSON … WITH (content nvarchar(max) '$.content')`	`$.result.choices[0].message`
An embedding (an array)	**`JSON_QUERY`**, then `CAST(… AS vector(1536))`	`$.result.data[0].embedding`
The whole `choices` array	`JSON_QUERY`	`$.result.choices`
The HTTP status text	`JSON_VALUE`	`$.response.status.http.description`

> *"The answer you want lives at $.result.choices[0].message.content. To get there, use JSON_VALUE, which extracts scalar values from JSON … If you ever need to extract a JSON object or array rather than a single value, use JSON_QUERY instead."*

COMMON EXAM TRAP

- Forgetting **`result.`** — the procedure wraps the API's body inside `result`, so the full path starts with `$.result`. (An exam option may omit it; pick the one that targets `choices[0].message.content`.)
- `JSON_MODIFY` writes JSON, it doesn't read it; `OPENJSON(@response)` without a path returns only the top-level properties; `JSON_QUERY(@response, '$.choices')` returns the whole array, not the text.
- `JSON_VALUE` silently returns NULL (lax mode) if the answer is longer than 4,000 characters — switch to `OPENJSON … WITH … nvarchar(max)`.

SCENARIO PATTERN

A chat agent must **serialize retrieved rows**, **extract the answer field** from the response, and **extract the embeddings** to store in a query cache → **`FOR JSON PATH`**, **`JSON_VALUE`**, **`JSON_QUERY`**. (`AI_GENERATE_CHUNKS` splits text; `VECTOR_DISTANCE` compares vectors; `FOR XML PATH` produces XML.)

16.6 The end-to-end stored procedure

This is the RAG procedure the training module builds, step by step:

```
CREATE PROCEDURE dbo.AskProductQuestion
    @Question nvarchar(1000),
    @Answer   nvarchar(max) OUTPUT
AS
BEGIN
    SET NOCOUNT ON;
    DECLARE @questionVector vector(1536), @context nvarchar(max),
            @payload nvarchar(max), @response nvarchar(max), @returnValue int;

    -- 1. Embed the question with the same model as the stored vectors
    SELECT @questionVector = AI_GENERATE_EMBEDDINGS(@Question USE MODEL my_embedding_model);

    -- 2. Retrieve the most relevant rows and serialize them as JSON
    SET @context = (
        SELECT TOP 3 p.Name AS ProductName, p.Color, p.Size, pm.Name AS Model
        FROM Production.Product AS p
        INNER JOIN Production.ProductModel AS pm ON p.ProductModelID = pm.ProductModelID
        ORDER BY VECTOR_DISTANCE('cosine', p.DescriptionVector, @questionVector)
        FOR JSON PATH
    );

    -- 3. Augment: system rules + user context and question
    SET @payload = JSON_OBJECT(
        'messages': JSON_ARRAY(
            JSON_OBJECT('role': 'system', 'content': 'You are an Adventure Works product
assistant. Answer questions using only the provided product data.'),
            JSON_OBJECT('role': 'user',   'content': 'Products: ' + @context + ' Question: '
+ @Question)
        ),
        'max_tokens': 500,
        'temperature': 0.5
    );

    -- 4. Generate: call the chat model
    EXECUTE @returnValue = sp_invoke_external_rest_endpoint
        @url = N'https://adventureworks-openai.openai.azure.com/openai/deployments/gpt-
5.2/chat/completions?api-version=2024-10-21',
        @method = 'POST',
        @payload = @payload,
        @credential = [https://adventureworks-openai.openai.azure.com],
        @response = @response OUTPUT;

    -- 5. Extract the answer
    IF @returnValue = 0
        SET @Answer = JSON_VALUE(@response, '$.result.choices[0].message.content');
    ELSE
        SET @Answer = 'Unable to process your question. Please try again.';
END;
```

Prerequisites: an external model `my_embedding_model`, precomputed `DescriptionVector` values, a database scoped credential named with the endpoint URL (managed identity), and for the caller `EXECUTE ANY EXTERNAL ENDPOINT`, `REFERENCES` on the credential, and `EXECUTE` on the model. For large tables, replace the exact `VECTOR_DISTANCE` retrieval with `VECTOR_SEARCH` (15.6); add `@retry_count` and status-code handling for production.

16.7 Chapter 16 self-check

1. Give two reasons to choose RAG over fine-tuning.
2. What does `sp_invoke_external_rest_endpoint` return when the model replies with HTTP 429?
3. Which permission does a user need to call external REST endpoints from T-SQL?
4. Your one-row request body comes out as `[{"messages":[…]}]`. What do you add?
5. Which path and function return the chat answer text?
6. Why must the question be embedded with the same model as the documents?

Answers: 1) Answers use current data with no retraining, and you can cite the records used (also: data stays in your database). 2) The return value is 429. 3) `EXECUTE ANY EXTERNAL ENDPOINT` (plus `REFERENCES` on the credential). 4) `WITHOUT_ARRAY_WRAPPER`. 5) `JSON_VALUE(@response, '$.result.choices[0].message.content')`. 6) Vectors from different models aren't comparable — distances would be meaningless.

16.8 Learn more

- Training: Design and implement RAG with SQL — use cases and architecture, prepare retrieval context, augment prompts, generate and process responses
- Reference: sp_invoke_external_rest_endpoint · FOR JSON · WITHOUT_ARRAY_WRAPPER · JSON_OBJECT · JSON_VALUE · JSON_QUERY
- Concepts: RAG in Microsoft Foundry · Intelligent applications with Azure SQL · RAG design and evaluation guide

---

Chapter 17 — Case-study walkthroughs

DP-800 case studies are long. You get a company description, an *Existing environment* section, *Problem statements*, and *Requirements*, then several questions that each test one requirement. This chapter shows you how to read them, using two scenarios that match the style of the real exam.

17.1 How to attack a case study

1. **Read the question first, then the requirement it points to.** A question like *"You need to meet the development requirements for the FeedbackJson column"* tells you exactly which paragraph to read.
2. **Underline the deciding words.** *Live* metadata, *passwordless*, *on-premises*, *tamper-evident*, *must NOT be changed*, *minimize latency*, *partitioned*. One word usually picks the feature.
3. **Map each requirement to one feature.** Most requirement sentences map to exactly one feature in this guide. The tables below show the mapping.
4. **Check the small print in the environment section.** Facts such as *"The FeedbackJson column has a full-text index"* or *"The support staff never has the UNMASK permission"* are there to be used.
5. **Each question is independent.** Do not carry a decision from one question into the next unless the case says so.

> **TIP**
> Case-study code exhibits are often simplified (a missing `WITH SCHEMABINDING`, a typo in a column name). Answer what the question is clearly testing, not the typo.

17.2 Case A — a fleet-telematics platform (Azure SQL Database)

The story. A logistics company runs a fleet intelligence platform on an old SQL Server database. It is moving to a new **Azure SQL Database** that will be the main store for AI workloads, vector search, modern API access, and RAG pipelines. Tables include vehicle health summaries, incident reports (with a long free-text description), maintenance events, telemetry, and customer feedback stored as JSON (the JSON column already has a **full-text index**). Support staff never get the `UNMASK` permission.

Requirement or problem (in the case)	What to use	Why (and where to read more)
Support staff must not see full email addresses and phone numbers	**Dynamic data masking** — `email()` on the email column, `default()` or `partial()` on the phone column	Masks values in query results for users without `UNMASK`; data is unchanged (9.2)
Analysts see only incidents for the fleets they are assigned to (one analyst, many fleets)	**Row-level security** — an inline TVF predicate that joins to a user-to-fleet mapping table, plus a security policy with a filter predicate	RLS filters rows for every query, from every tool; it also applies to `dbo` unless the predicate allows it (9.3)
Masking must still work when RLS removes rows	Nothing extra — RLS and DDM **stack**	RLS decides *which rows*, DDM decides *how the columns look* (9.2, 9.3)
Telemetry must be stored in a **partitioned** table with predictable ingestion and retention	A partition function (`RANGE RIGHT` on the date column) + partition scheme, and `CREATE TABLE … ON scheme(column)`	Retention becomes a partition-level operation such as `TRUNCATE TABLE … WITH (PARTITIONS (n))` (4.5, 4.6). A temporal table is **not** partitioning
Latitude, longitude, and accuracy JSON properties must be filtered with an **index seek**	A native `json` column + `CREATE JSON INDEX … FOR ('$.location.latitude', …)` (the table needs a clustered primary key)	Only the listed paths (and their children) are indexed. A sibling such as `$.location.heading` is **not** covered (4.2)
Any change to a maintenance row must update `LastModifiedUtc`; avoid recursive updates	An `AFTER UPDATE` trigger that updates the table joined to `inserted` **on the primary key** (`m.MaintenanceId = i.MaintenanceId`). For the "avoid recursive updates" box, the exam most likely expects `WHERE m.LastModifiedUtc <> i.LastModifiedUtc`	Joining on a non-unique column (such as `VehicleId`) re-stamps unrelated rows. Strictly, that `WHERE` filters out every row (in an `AFTER` trigger `inserted` already holds the new values), and direct recursion only happens when `RECURSIVE_TRIGGERS` is ON (5.5)
The ingestion pipeline sometimes fails on malformed JSON and duplicate payloads	`CHECK (ISJSON(Payload) = 1)` + a **unique index on a hash** of the payload (persisted `HASHBYTES('SHA2_256', …)` column)	Rejects bad JSON at the door; a LOB column can't be an index key, but a 32-byte hash can (4.1, 4.3)
A dashboard query (`WHERE FleetId = @FleetId ORDER BY LastUpdatedUtc DESC`) does a clustered index scan	A nonclustered index **on FleetId** that **INCLUDEs** the other selected columns	Seek on the filter column, cover the rest, no key lookups (2.3)
Analysts can't find similar incidents by	Generate embeddings from the **rich free-text column** (the incident description)	Embeddings capture meaning; short labels, numbers, and IDs don't (14.3)

Requirement or problem (in the case)	What to use	Why (and where to read more)
weather, traffic, and location details		
Embeddings in dedicated tables; chunking must preserve meaning	A separate chunk/embedding table; chunk on sentence or paragraph boundaries (with overlap)	14.4
Retrieve the most relevant health summaries with **minimum latency**	A **vector index** (DiskANN) on the `vector(1536)` column, queried with `VECTOR_SEARCH`	B-tree and full-text indexes can't index vectors; exact search scans every row (15.5)
Hybrid search must combine vector similarity with keyword filtering or boosting	Full-text search (`CONTAINSTABLE` / `FREETEXTTABLE`) + vector search, merged with weights or RRF	15.7
Developers use VS Code + GitHub Copilot and must work with **live** metadata	An **MCP server** connected to the database	A `.dacpac`, an instruction file, or a project in the repo are static copies (8.6)
Extract feedback text from JSON, filter by keyword, fuzzy-score against a known issue, order by score	`JSON_VALUE(FeedbackJson, '$.text')` · `CONTAINS(FeedbackJson, @Keyword)` · `EDIT_DISTANCE_SIMILARITY(…)` · `ORDER BY SimilarityScore DESC`	`CONTAINS` uses the existing full-text index; `EDIT_DISTANCE` compares whole strings, so it is the wrong tool for "contains a word" (7.1, 7.3, 15.2)

SCENARIO PATTERN

Watch for the word that changes the answer: **"index seek"** on JSON properties → JSON index or computed column + index; **"live metadata"** → MCP; **"rich natural language fields"** → the long description column; **"minimize latency"** for similarity → vector index + `VECTOR_SEARCH`.

17.3 Case B — a healthcare provider (Azure SQL Database + GitHub)

The story. A healthcare company has one Azure SQL database (DB1) with tables for patients, employees, procedures, transactions, useful prompts, and procedure documents. A column master key is stored in **Azure Key Vault**. An **on-premises** application uses a hard-coded username and password. A long-running stored procedure sometimes produces inconsistent updates. All database changes will be managed in **GitHub**, every pull request to production must be **validated before merge**, and deployments must use the **Release** configuration.

Requirement or problem (in the case)	What to use	Why (and where to read more)
Every PR to production is validated before it can be merged; deployments use Release	A GitHub Actions workflow `on: pull_request: branches: [ "main" ]` that runs `dotnet build Database.sqlproj -c Release` (made a required status check)	`push` fires only *after* the merge — too late. `merge` is not a GitHub event (12.5, 12.8)
The on-premises app must use a **passwordless** connection	A **service principal** (Microsoft Entra app registration, ideally with a certificate rather than a client secret) + a contained database user `FROM EXTERNAL PROVIDER`	Managed identities exist only for Azure resources (or Arc-enabled servers). A shared access key is a secret and not accepted by Azure SQL (10.1)
`TaxID` and `Salary` must be encrypted at rest; the CMK is in Key Vault	**Always Encrypted** (the column master key lives in Azure Key Vault)	TDE encrypts files but anyone who can query sees plain values; DDM changes nothing on disk (9.1)
Auditors need a **tamper-evident** history with cryptographic proof of changes to employee data	**Ledger tables** (updatable ledger table for a table that is updated)	Ledger hashes every change into a verifiable chain; normal auditing does not prove the data wasn't altered (10.3)
Records read by the long-running procedure must NOT be changed by other transactions while it runs	`SET TRANSACTION ISOLATION LEVEL REPEATABLE READ`	Least strict level that keeps shared locks on rows read; SERIALIZABLE adds range locks you don't need; SNAPSHOT doesn't stop writers (11.2)
Queries on procedure documents must use **reciprocal rank fusion**	Hybrid search: rank full-text results and vector results separately, then score `1 / (k + rank)` and add them up	15.7
Prompts stored in a table are sent to an **Azure OpenAI** endpoint	`sp_invoke_external_rest_endpoint` with a database scoped credential (managed identity)	Not `xp_cmdshell`, `sp_OACreate`, or a made-up `sp_addendpoint` (16.2)
Users ask questions in **Copilot in Microsoft Fabric**	Copilot in Fabric works over Fabric items, needs a paid F2+ (or P1+) capacity and the tenant setting, so DB1's data must be made available in Fabric (for example by mirroring)	8.3
Return a patient's transactions between two dates with a **running total**, and let developers **join** to the result	An **inline table-valued function** with `SUM(Amount) OVER (PARTITION BY CustomerId ORDER BY TransactionDate, TransactionId ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`	A procedure can't be used in `FROM` / `JOIN`; a view can't take parameters (5.3, 6.2)

Requirement or problem (in the case)	What to use	Why (and where to read more)
A DAB file must allow: read procedures without authentication; read **and insert** transactions when authenticated; execute a stored procedure	`anonymous: read` on Procedures · `authenticated: read, create` on Transactions · `authenticated: execute` on the procedure entity	In DAB, **create = insert**; **update** is a separate action (13.3)
List patients who had a procedure in the last 30 days	`WHERE EXISTS (SELECT … FROM dbo.Procedures pr WHERE p.PatientId = pr.PatientId AND pr.TransactionDate >= DATEADD(DAY, -30, SYSUTCDATETIME()))`	A correlated subquery with `EXISTS` returns each patient once (6.3)
RAG retrieval over procedure documents is slow and uses too many tokens (each document has **one** embedding over the whole text)	**Chunking** — split documents, embed each chunk, retrieve only the relevant chunks	Embeddings already exist; a smaller model or JSON formatting doesn't shrink the retrieved text (14.4)

COMMON EXAM TRAP

In this case the deciding facts are *on-premises* (→ service principal, not managed identity), *tamper-evident* (→ ledger, not auditing), and *must NOT be changed … while it runs* (→ REPEATABLE READ, not SNAPSHOT). Each word rules out the most tempting wrong answer.

Chapter 18 — Practice questions

These questions are written for this guide. They test the same rules the real exam tests, in the same style. Try all of them first, then check the answers.

▸ Design and develop database solutions

1. In a SQL database in Microsoft Fabric, a table holds three years of orders, partitioned by month on `OrderDate`. You must remove the oldest month with the least impact on other queries. What do you run?
2. You partition a `datetime2` column by year for 2024 and 2025. Which `RANGE` type and which boundary values?
3. A busy OLTP `Orders` table needs near-real-time revenue reports without slowing inserts much. Which index?
4. A 10 TB fact table is loaded nightly and queried with big scans and `SUM()`. Storage must be minimal. Which index?
5. A report filters on `$.severity` inside an `nvarchar(max)` JSON column and returns three columns. How do you avoid table scans *and* key lookups?
6. A leaderboard table needs the lowest possible write latency and point lookups by `PlayerId`. Data can be lost on restart. Which table type and index?
7. You must be able to ask "what did this row look like last Tuesday at 10:00?". Which table type and clause?
8. Some rows already have `NULL` in `Owner`. Future rows must always have an owner, and the constraint must be added today. What do you add?
9. A database-level DDL trigger must not fire during tonight's deployment, but must still exist afterwards. Which statement?
10. Reports join a multi-statement table-valued function to a big table and get poor, inconsistent plans. What do you change?
11. You want a view that stores monthly revenue per customer so reads are fast, without changing any table. Name three requirements.
12. Return exactly one row per customer per day — the earliest ticket — even when two tickets have the same timestamp. Which window function and filter?
13. A phone column contains `+44 (20) 7946-0018`. Return only the digits. Which expression?
14. In a `CASE WHEN`, you need to test whether a message contains *any* phone number. Which regex function?
15. City names contain typos ("Seatle"). You need a similarity score from 0 to 100 between two strings. Which function?
16. A stored procedure must undo its transaction on any error and return the *original* error to the caller. What goes in the `CATCH` block?

▸ AI-assisted tools

1. Your team's repository has `.github/copilot-instructions.md`. You want Copilot Chat to ignore it **for you only**. What do you change?
2. The team must share a GitHub MCP server configuration through the repository, using OAuth (no tokens in files). What do you do in VS Code?
3. A developer asks Copilot in SSMS to run a query. What decides which rows it can read?

▸ Secure, optimize, and deploy

1. `NationalID` is used for point lookups; `Notes` is only displayed. Both must use Always Encrypted with the highest security. Which encryption type for each?
2. A support role must query `Customer` but never see `TaxID`. Which two statements?
3. An App Service app must connect to Azure SQL Database without passwords, and all traffic must stay private. What two things do you configure?
4. An on-premises application must connect to Azure SQL Database without a password. Which identity?
5. A stored procedure must make sure rows it has read are not changed by others until it finishes, with the least blocking. Which isolation level?
6. Session 72 is `sleeping`, `open_transaction_count = 1`, and many sessions wait on it. What is the cause, and why can't you see it in `sys.dm_exec_requests`?
7. After a deployment a query is slower. Query Store shows two plans, and the old one was faster. What is the fastest fix with no code change?
8. A CI build of an Azure SQL Database project fails with unresolved references to system objects. What do you add?
9. You need an XML report of what a publish would change in production. Which SqlPackage action?
10. Reference data must deploy with every release, and pipeline reruns must not create duplicates. How?
11. In Data API builder, authenticated users may **insert** into `Transactions` but not change existing rows. Which action do you grant?
12. An Azure Functions app must run when rows in `dbo.ToDo` change, using the SQL trigger binding. What must you configure in the database?

▸ Implement AI capabilities

1. Embeddings must be refreshed whenever articles change, with **minimum CPU** on the database. Which method?
2. A SQL Server 2025 instance has no outbound internet. Only one user may generate embeddings with a model. Which two actions?
3. You switch from `text-embedding-ada-002` to `text-embedding-3-small` (both 1,536 dimensions). What must you do first?
4. A semantic search uses `VECTOR_DISTANCE` over 5 million rows and spikes CPU. What do you change?
5. Two embedding tables were built with different chunk sizes. Which distance metric reduces the effect of chunk size?

6. You must combine a keyword result list and a vector result list without converting their scores. Which technique, and what is its formula?
7. The chat-completions request body must be one JSON object, not an array. How do you finish the `SELECT` that builds it?
8. Which expression extracts the answer text from the `@response` of `sp_invoke_external_rest_endpoint` calling chat completions?
9. Which scenario is the best fit for RAG: summarising text a user pasted, answering questions from company policies that change often, writing slogans, or training a custom model?

▸ Answers and explanations

1. **`TRUNCATE TABLE dbo.Orders WITH (PARTITIONS (n));`** It deallocates only that partition's pages (minimal logging). `DELETE` logs every row and takes many locks; `TRUNCATE` without `WITH (PARTITIONS …)` empties the whole table. (On SQL Server or Azure SQL, switching the partition out and truncating the staging table also works — but Fabric doesn't allow `SWITCH PARTITION`.)
2. **`RANGE RIGHT FOR VALUES ('2024-01-01', '2025-01-01')`.** Each boundary starts a new partition, so midnight on 1 January lands in the new year. Two boundaries make three partitions.
3. **A nonclustered columnstore index** on the rowstore table — real-time operational analytics. A clustered columnstore index would replace the OLTP row storage.
4. **A clustered columnstore index.** It *is* the table, highly compressed, ideal for scans and aggregates.
5. **Add a computed column `AS JSON_VALUE([log], '$.severity')` and index it with `INCLUDE` (the returned columns).** (With the native `json` type, `CREATE JSON INDEX` also avoids the scan, but it can't `INCLUDE` columns, so it doesn't remove key lookups.) `JSON_QUERY` returns objects/arrays, not scalars.
6. **A memory-optimized table with `DURABILITY = SCHEMA_ONLY` and a `HASH` index.** No logging = faster writes; hash = point lookups.
7. **A system-versioned temporal table**, queried with `FOR SYSTEM_TIME AS OF '…'`.
8. **A CHECK constraint `CHECK (Owner IS NOT NULL)` added `WITH NOCHECK`.** Existing rows are not validated; all future inserts and updates are.
9. **`DISABLE TRIGGER ddl_Audit ON DATABASE;`** (and `ENABLE TRIGGER` afterwards). `ALTER TRIGGER` re-enables a trigger.
10. **Rewrite it as an inline table-valued function** (`RETURNS TABLE AS RETURN (SELECT …)`). The optimizer expands it like a view and uses real statistics, instead of a fixed row guess.
11. **`WITH SCHEMABINDING`, `COUNT_BIG(*)` in a view with `GROUP BY`, and a unique clustered index as the first index.** No `ORDER BY` in the view.
12. **`ROW_NUMBER() OVER (PARTITION BY CustomerId, CAST(CreatedAt AS date) ORDER BY CreatedAt, TicketId)` and `WHERE rn = 1`** (the tie-breaker makes the choice deterministic). `DENSE_RANK` / `RANK` give ties the same number, so you could get two rows.
13. **`REGEXP_REPLACE(Phone, '[^0-9]', '')`** (or `'\D'`). An empty replacement deletes every match.
14. **`REGEXP_LIKE`** — it returns true/false. `REGEXP_COUNT` counts matches and `REGEXP_INSTR` returns a position.

15. **`EDIT_DISTANCE_SIMILARITY`** (`JARO_WINKLER_SIMILARITY` also gives a similarity score). `EDIT_DISTANCE` returns a number of edits, not a percentage.
16. **`IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;` then `THROW;`** A parameterless `THROW` re-raises the caught error; the statement before it must end with a semicolon.
17. **Turn off `github.copilot.chat.codeGeneration.useInstructionFiles` in your *User* settings.** Deleting the file or changing workspace settings affects everyone.
18. **Command Palette → MCP: Add Server → HTTP → `https://api.githubcopilot.com/mcp/` → save to Workspace** (writes `.vscode/mcp.json`). VS Code signs in with OAuth; no PAT is stored.
19. **The developer's own database login and its permissions.** Copilot has no identity or rights of its own.
20. **Deterministic for `NationalID`, randomized for `Notes`.** Only deterministic encryption supports equality lookups (without enclaves); randomized hides repeated values.
21. **`GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;` and `DENY SELECT ON OBJECT::Sales.Customer(TaxID) TO AppSupport;`** DENY beats GRANT; `REVOKE` only removes an entry.
22. **A managed identity with Microsoft Entra authentication, and a private endpoint with public network access disabled.**
23. **A service principal** (Entra app registration). Managed identities are only for Azure resources.
24. **REPEATABLE READ.** SERIALIZABLE also blocks inserts into ranges; SNAPSHOT doesn't stop other writers.
25. **An explicit transaction was started and never committed or rolled back.** A sleeping session has no active request, so it appears only in `sys.dm_exec_sessions`; use `sys.dm_exec_input_buffer` to see its last command.
26. **`EXEC sp_query_store_force_plan @query_id = …, @plan_id = …;`** (or **Force Plan** in the Regressed Queries view).
27. **A package reference to `Microsoft.SqlServer.Dacpacs.Azure.Master`** (the Azure SQL Database master model). `Microsoft.SqlServer.Dacpacs.Master` is for SQL Server.
28. **`/Action:DeployReport`.** `DriftReport` compares the database with its registration snapshot, not with your project.
29. **A post-deployment script with idempotent `MERGE` (or `IF NOT EXISTS`) logic, matching on the business key.**
30. **`create`** (REST `POST`). `update` is a separate action for changing existing rows.
31. **Enable change tracking on the database and on the table.** The SQL trigger binding polls change tracking — not CDC, not DML triggers.
32. **Change data capture + an Azure Functions app** that regenerates embeddings outside the database.
33. **Create an external model with `API_FORMAT = 'ONNX Runtime'` and local paths, and `GRANT EXECUTE` on that external model to the user.**
34. **Regenerate all existing embeddings.** Vectors from different models are not comparable, even with the same dimension count.
35. **Create a vector index (DiskANN) on the column and query with `VECTOR_SEARCH` using the same metric.** `VECTOR_DISTANCE` is always exact and never uses an index.

36. **Cosine distance** — it compares direction only and ignores vector length.
37. **Reciprocal rank fusion (RRF):** for each document add up `1 / (k + rank)` from each list (k is usually 60).
38. **`FOR JSON PATH, WITHOUT_ARRAY_WRAPPER`.**
39. **`JSON_VALUE(@response, '$.result.choices[0].message.content')`.** The procedure wraps the API body in a `result` property.
40. **Answering questions from company policies that change often.** RAG grounds answers in current private data, with citations, without retraining.

Chapter 19 — Labs, exam day, and quick reference

19.1 The hands-on lab track

Every official DP-800 module ends with an exercise. Do them in this order — together they cover every skill area.

#	Exercise (Microsoft Learn)	What it proves
1	Create and maintain database objects	Tables, indexes, constraints, JSON, partitions
2	Implement programmability objects in SQL Server	Views, functions, procedures, triggers
3	Work with JSON functions	Advanced T-SQL
4	Configure AI-assisted tools for database development	Copilot, instruction files, MCP
5	Implement security features	Encryption, masking, RLS, permissions
6	Optimize query performance	Plans, Query Store, blocking
7	Implement CI/CD by using SQL database projects	SQL projects, GitHub Actions
8	Configure data API builder for a product catalog	DAB, REST, GraphQL
9	Generate and update embeddings in Azure SQL Database	External models, embeddings
10	Implement intelligent search	Full-text, vector, hybrid
11	Implement a RAG solution	End-to-end RAG in T-SQL

> **HANDS-ON LAB**
>
> **Build this once, from memory.** In one database: a `Products` table with a `json` attributes column and a JSON index; a chunk table with a `VECTOR(1536)` column; an external model; a stored procedure that embeds a question, runs a hybrid search (full-text + `VECTOR_SEARCH`, fused with RRF), turns the top chunks into JSON with `FOR JSON PATH`, calls a chat model with `sp_invoke_external_rest_endpoint` using a managed identity credential, and returns the answer with `JSON_VALUE`. Put the schema in an SDK-style SQL project, validate it in a pull-request workflow, and expose `Products` through Data API builder. If you can do that, you are ready.

19.2 The week before

- Re-read the change log on the official DP-800 study guide and check the certification page for a practice assessment.
- Try the exam sandbox so the drop-down-in-code questions feel normal.
- Re-do the practice questions in Chapter 18 and read Appendix B one more time.

- Write these from memory: a TRY…CATCH block, a running-total window function, a partition function, a security policy, a `CREATE VECTOR INDEX` + `VECTOR_SEARCH` query, a pull-request workflow, and a DAB entity with permissions.

19.3 The 60-second memory dump

Topic	The one line
Platforms	SQL Server 2025 (you run it) · Azure SQL MI · Azure SQL Database · SQL database in Fabric (SaaS)
Columnstore	CCI = big batch-loaded fact table · NCCI = analytics on a busy OLTP table
Covering index	Equality column first, range/sort column next, `INCLUDE` the rest
Filtered index	`WHERE column <comparison> constant` (e.g. `Active = 1`, `Status IS NULL`) — a subset of rows; no functions, no `LIKE`
Partition function	`RANGE RIGHT` for dates; n boundaries → n + 1 partitions
Remove old data	`TRUNCATE TABLE … WITH (PARTITIONS (n))`
In-memory	`SCHEMA_ONLY` = no logging, data lost on restart; `HASH` index = point lookups
Temporal / ledger	Temporal = history ("as of") · Ledger = tamper-evidence (cryptographic proof)
JSON	`JSON_VALUE` scalar · `JSON_QUERY` object/array · `OPENJSON` rows · `FOR JSON PATH` builds JSON
JSON index	Computed column + index, or `CREATE JSON INDEX` (clustered PK needed; listed paths only)
Regex	LIKE = match? · COUNT = how many · INSTR = where · SUBSTR = what · REPLACE = change
Fuzzy	`EDIT_DISTANCE` = edits · `EDIT_DISTANCE_SIMILARITY` / `JARO_WINKLER_SIMILARITY` = score
Graph	`MATCH(a-(e1)->b-(e2)->c)`, comma-separated FROM, one alias per edge
Error handling	`IF @@TRANCOUNT > 0 ROLLBACK; THROW;`
SARGable	Never wrap the column; use `col >= @d AND col < DATEADD(day, 1, @d)`
Functions	Inline TVF = parameterised view · MSTVF = poor estimates · scalar UDF needs `RETURN`
Indexed view	`SCHEMABINDING` + `COUNT_BIG(*)` + unique clustered index first
Copilot instructions	`.github/copilot-instructions.md` (team) · personal > repository > organization
MCP	Live metadata · `.vscode/mcp.json` = workspace · `.mcp.json` = Visual Studio
Copilot security	Runs as YOU — your SQL permissions are the limit; never paste PII or secrets
Always Encrypted	Deterministic = lookups/joins · Randomized = most secure · enclaves = richer queries

Topic	The one line
DDM / RLS	DDM = how values look (UNMASK) · RLS = which rows (applies to dbo too)
Permissions	DENY beats GRANT · REVOKE removes an entry · EXECUTE for procedures
Passwordless	Azure app → managed identity · on-premises app → service principal
Network	Private endpoint + deny public access · `0.0.0.0` rule = allow all Azure
Model credential	`IDENTITY = 'Managed Identity'`, `SECRET = '{"resourceid":"https://cognitiveservices.azure.com"}'`
Isolation	REPEATABLE READ = rows read can't change · SERIALIZABLE = + range locks · RCSI = default in Azure SQL DB / Fabric
Blocking	Sleeping + open transaction = forgot to commit · deadlock = victim error 1205
Query Store	QPI finds it · Regressed Queries proves it · `sp_query_store_force_plan` fixes it
SQL projects	`dotnet build` validates → `.dacpac` · Publish deploys · DeployReport previews
System objects	`Microsoft.SqlServer.Dacpacs.Azure.Master` for Azure SQL Database
Workflows	`pull_request` validates before merge · `push` after merge · `workflow_dispatch` = manual
DAB	`/api/{entity}` · `/graphql` · `@env('VAR')` · create = insert, update = change
Change handling	Functions SQL trigger needs Change Tracking (DB + table) · CDC = before/after history · CES = events to Event Hubs
External model	`CREATE EXTERNAL MODEL` · `GRANT EXECUTE ON EXTERNAL MODEL` · ONNX Runtime = local/offline
Embeddings	Embed meaning (descriptions), filter facts (price) · same model for docs and questions · new model = re-embed all
Chunking	Smaller retrieved text, fewer tokens, better matches
Vector search	`VECTOR_DISTANCE` = exact kNN · `VECTOR_SEARCH` + vector index = ANN · metric must match
Metrics	Cosine ignores length · dot = cosine only when normalized · Euclidean uses length
Hybrid	Full-text + vector · weighted formula or RRF `1/(k + rank)`
RAG	Embed question → retrieve → `FOR JSON PATH` → `sp_invoke_external_rest_endpoint` → `JSON_VALUE($.result.choices[0].message.content)`

19.4 Reading exam questions

- Read the **last sentence first**: "What should you insert at line 05?" tells you to look at one line.

- Watch the words that change the answer: **least privilege**, **minimize storage**, **minimize CPU**, **without changing code**, **passwordless**, **on-premises**, **live**, **every time**, **approximate**.
- In code questions, check **argument order** (`DATEDIFF(datepart, start, end)`), **scalar vs object** (`JSON_VALUE` vs `JSON_QUERY`), and **which object you are in** (a function needs `RETURN`; a CTE's outer query can only see the CTE's columns).
- In YAML questions, check **when** a job runs (`on:`, `needs:`, `if:`) and **where** a file goes (`bin/Debug` vs `bin/Release`).
- "Which TWO" questions: each answer is part of the solution — for example, change tracking at the **database** level *and* on the **table**.
- In Yes/No series, judge each solution separately; earlier answers do not carry over.

19.5 Glossary

Term	Meaning in plain English
ACID / transaction	A group of changes that either all happen or none happen
Always Encrypted	Encryption done by the client driver; the database engine stores and normally sees only ciphertext (with secure enclaves, plaintext is processed only inside a protected enclave)
ANN	Approximate nearest neighbour: a fast vector search that uses an index and may miss a few results
AUTD	The *Always-up-to-date* update policy of Azure SQL Managed Instance (gets the newest features)
Azure Arc	Connects a server outside Azure to Azure, so it can be managed like an Azure resource and use a managed identity
Cardinality estimate	The optimizer's guess of how many rows a step returns
CCI / NCCI	Clustered / nonclustered columnstore index
CDC	Change data capture: records the before-and-after values of every change, read from the transaction log
CES	Change event streaming: pushes row changes to Azure Event Hubs (preview)
Chunk	A smaller piece of a long text, embedded separately
Clustered index	The table's rows stored in key order — the index *is* the table
Columnstore	Storage that keeps each column together and compressed; great for scans and aggregates
CTE	Common table expression: a named, temporary result set defined with `WITH`
dacpac	The build output of a SQL database project: a file that describes the whole schema
DAB	Data API builder: turns tables, views, and procedures into REST, GraphQL, and MCP endpoints with a config file
DDM	Dynamic data masking: hides part of a value in query results; stored data is unchanged
DiskANN	The graph-based vector index algorithm used by SQL vector indexes
Delta Parquet	An open, column-based file format used in data lakes such as OneLake
DMV	Dynamic management view: a system view showing what the server is doing right now
DTU / vCore	The two Azure SQL purchasing models: bundled units (DTU) or chosen CPU cores (vCore)
Embedding	A list of numbers (a vector) that represents the meaning of a text
Entra ID	Microsoft's cloud identity service (formerly Azure Active Directory)

Term	Meaning in plain English
ETL	Extract, transform, load: copying data from sources into another store
External model	A database object that tells SQL where an AI model is and how to call it
Fine-tuning	Retraining a model on your data to change its behaviour (not needed for RAG)
Grounding	Giving a model trusted facts to base its answer on
Hallucination	A confident answer from a model that is not true
Hybrid search	Keyword (full-text) search and vector search combined
kNN	k-nearest neighbours: exact vector search that compares the query with every row
Key lookup	An extra trip to the clustered index to fetch columns a nonclustered index doesn't have
Latch	A very short internal lock on a memory page (memory-optimized tables avoid them)
Ledger table	A table whose changes are cryptographically chained so tampering can be detected
LLM / SLM	Large / small language model: an AI model that reads and writes text
Managed identity	An identity Azure gives to an Azure resource, so it can sign in without a password
MCP	Model Context Protocol: a standard way for AI assistants to call tools and read live data
OIDC	OpenID Connect: lets a GitHub workflow sign in to Azure with a short-lived token instead of a stored secret
OLTP	Online transaction processing: many small, fast reads and writes, such as placing orders
Partition	One slice of a table, chosen by the value in the partitioning column
PII	Personally identifiable information, such as names, email addresses, or phone numbers
QPI	Query Performance Insight: the Azure portal view of Query Store data
Query Store	A built-in "flight recorder" that keeps query plans and runtime statistics over time
RAG	Retrieval-augmented generation: find relevant data first, then let the model answer from it
RCSI	Read committed snapshot isolation: readers see the last committed version and don't block writers
RLS	Row-level security: a policy that filters which rows each user can see
RRF	Reciprocal rank fusion: merges ranked lists by adding `1 / (k + rank)` for each list
SARGable	A predicate that lets SQL use an index seek (the column is not wrapped in a function)
SIEM	Security information and event management: a system that collects and analyses security logs
Service principal	An identity for an application registered in Entra ID; used by apps outside Azure

Term	Meaning in plain English
SqlPackage	The command-line tool that publishes, extracts, scripts, and compares `.dacpac` files
TDE	Transparent data encryption: encrypts the database files at rest
Token	A small piece of text (part of a word) that models read and charge by
TVF	Table-valued function: a function that returns a table
Vector	An ordered list of numbers, for example 1,536 floats from an embedding model

19.6 Where to go next

- **Renew** every 12 months with the free renewal assessment.
- Watch Data Exposed for new SQL features — the AI features in this exam change fast.
- Ask questions on Microsoft Q&A.

> **KEY TAKEAWAY**
>
> For every scenario, be able to say in one sentence: *which object stores it, which index finds it, which control protects it, how it gets deployed, and how AI uses it.* That is what 700 points looks like.

Appendix A — One-page decision tables

A.1 Which index or table feature?

Signal in the question	Answer
"Point lookup", "seek", "few rows by key"	Rowstore B-tree index (clustered or nonclustered)
"Avoid key lookups", "covering"	Nonclustered index with `INCLUDE`
"Avoid the sort" for `WHERE a = @x ORDER BY b DESC`	Index key `(a, b DESC)`
"Only rows where Active = 1"	Filtered index
"Large fact table", "batch loaded", "minimize storage", "aggregates"	Clustered columnstore index
"Real-time analytics on an OLTP table"	Nonclustered columnstore index
"Small lookup table", "minimize storage"	Clustered rowstore index
"Filter on a JSON property with a seek"	`JSON_VALUE` computed column + index, or `CREATE JSON INDEX`
"Precompute an aggregate without changing tables"	Indexed view
"Lowest write latency", "data can be lost on restart"	Memory-optimized table, `SCHEMA_ONLY`
"Point lookups on a memory-optimized table"	`HASH` index (range queries → memory-optimized nonclustered)
"Row as it was at a point in time"	Temporal table, `FOR SYSTEM_TIME AS OF`
"Tamper-evident", "cryptographic proof"	Ledger table
"Relationships", "friend of a friend"	Graph tables (`AS NODE`, `AS EDGE`) + `MATCH`
"Query files in storage without loading them"	External table / data virtualization
"Predictable ingestion and retention"	Partitioned table
"Low-latency similarity search on vectors"	Vector index (DiskANN) + `VECTOR_SEARCH`

A.2 Which programmability object?

Signal	Answer
"Runs automatically on insert/update/delete"	DML trigger (`AFTER` or `INSTEAD OF`)
"Runs on CREATE/ALTER/DROP"	DDL trigger
"Changes data", "multiple steps", "transaction"	Stored procedure
"Returns one value", "reused in SELECT"	Scalar function
"Takes parameters and can be joined"	Inline table-valued function
"Saved SELECT without parameters"	View

A.3 Which T-SQL function?

Need	Function
One scalar from JSON	`JSON_VALUE`
An object or array from JSON	`JSON_QUERY`
JSON → rows and columns	`OPENJSON` (in `FROM`)
Rows → JSON	`FOR JSON PATH` (`WITHOUT_ARRAY_WRAPPER` for one object)
Build JSON from values	`JSON_OBJECT`, `JSON_ARRAY`, `JSON_ARRAYAGG`, `JSON_OBJECTAGG`
Is a value in a JSON doc?	`JSON_CONTAINS`
Is the text valid JSON?	`ISJSON`
Change one JSON property	`JSON_MODIFY`
Does text match a pattern?	`REGEXP_LIKE`
Get / count / locate / replace matches	`REGEXP_SUBSTR` / `REGEXP_COUNT` / `REGEXP_INSTR` / `REGEXP_REPLACE`
Matches or pieces as rows	`REGEXP_MATCHES` / `REGEXP_SPLIT_TO_TABLE`
Typos: number of edits	`EDIT_DISTANCE`
Typos: similarity score	`EDIT_DISTANCE_SIMILARITY`, `JARO_WINKLER_SIMILARITY`
First row per group	`ROW_NUMBER() OVER (PARTITION BY … ORDER BY …)` + `WHERE rn = 1`
Running total	`SUM() OVER (PARTITION BY … ORDER BY … ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`
"At least one matching row exists"	`EXISTS` (correlated)

A.4 Which security control?

Signal	Answer
"Encrypted at rest" + "DBAs must not read it" / "CMK in Key Vault"	Always Encrypted
"Point lookups on an encrypted column"	Always Encrypted, deterministic
"No patterns or repetitions visible"	Always Encrypted, randomized
"Range queries / LIKE on encrypted data"	Always Encrypted with secure enclaves
"Encrypt the files and backups"	TDE
"Show only part of the value"	Dynamic data masking
"Only their own tenant's rows"	Row-level security
"Hide one column"	`DENY SELECT` on the column
"Tamper-evident history"	Ledger tables
"Who did what, when"	Auditing
"Passwordless, app in Azure"	Managed identity + Microsoft Entra authentication
"Passwordless, app on-premises"	Service principal
"Traffic must stay private"	Private endpoint + deny public network access
"Allow connections from Azure services"	Firewall rule `0.0.0.0` – `0.0.0.0`
"Call Azure OpenAI from T-SQL without keys"	Database scoped credential `IDENTITY = 'Managed Identity'`

A.5 Which performance tool or fix?

Signal	Answer
"Rows read must not change until commit"	REPEATABLE READ
"No phantoms / no inserts into the range"	SERIALIZABLE
"Consistent view without blocking writers"	SNAPSHOT
"Readers blocked by writers" in a new Azure SQL DB / Fabric DB	They aren't — RCSI is on by default
"Sleeping session, open transaction, others wait"	Uncommitted transaction — commit/rollback or KILL, fix the app
"Find long-running / top CPU queries" in Azure SQL Database	Query Performance Insight (built on Query Store)
"Query has several plans, newer one slower"	Regressed Queries → force the old plan
"Restore previous performance, no code change"	`sp_query_store_force_plan`
"Shape a plan with a hint, no code change"	Query Store hints (`sp_query_store_set_hints`)
"Key Lookup + Sort in the plan"	Composite key + `INCLUDE`
"Index scan because of `CAST(column …)`"	Rewrite as a SARGable half-open range

A.6 Which CI/CD action?

Signal	Answer
"Validate before merge"	`on: pull_request` + `dotnet build`
"Run manually"	`workflow_dispatch`
"Deploy the schema"	`SqlPackage /Action:Publish` or `azure/sql-action` with `action: publish`
"Report what would change"	`/Action:DeployReport`
"Changes since the database was registered"	`/Action:DriftReport`
"Live database → .dacpac"	`/Action:Extract`
"Generate the deployment script for review"	`/Action:Script`
"Unresolved references to system objects"	`Microsoft.SqlServer.Dacpacs.Azure.Master` package (or artifact reference)
"Reference data, safe to rerun"	Post-deployment script with `MERGE` / `IF NOT EXISTS`
"Secrets in the workflow"	`${{ secrets.NAME }}` or OIDC with `azure/login`
"Require specific reviewers for some files"	`CODEOWNERS` + branch protection
"Approval before production deploy"	GitHub environment with required reviewers

A.7 Which change-handling or embedding-maintenance method?

Signal	Answer
"Azure Functions SQL trigger binding"	Change Tracking on database **and** table
"Before and after values", "history of changes"	Change data capture (CDC)
"Stream changes to Event Hubs as events"	Change event streaming (CES)
"Low-code workflow"	Azure Logic Apps
"Refresh embeddings instantly, no batch, no CPU limit"	Table trigger
"Refresh embeddings, minimize database CPU, high volume"	CDC + Azure Functions
"Balance latency and load, process in batches"	Change Tracking + background job

A.8 Which AI building block?

Signal	Answer
"Call a REST endpoint (Azure OpenAI) from T-SQL"	`sp_invoke_external_rest_endpoint`
"Generate embeddings in T-SQL"	`AI_GENERATE_EMBEDDINGS(text USE MODEL m)`
"Split text into pieces in T-SQL"	`AI_GENERATE_CHUNKS`
"No outbound internet"	External model with `API_FORMAT = 'ONNX Runtime'`
"Only one user may use the model"	`GRANT EXECUTE ON EXTERNAL MODEL::m TO user`
"Exact keywords, names, codes"	Full-text search
"Same meaning, different words"	Vector search
"Both"	Hybrid search (weights or RRF)
"Exact results, small table"	kNN with `VECTOR_DISTANCE`
"Fast results, large table"	ANN with vector index + `VECTOR_SEARCH`
"Answers from current company data, with citations, no retraining"	RAG
"Change the model's style or behaviour"	Fine-tuning (not RAG)

Appendix B — Where popular answer keys go wrong

Many people practise with community question banks ("dumps"). For DP-800, the answer key that circulates with the most common one is wrong on **17 of 101** questions when checked against Microsoft Learn. If you memorise those keys, you will learn the wrong rule. This appendix gives the **correct rule** for each one, so the concept sticks instead of the letter. IDs (T1Q1 and so on) match the companion DP-800 practice site.

ID	Topic	The trap in the common key	The correct rule	Read
T1Q1	Filter JSON text by a keyword	`EDIT_DISTANCE (text, @Keyword) < 3`	`EDIT_DISTANCE` compares **whole strings**, so a long paragraph is never "within 3 edits" of one word. With a full-text index, use `CONTAINS(column, @Keyword)`	7.3, 15.2
T1Q16	Patients with a procedure in the last 30 days	`WHERE p.PatientId IN (… correlated…)`	`IN` returns the same rows, but with a correlated subquery the idiomatic predicate is `WHERE EXISTS` (close call)	6.3
T1Q19	Reduce read time for hourly analytics on an OLTP table	Filtered index	Read savings for scans come from **columnstore**. Use a **nonclustered columnstore index** on the rowstore table; keep the filtered index for the "only rows where Active = 1" requirement	2.4
T1Q20	Index for a small lookup table, minimize storage	Nonclustered rowstore index	A **clustered** rowstore index *is* the table — no second copy. A nonclustered index adds a separate structure	2.3
T2Q1	Does a JSON index on `$.location.latitude /longitude/accuracy` serve `$.location.heading`?	Yes	**No.** A JSON index covers only the listed paths and their children; `heading` is a sibling	4.2
T2Q3	Trigger that stamps `LastModifiedUtc`	Join `inserted` on `VehicleId`	Join on the **primary key** (`MaintenanceId`). A non-unique column re-stamps every row of that vehicle	5.5
T2Q11	Update embeddings every time content changes, with no nightly batch	Change tracking	**Table triggers** match "every time" and "no batch process" when there is no CPU constraint. Change tracking needs a separate batch consumer. (Close call — if the stem says *minimize write impact* or *minimize CPU*, pick CT or CDC + Functions instead)	14.6
T2Q18	Fabric API for GraphQL: can a user with *View and Edit* (but not *Run Queries and Mutations*) change field mappings?	No	**Yes.** Editing the API definition needs Edit; *calling* the API needs Execute (Run Queries and Mutations)	10.5
T2Q24	Report whether production has drifted from the model in source control	`SqlPackage /Action:DriftReport`	**DeployReport** (dacpac vs live database). DriftReport compares the database with its own *registration snapshot* and never reads your project	12.6

ID	Topic	The trap in the common key	The correct rule	Read
T2Q30	Fix unresolved system-object references with an artifact reference to the Azure SQL `master.dacpac`	No	**Yes.** It works in SDK-style projects too. The NuGet package `Microsoft.SqlServer.Dacpacs.Azure.Master` is the *recommended* way, but both meet the goal	12.2
T2Q34	Validate every PR before merge	`on: push`	**`on: pull_request`** — push runs only after the merge	12.5
T2Q45	Are SELECTs blocked by an open UPDATE in a new Fabric SQL database?	Yes	**No.** RCSI is ON by default in Azure SQL Database and SQL database in Fabric, so readers read the last committed version and take no shared locks	11.2
T3Q4	Turn relational order rows into JSON	`JSON_QUERY`	**`FOR JSON PATH`** (outer query and nested subquery). `JSON_QUERY` *reads* an object/array from existing JSON	7.1, 16.3
T3Q6	Which column gets the vector index?	`product_name`	The **vector column** (`embedding`). A vector index can only be built on a `vector` column	15.5
T3Q8	Extract the embedding array from a REST response	`AI_GENERATE_CHUNKS`	**`JSON_QUERY(@response, '$.result.data[0].embedding')`** — an array needs `JSON_QUERY`, not `JSON_VALUE`; chunking splits text	16.5
T3Q12	Embed `product_name + category + description` ? Must `price` be in the embedding?	No / Yes	**Yes / No.** Embed the meaningful text; keep `brand` and `price` as normal columns for `WHERE` filters	14.3
T3Q16	RAG retrieval is slow and uses too many tokens; each document already has one embedding	Embeddings	**Chunking.** Whole-document embeddings make every hit pull the entire document into the prompt	14.4

KEY TAKEAWAY

When a key and the documentation disagree, the exam is written from the documentation. Learn the rule in the right-hand column, not the letter.

Appendix C — Exam-dump coverage map

This table maps each of the 101 questions in the DP-800 community question bank (IDs as used by the companion practice site) to the section of this guide that teaches the concept behind it. Use it in both directions: when you miss a practice question, read the section; when you finish a section, try its questions.

Question IDs	Concept tested	Section
T1Q19, T1Q20	Columnstore (CCI vs NCCI), filtered index, clustered vs nonclustered for storage	2.3, 2.4
T1Q15, T2Q4, T2Q16	Covering indexes, `INCLUDE`, key order, removing key lookups and sorts	2.3, 11.3
T3Q23	Memory-optimized `SCHEMA_ONLY` table with a `HASH` index	3.1
T2Q5	`CHECK (ISJSON(…) = 1)` and a unique index on a payload hash	4.1, 4.3
T1Q15, T2Q1	Indexing JSON: computed column + index, `CREATE JSON INDEX` paths	4.2
T1Q14	`CHECK` constraint added `WITH NOCHECK`	4.3
T1Q23, T2Q1	Partition function `RANGE RIGHT`, partitioned tables	4.5
T2Q25, T2Q26, T2Q27, T2Q35	Removing a month: `TRUNCATE … WITH (PARTITIONS)` vs `DELETE` vs invalid DDL	4.6
T2Q23	Indexed views (`SCHEMABINDING`, `COUNT_BIG(*)`, unique clustered index)	5.1
T1Q7, T1Q25, T2Q20	Scalar UDF syntax, `RETURN`, `DATEDIFF` argument order, `GETDATE()` in UDFs, two-part names	5.2
T2Q20, T2Q32	Inline TVF vs multi-statement TVF; parameterised, joinable results	5.3
T1Q10, T2Q3	`DISABLE TRIGGER`; `AFTER UPDATE` trigger joined to `inserted` on the key; recursion	5.5
T1Q18	CTE + JSON + regex in one query	6.1, 7.1, 7.2
T1Q21, T2Q32	`ROW_NUMBER` first-row-per-group; running totals with `OVER`	6.2
T1Q16	Correlated subquery with `EXISTS`	6.3
T1Q2	`TRY…CATCH`, `SCOPE_IDENTITY()`, `@@TRANCOUNT`, `THROW`	6.4
T1Q6, T2Q50	SARGable half-open date ranges	6.5
T1Q1, T1Q18, T1Q21, T3Q4	`JSON_VALUE` vs `JSON_QUERY` vs `OPENJSON`; `FOR JSON PATH`	7.1
T1Q5, T1Q9, T1Q13, T1Q18	`REGEXP_REPLACE`, `REGEXP_SUBSTR`, `REGEXP_LIKE`, LOB input size	7.2
T1Q1	`EDIT_DISTANCE` vs `EDIT_DISTANCE_SIMILARITY`	7.3
T1Q4	Graph `MATCH` with two hops	7.4

Question IDs	Concept tested	Section
T1Q11, T2Q8, T2Q49	Copilot runs as the developer; don't share PII or secrets	8.2
T1Q3, T3Q17, T3Q20	Instruction files, user vs workspace settings, precedence	8.5
T1Q22, T1Q24, T3Q3	MCP servers in VS Code and Visual Studio; live metadata	8.6
T2Q22, T2Q43, T2Q48	Always Encrypted: deterministic vs randomized; vs TDE and DDM	9.1
T2Q2, T2Q44	Dynamic data masking	9.2
T2Q2, T2Q44	Row-level security	9.3
T2Q39	`GRANT` / `DENY` / `REVOKE`, column-level deny	9.4
T2Q9, T2Q21, T2Q33, T2Q43	Managed identity, `CREATE USER … FROM EXTERNAL PROVIDER`, service principal	10.1
T2Q9, T2Q40, T2Q42	Private endpoint; firewall rule `0.0.0.0`	10.2
T2Q6	Database scoped credential with managed identity for Azure OpenAI	10.4
T2Q18	Fabric API for GraphQL permissions	10.5
T1Q26, T2Q45	Isolation levels; RCSI default	11.2
T2Q16	Reading an execution plan	11.3
T2Q14	Blocking-chain DMV query	11.4, 11.6
T2Q15, T2Q38, T2Q46	Query Store, Query Performance Insight, plan forcing, top-CPU query	11.5
T2Q19, T2Q45	Sleeping session with an open transaction	11.6
T2Q12, T2Q28, T2Q29, T2Q30, T2Q36	Build validation; master.dacpac references for system objects	12.2
T2Q41	Idempotent reference data in a post-deployment script	12.3
T2Q10	`git fetch`, `git merge origin/main`, `gh pr create`	12.4
T2Q12, T2Q34, T2Q37	Workflow triggers, `needs` / `if`, `-c Release` output path, OIDC	12.5
T2Q24	DeployReport vs DriftReport	12.6
T2Q17	`azure/sql-action` publish with a repository secret	12.7
T1Q8	`dab init` with `@env()`; Container Apps `secretref:`	13.2, 13.5
T2Q31, T2Q47	DAB roles and actions; REST URL pattern	13.3, 13.4
T2Q40, T2Q42	DAB on Container Apps: ingress and SQL firewall	13.5
T1Q12, T1Q17, T3Q24	Azure Functions SQL trigger and change tracking; enabling CDC	13.7

Question IDs	Concept tested	Section
T2Q13, T3Q25	External models (ONNX local, `GRANT EXECUTE`); switching endpoints	14.2
T3Q1, T3Q12	Which columns to embed	14.3
T3Q16	Chunking	14.4
T3Q7, T3Q19, T3Q25	Generating embeddings; changing the embedding model	14.5
T2Q7, T2Q11, T3Q24	Embedding maintenance methods	14.6
T1Q1	Full-text `CONTAINS`	15.2
T3Q12	Vector data type (float32 default)	15.3
T3Q2, T3Q5, T3Q6, T3Q18	Vector indexes, metrics, `VECTOR_SEARCH` vs `VECTOR_DISTANCE`	15.5, 15.6
T3Q14, T3Q19	Hybrid search: weighted scoring, ANN candidates + `CONTAINSTABLE`	15.7
T3Q9, T3Q21	Azure AI Search hybrid query with semantic ranking	15.9
T3Q11, T3Q13	RAG use cases and retrieval design	16.1
T2Q6, T3Q15	`sp_invoke_external_rest_endpoint` and its credential	16.2
T3Q4, T3Q8	Rows to JSON for the model	16.3
T3Q10	`FOR JSON PATH, WITHOUT_ARRAY_WRAPPER` request body	16.4
T3Q8, T3Q22	Extracting the answer and embeddings from the response	16.5
T1Q1, T2Q1–T2Q5, T3Q1–T3Q3	Case study A (fleet telematics)	17.2
T1Q16, T1Q26, T2Q31–T2Q34, T3Q15, T3Q16	Case study B (healthcare)	17.3

Appendix D — Syntax cheat sheet

The patterns the exam asks you to complete, in one place. Each block is explained in the chapter shown.

▸ Advanced T-SQL (Chapters 6–7)

```
-- First row per group (6.2)
WITH r AS (SELECT *, ROW_NUMBER() OVER (PARTITION BY CustomerId ORDER BY CreatedAt) AS rn
FROM dbo.Tickets)
SELECT * FROM r WHERE rn = 1;

-- Running total (6.2)
SUM(Amount) OVER (PARTITION BY CustomerId ORDER BY TxDate, TxId ROWS BETWEEN UNBOUNDED
PRECEDING AND CURRENT ROW)

-- Correlated existence test (6.3)
WHERE EXISTS (SELECT 1 FROM dbo.Procedures pr WHERE pr.PatientId = p.PatientId AND pr.TxDate
>= DATEADD(DAY, -30, SYSUTCDATETIME()))

-- Error handling (6.4)
BEGIN TRY BEGIN TRANSACTION; /* work */ COMMIT TRANSACTION; END TRY
BEGIN CATCH IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION; THROW; END CATCH;

-- SARGable single day (6.5)
WHERE CreateDate >= @d AND CreateDate < DATEADD(day, 1, @d)

-- JSON (7.1)
JSON_VALUE(doc, '$.customer.email')            -- scalar
JSON_QUERY(doc, '$.items')                     -- object / array
SELECT … FROM t CROSS APPLY OPENJSON(t.doc, '$.items') WITH (product nvarchar(100)
'$.product', qty int '$.qty') AS i;
SELECT … FOR JSON PATH, WITHOUT_ARRAY_WRAPPER;
JSON_OBJECT('id': 1, 'tags': JSON_ARRAY('a', 'b'))
JSON_ARRAYAGG(Name ORDER BY Name)               -- aggregate, with GROUP BY
JSON_CONTAINS(doc, 'sql', '$.tags[*]')
JSON_MODIFY(doc, '$.status', 'closed')

-- Regex (7.2)
REGEXP_LIKE(s, '^\d{5}$')                      -- boolean
REGEXP_REPLACE(email, '\+.*@', '@')            -- remove subaddress
REGEXP_REPLACE(phone, '[^0-9]', '')            -- digits only
REGEXP_SUBSTR(s, '@(.+)$', 1, 1, 'i', 1)       -- capture group 1
REGEXP_COUNT(s, '\d') · REGEXP_INSTR(s, '\d') · REGEXP_MATCHES(s, p) ·
REGEXP_SPLIT_TO_TABLE(s, ',\s*')

-- Fuzzy (7.3) — preview; SQL Server 2025 needs PREVIEW_FEATURES = ON
EDIT_DISTANCE(a, b) · EDIT_DISTANCE_SIMILARITY(a, b) · JARO_WINKLER_DISTANCE(a, b) ·
JARO_WINKLER_SIMILARITY(a, b)

-- Graph (7.4)
FROM Person p1, Knows k1, Person p2, Knows k2, Person p3
WHERE MATCH(p1-(k1)->p2-(k2)->p3) AND p1.PersonID = @id
```

▶ Objects and programmability (Chapters 2–5)

```
CREATE INDEX IX ON dbo.Orders (CustomerId, OrderDate DESC) INCLUDE (Status, TotalAmount);
-- covering
CREATE INDEX IX_Active ON dbo.Orders (CustomerId) WHERE Active = 1;
-- filtered
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCI ON dbo.Orders (OrderDate, CustomerId, Total);
-- OLTP + analytics
CREATE CLUSTERED COLUMNSTORE INDEX CCI ON dbo.FactSales;
-- warehouse

CREATE TABLE dbo.Cache (Id int PRIMARY KEY NONCLUSTERED HASH WITH (BUCKET_COUNT = 1048576),
v int)
WITH (MEMORY_OPTIMIZED = ON, DURABILITY = SCHEMA_ONLY);

… PERIOD FOR SYSTEM_TIME (ValidFrom, ValidTo)) WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE =
dbo.EmpHistory));
… WITH (SYSTEM_VERSIONING = ON (HISTORY_TABLE = HR.SalaryHistory), LEDGER = ON);
-- updatable ledger
… WITH (LEDGER = ON (APPEND_ONLY = ON));
-- append-only ledger

ALTER TABLE t ADD sev AS JSON_VALUE([log], '$.severity');  CREATE INDEX ix ON t(sev) INCLUDE
(…);
CREATE JSON INDEX ji ON t (doc) FOR ('$.location.latitude', '$.location.longitude');
-- needs clustered PK
ALTER TABLE t WITH NOCHECK ADD CONSTRAINT CK CHECK (Owner IS NOT NULL);
ALTER TABLE t ADD CONSTRAINT CK_json CHECK (ISJSON(Payload) = 1);

CREATE PARTITION FUNCTION pf (datetime2) AS RANGE RIGHT FOR VALUES ('2020-01-01', '2021-01-
01');
CREATE PARTITION SCHEME ps AS PARTITION pf ALL TO ([PRIMARY]);
TRUNCATE TABLE dbo.Orders WITH (PARTITIONS (1));

CREATE VIEW dbo.v WITH SCHEMABINDING AS SELECT k, SUM(ISNULL(x,0)) AS s, COUNT_BIG(*) AS c
FROM dbo.t GROUP BY k;
CREATE UNIQUE CLUSTERED INDEX IXv ON dbo.v (k);

CREATE FUNCTION dbo.f (@d datetime2) RETURNS int AS BEGIN RETURN DATEDIFF(day, @d,
GETDATE()); END;
CREATE FUNCTION dbo.tvf (@id int) RETURNS TABLE AS RETURN (SELECT … WHERE CustomerId = @id);
DISABLE TRIGGER ddl_Audit ON DATABASE;
```

▸ Security and performance (Chapters 9–11)

```
… Col char(11) COLLATE Latin1_General_BIN2 ENCRYPTED WITH (ENCRYPTION_TYPE = DETERMINISTIC,
      ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256', COLUMN_ENCRYPTION_KEY = CEK1)
Email varchar(100) MASKED WITH (FUNCTION = 'email()')            -- also default(),
partial(0,"XXX",4), random(1,9)
GRANT UNMASK ON dbo.Customers(Email) TO Lead;

CREATE FUNCTION Security.fn_pred (@TenantId int) RETURNS TABLE WITH SCHEMABINDING
AS RETURN SELECT 1 AS ok WHERE @TenantId = CAST(SESSION_CONTEXT(N'TenantId') AS int);
CREATE SECURITY POLICY Security.p ADD FILTER PREDICATE Security.fn_pred(TenantId) ON
dbo.Tickets,
    ADD BLOCK PREDICATE Security.fn_pred(TenantId) ON dbo.Tickets AFTER INSERT WITH (STATE =
ON);

GRANT SELECT ON OBJECT::Sales.Customer TO AppSupport;  DENY SELECT ON
OBJECT::Sales.Customer(TaxID) TO AppSupport;
CREATE USER [OrderApi-Id] FROM EXTERNAL PROVIDER;  ALTER ROLE db_datareader ADD MEMBER
[OrderApi-Id];

CREATE DATABASE SCOPED CREDENTIAL [https://x.openai.azure.com]
    WITH IDENTITY = 'Managed Identity', SECRET =
'{"resourceid":"https://cognitiveservices.azure.com"}';

SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;
EXEC sp_query_store_force_plan @query_id = 42, @plan_id = 7;
EXEC sys.sp_query_store_set_hints @query_id = 42, @query_hints = N'OPTION (RECOMPILE)';
```

▸ CI/CD and integration (Chapters 12–13)

```
on:
  pull_request: { branches: [ "main" ] }   # validate before merge
  push: { branches: [ "main" ] }           # deploy after merge
  workflow_dispatch:                        # manual run
permissions: { id-token: write, contents: read }
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: dotnet build Database.sqlproj -c Release             # →
bin/Release/Database.dacpac
      - uses: azure/sql-action@v2
        if: github.event_name != 'pull_request'                    # never deploy from a PR
        with: { connection-string: "${{ secrets.SQL_CONNECTION_STRING }}", path:
./bin/Release/Database.dacpac, action: publish }
```

```
dotnet add Database.sqlproj package Microsoft.SqlServer.Dacpacs.Azure.Master
sqlpackage /Action:DeployReport /SourceFile:bin/Release/Db.dacpac
/TargetConnectionString:"…" /OutputPath:report.xml
git fetch origin && git merge origin/main && gh pr create --base main --head feature/x --
title "…" --body "…"
dab init --database-type mssql --connection-string "@env('MSSQL_CONNECTION_STRING')" --host-
mode Production --config dab-config.json
```

```
ALTER DATABASE db SET CHANGE_TRACKING = ON (CHANGE_RETENTION = 2 DAYS, AUTO_CLEANUP = ON);
ALTER TABLE dbo.ToDo ENABLE CHANGE_TRACKING;
EXEC sys.sp_cdc_enable_db;
EXEC sys.sp_cdc_enable_table @source_schema = N'dbo', @source_name = N'Articles', @role_name
= NULL,
     @captured_column_list = N'ArticleId, Title, Body', @supports_net_changes = 1;
```

▸ AI: models, search, and RAG (Chapters 14–16)

```
CREATE EXTERNAL MODEL M WITH (LOCATION = 'https://…/embeddings?api-version=…', API_FORMAT =
'Azure OpenAI',
       MODEL_TYPE = EMBEDDINGS, MODEL = 'text-embedding-3-small', CREDENTIAL = [https://…]);
GRANT EXECUTE ON EXTERNAL MODEL::M TO AppUser;
DECLARE @qv vector(1536) = AI_GENERATE_EMBEDDINGS(@q USE MODEL M);
SELECT c.chunk FROM docs d CROSS APPLY AI_GENERATE_CHUNKS(SOURCE = d.Body, CHUNK_TYPE =
FIXED, CHUNK_SIZE = 2000, OVERLAP = 10) AS c;

CREATE FULLTEXT CATALOG ft AS DEFAULT;  CREATE FULLTEXT INDEX ON dbo.Products (Description)
KEY INDEX PK_Products;
JOIN CONTAINSTABLE(dbo.Products, Description, @keywords) AS k ON p.ProductID = k.[KEY]
-- KEY, RANK 0–1000

CREATE VECTOR INDEX vix ON dbo.Docs (Embedding) WITH (METRIC = 'cosine', TYPE = 'DiskANN');
SELECT TOP (10) Id, VECTOR_DISTANCE('cosine', @qv, Embedding) AS d FROM dbo.Docs ORDER BY d;
-- exact kNN
SELECT TOP (10) WITH APPROXIMATE t.Id, s.distance
-- ANN (current)
FROM VECTOR_SEARCH(TABLE = dbo.Docs AS t, COLUMN = Embedding, SIMILAR_TO = @qv, METRIC =
'cosine') AS s
ORDER BY s.distance;
-- older form: VECTOR_SEARCH(…, METRIC = 'cosine', TOP_N = 10)   (deprecated)
-- RRF: SUM over lists of 1.0 / (60 + position), FULL OUTER JOIN the lists, ORDER BY score
DESC  (15.7)

EXECUTE @rc = sp_invoke_external_rest_endpoint @url = N'https://…/chat/completions?api-
version=…',
       @method = 'POST', @payload = @payload, @credential = [https://…], @response =
@response OUTPUT;
SET @answer = JSON_VALUE(@response, '$.result.choices[0].message.content');
DECLARE @v vector(1536) = CAST(JSON_QUERY(@response, '$.result.data[0].embedding') AS
vector(1536));
```

Appendix E — Feature availability by platform

Status as documented on Microsoft Learn in October 2026. "Preview" features are subject to change; on SQL Server 2025 most of them need `ALTER DATABASE SCOPED CONFIGURATION SET PREVIEW_FEATURES = ON`. For Azure SQL Managed Instance, the newest features need the **SQL Server 2025** or **Always-up-to-date** update policy (shown as "2025/AUTD").

Feature	SQL Server 2025	Azure SQL Database	Azure SQL MI	SQL database in Fabric
`json` data type, `CREATE JSON INDEX`	GA	GA	GA (2025/AUTD)	GA
`JSON_ARRAYAGG`, `JSON_OBJECTAGG`, `JSON_CONTAINS`	Yes	Yes	Yes	Yes
Regular expression functions (`REGEXP_LIKE` and the TVFs need compatibility level 170 on every platform)	Yes	Yes	Yes (2025/AUTD)	Yes
Fuzzy string functions (`EDIT_DISTANCE`, `JARO_WINKLER_*`)	Preview	Preview	Preview (2025/AUTD)	Preview
`vector` type and vector functions	GA	GA	GA (2025/AUTD)	GA
`vector(n, float16)`	Preview	GA	GA	GA
Vector index + `VECTOR_SEARCH`	Preview	GA (latest version: full DML, `TOP (N) WITH APPROXIMATE`)	GA (AUTD); preview (2025 policy)	GA (latest version)
`CREATE EXTERNAL MODEL`, `AI_GENERATE_EMBEDDINGS`	Yes	Yes	Yes (AUTD)	Yes
Local ONNX Runtime model	Preview, Windows only	—	—	—
`AI_GENERATE_CHUNKS` (compatibility level 170 on every platform)	Yes	Yes	Yes (2025/AUTD)	Yes
`sp_invoke_external_rest_endpoint`	Yes — **off** by default	Yes — **on** by default	Yes (2025/AUTD) — **off** by default	Yes — **on** by default

Feature	SQL Server 2025	Azure SQL Database	Azure SQL MI	SQL database in Fabric
Managed identity for database scoped credentials	Yes, when Arc-enabled (`allow server scoped db credentials`)	Yes	Yes	Not documented — Fabric uses `USER IDENTITY` (the caller's Entra account) by default
Change event streaming (CES)	Preview	Preview	Preview	Preview
Change Tracking	Yes	Yes	Yes	Yes
Change data capture	Yes (SQL Agent)	Yes (scheduler; DTU needs S3+)	Yes (SQL Agent)	**No**
Full-text search	Yes	Yes	Yes	Preview
Statistical semantic search (`SEMANTICKEYPHRASETABLE` …)	Yes	No	No	No
In-Memory OLTP (memory-optimized tables)	Yes	Premium / Business Critical (Hyperscale: subset)	Business Critical only	**No**
Ledger tables	Yes	Yes	Yes	**No**
Always Encrypted	Yes	Yes	Yes	**No** (can't create Always Encrypted tables)
Temporal tables	Yes	Yes	Yes	Yes (history table not mirrored)
Graph tables	Yes	Yes	Yes	Yes (not mirrored)
External tables / data virtualization	Yes (PolyBase)	Yes (labelled preview)	Yes	Yes, OneLake only (preview)
Partitioning, `TRUNCATE … WITH (PARTITIONS)`	Yes	Yes (PRIMARY filegroup only)	Yes	Yes (PRIMARY filegroup only)
`ALTER TABLE … SWITCH PARTITION`	Yes	Yes	Yes	**No**
DDL triggers	Database and server	Database only	Database and server	Database only
RCSI on by default	No	**Yes**	No	**Yes**

Feature	SQL Server 2025	Azure SQL Database	Azure SQL MI	SQL database in Fabric
Query Store on by default	Yes	Yes	Yes	Yes
Authentication	Windows, SQL, Microsoft Entra	SQL, Microsoft Entra	Windows, SQL, Microsoft Entra	**Microsoft Entra only**
Copilot	GitHub Copilot in SSMS / VS Code	GitHub Copilot in SSMS / VS Code	GitHub Copilot in SSMS / VS Code	Also Copilot in Fabric (paid F2+ capacity)

KEY TAKEAWAY

If a question is set in **SQL database in Fabric**, rule out memory-optimized tables, ledger tables, Always Encrypted, change data capture, partition switching, SQL logins, and server-scoped DDL triggers. If it is set in **SQL Server 2025**, remember the switches: `PREVIEW_FEATURES` for vector indexes, fuzzy matching, and local ONNX models; `sp_configure 'external rest endpoint enabled'` before calling a REST or Azure OpenAI model (also off by default on Managed Instance); and `sp_configure 'external AI runtimes enabled'` for a local ONNX model.

Appendix F — Every Microsoft Learn link in this guide

Every link below already carries the Student Ambassadors Contributor ID `wt.mc_id=studentamb_657938`. Share any of them directly.

Resource	URL (includes Contributor ID)
Microsoft Certified: SQL AI Developer Associate	https://learn.microsoft.com/credentials/certifications/developing-ai-enabled-database-solutions/?wt.mc_id=studentamb_657938
official DP-800 study guide	https://learn.microsoft.com/credentials/certifications/resources/study-guides/dp-800?wt.mc_id=studentamb_657938
exam duration and experience	https://learn.microsoft.com/credentials/support/exam-duration-exam-experience?wt.mc_id=studentamb_657938
exam scoring and score reports	https://learn.microsoft.com/credentials/certifications/exam-scoring-reports?wt.mc_id=studentamb_657938
retake policy	https://learn.microsoft.com/credentials/support/retake-policy?wt.mc_id=studentamb_657938
renew your certification	https://learn.microsoft.com/credentials/certifications/renew-your-microsoft-certification?wt.mc_id=studentamb_657938
Request accommodations	https://learn.microsoft.com/credentials/certifications/request-accommodations?wt.mc_id=studentamb_657938
Design and implement database objects with SQL	https://learn.microsoft.com/training/modules/design-implement-database-objects/?wt.mc_id=studentamb_657938
Implement programmability objects	https://learn.microsoft.com/training/modules/implement-programmability-objects/?wt.mc_id=studentamb_657938
Write advanced T-SQL code	https://learn.microsoft.com/training/modules/write-advanced-sql-code/?wt.mc_id=studentamb_657938
Implement SQL solutions by using AI-assisted tools	https://learn.microsoft.com/training/modules/design-implement-sql-solutions-ai-assisted-tools/?wt.mc_id=studentamb_657938
Implement data security and compliance	https://learn.microsoft.com/training/modules/implement-data-security-compliance/?wt.mc_id=studentamb_657938
Optimize database performance	https://learn.microsoft.com/training/modules/optimize-database-performance/?wt.mc_id=studentamb_657938
Implement CI/CD by using SQL database projects	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/?wt.mc_id=studentamb_657938
Integrate SQL solutions with Azure services	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/?wt.mc_id=studentamb_657938
Models and embeddings	https://learn.microsoft.com/training/modules/design-implement-models-embeddings-with-sql/?wt.mc_id=studentamb_657938
Intelligent search	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/?wt.mc_id=studentamb_657938
Design and implement RAG with SQL	https://learn.microsoft.com/training/modules/design-implement-rag-with-sql/?wt.mc_id=studentamb_657938
Design and develop database solutions	https://learn.microsoft.com/training/paths/design-develop-database-solutions/?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Secure, optimize, and deploy database solutions	https://learn.microsoft.com/training/paths/secure-optimize-deploy-database-solutions/?wt.mc_id=studentamb_657938
Implement AI capabilities in database solutions	https://learn.microsoft.com/training/paths/implement-ai-capabilities-database-solutions/?wt.mc_id=studentamb_657938
Azure SQL Database free offer	https://learn.microsoft.com/azure/azure-sql/database/free-offer?wt.mc_id=studentamb_657938
Microsoft Fabric trial	https://learn.microsoft.com/fabric/fundamentals/fabric-trial?wt.mc_id=studentamb_657938
SQL database in Fabric	https://learn.microsoft.com/fabric/database/sql/overview?wt.mc_id=studentamb_657938
Understand your SQL platform choices	https://learn.microsoft.com/training/modules/design-implement-database-objects/2-understanding-platform-choices?wt.mc_id=studentamb_657938
What's new in SQL Server 2025	https://learn.microsoft.com/sql/sql-server/what-s-new-in-sql-server-2025?wt.mc_id=studentamb_657938
Limitations in SQL database in Fabric	https://learn.microsoft.com/fabric/database/sql/limitations?wt.mc_id=studentamb_657938
Azure SQL: IaaS vs PaaS	https://learn.microsoft.com/azure/azure-sql/azure-sql-iaas-vs-paas-what-is-overview?wt.mc_id=studentamb_657938
Build effective tables	https://learn.microsoft.com/training/modules/design-implement-database-objects/3-design-implement-tables?wt.mc_id=studentamb_657938
Optimize with indexes	https://learn.microsoft.com/training/modules/design-implement-database-objects/4-design-implement-indexes?wt.mc_id=studentamb_657938
Index architecture and design guide	https://learn.microsoft.com/sql/relational-databases/sql-server-index-design-guide?wt.mc_id=studentamb_657938
Create indexes with included columns	https://learn.microsoft.com/sql/relational-databases/indexes/create-indexes-with-included-columns?wt.mc_id=studentamb_657938
Create filtered indexes	https://learn.microsoft.com/sql/relational-databases/indexes/create-filtered-indexes?wt.mc_id=studentamb_657938
Columnstore indexes overview	https://learn.microsoft.com/sql/relational-databases/indexes/columnstore-indexes-overview?wt.mc_id=studentamb_657938
Columnstore design guidance	https://learn.microsoft.com/sql/relational-databases/indexes/columnstore-indexes-design-guidance?wt.mc_id=studentamb_657938
Real-time operational analytics	https://learn.microsoft.com/sql/relational-databases/indexes/get-started-with-columnstore-for-real-time-operational-analytics?wt.mc_id=studentamb_657938
Use specialized table types	https://learn.microsoft.com/training/modules/design-implement-database-objects/5-specialized-table-types?wt.mc_id=studentamb_657938
Defining durability	https://learn.microsoft.com/sql/relational-databases/in-memory-oltp/defining-durability-for-memory-optimized-objects?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Indexes for memory-optimized tables	https://learn.microsoft.com/sql/relational-databases/in-memory-oltp/indexes-for-memory-optimized-tables?wt.mc_id=studentamb_657938
In-memory technologies in Azure SQL	https://learn.microsoft.com/azure/azure-sql/in-memory-oltp-overview?wt.mc_id=studentamb_657938
Temporal tables	https://learn.microsoft.com/sql/relational-databases/tables/temporal/overview?wt.mc_id=studentamb_657938
Ledger overview	https://learn.microsoft.com/sql/relational-databases/security/ledger/ledger-overview?wt.mc_id=studentamb_657938
Updatable ledger tables	https://learn.microsoft.com/sql/relational-databases/security/ledger/ledger-updatable-ledger-tables?wt.mc_id=studentamb_657938
SQL Graph architecture	https://learn.microsoft.com/sql/relational-databases/graphs/sql-graph-architecture?wt.mc_id=studentamb_657938
Data virtualization in Azure SQL Database	https://learn.microsoft.com/azure/azure-sql/database/data-virtualization-overview?wt.mc_id=studentamb_657938
CREATE EXTERNAL TABLE	https://learn.microsoft.com/sql/t-sql/statements/create-external-table-transact-sql?wt.mc_id=studentamb_657938
Manage JSON columns and indexes	https://learn.microsoft.com/training/modules/design-implement-database-objects/7-design-implement-json?wt.mc_id=studentamb_657938
Enforce data integrity with constraints	https://learn.microsoft.com/training/modules/design-implement-database-objects/6-design-implement-constraints?wt.mc_id=studentamb_657938
Partition tables for scale	https://learn.microsoft.com/training/modules/design-implement-database-objects/8-design-implement-partitioning?wt.mc_id=studentamb_657938
json data type	https://learn.microsoft.com/sql/t-sql/data-types/json-data-type?wt.mc_id=studentamb_657938
CREATE JSON INDEX	https://learn.microsoft.com/sql/t-sql/statements/create-json-index-transact-sql?wt.mc_id=studentamb_657938
Index JSON data	https://learn.microsoft.com/sql/relational-databases/json/index-json-data?wt.mc_id=studentamb_657938
Store JSON documents	https://learn.microsoft.com/sql/relational-databases/json/store-json-documents-in-sql-tables?wt.mc_id=studentamb_657938
Unique and check constraints	https://learn.microsoft.com/sql/relational-databases/tables/unique-constraints-and-check-constraints?wt.mc_id=studentamb_657938
Primary and foreign keys	https://learn.microsoft.com/sql/relational-databases/tables/primary-and-foreign-key-constraints?wt.mc_id=studentamb_657938
ALTER TABLE	https://learn.microsoft.com/sql/t-sql/statements/alter-table-transact-sql?wt.mc_id=studentamb_657938
Sequence numbers	https://learn.microsoft.com/sql/relational-databases/sequence-numbers/sequence-numbers?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
CREATE SEQUENCE	https://learn.microsoft.com/sql/t-sql/statements/create-sequence-transact-sql?wt.mc_id=studentamb_657938
Partitioned tables and indexes	https://learn.microsoft.com/sql/relational-databases/partitions/partitioned-tables-and-indexes?wt.mc_id=studentamb_657938
CREATE PARTITION FUNCTION	https://learn.microsoft.com/sql/t-sql/statements/create-partition-function-transact-sql?wt.mc_id=studentamb_657938
TRUNCATE TABLE	https://learn.microsoft.com/sql/t-sql/statements/truncate-table-transact-sql?wt.mc_id=studentamb_657938
views	https://learn.microsoft.com/training/modules/implement-programmability-objects/2-create-views?wt.mc_id=studentamb_657938
stored procedures	https://learn.microsoft.com/training/modules/implement-programmability-objects/3-create-stored-procedures?wt.mc_id=studentamb_657938
scalar functions	https://learn.microsoft.com/training/modules/implement-programmability-objects/4-create-scalar-functions?wt.mc_id=studentamb_657938
TVFs	https://learn.microsoft.com/training/modules/implement-programmability-objects/5-create-table-valued-functions?wt.mc_id=studentamb_657938
triggers	https://learn.microsoft.com/training/modules/implement-programmability-objects/6-create-triggers?wt.mc_id=studentamb_657938
choosing the right object	https://learn.microsoft.com/training/modules/implement-programmability-objects/7-choosing-when-to-use-each-programmability-object?wt.mc_id=studentamb_657938
CREATE VIEW	https://learn.microsoft.com/sql/t-sql/statements/create-view-transact-sql?wt.mc_id=studentamb_657938
Create indexed views	https://learn.microsoft.com/sql/relational-databases/views/create-indexed-views?wt.mc_id=studentamb_657938
CREATE FUNCTION	https://learn.microsoft.com/sql/t-sql/statements/create-function-transact-sql?wt.mc_id=studentamb_657938
Scalar UDF inlining	https://learn.microsoft.com/sql/relational-databases/user-defined-functions/scalar-udf-inlining?wt.mc_id=studentamb_657938
CREATE PROCEDURE	https://learn.microsoft.com/sql/t-sql/statements/create-procedure-transact-sql?wt.mc_id=studentamb_657938
CREATE TRIGGER	https://learn.microsoft.com/sql/t-sql/statements/create-trigger-transact-sql?wt.mc_id=studentamb_657938
inserted and deleted tables	https://learn.microsoft.com/sql/relational-databases/triggers/use-the-inserted-and-deleted-tables?wt.mc_id=studentamb_657938
DDL triggers	https://learn.microsoft.com/sql/relational-databases/triggers/ddl-triggers?wt.mc_id=studentamb_657938
DISABLE TRIGGER	https://learn.microsoft.com/sql/t-sql/statements/disable-trigger-transact-sql?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
TRIGGER_NESTLEVEL	https://learn.microsoft.com/sql/t-sql/functions/trigger-nestlevel-transact-sql?wt.mc_id=studentamb_657938
CTEs	https://learn.microsoft.com/training/modules/write-advanced-sql-code/2-common-table-expressions?wt.mc_id=studentamb_657938
window functions	https://learn.microsoft.com/training/modules/write-advanced-sql-code/3-window-functions?wt.mc_id=studentamb_657938
correlated subqueries	https://learn.microsoft.com/training/modules/write-advanced-sql-code/8-correlated-queries?wt.mc_id=studentamb_657938
TRY…CATCH	https://learn.microsoft.com/training/modules/write-advanced-sql-code/9-error-handling?wt.mc_id=studentamb_657938
WITH common_table_expression	https://learn.microsoft.com/sql/t-sql/queries/with-common-table-expression-transact-sql?wt.mc_id=studentamb_657938
OVER clause	https://learn.microsoft.com/sql/t-sql/queries/select-over-clause-transact-sql?wt.mc_id=studentamb_657938
ROW_NUMBER	https://learn.microsoft.com/sql/t-sql/functions/row-number-transact-sql?wt.mc_id=studentamb_657938
EXISTS	https://learn.microsoft.com/sql/t-sql/language-elements/exists-transact-sql?wt.mc_id=studentamb_657938
Subqueries	https://learn.microsoft.com/sql/relational-databases/performance/subqueries?wt.mc_id=studentamb_657938
TRY…CATCH	https://learn.microsoft.com/sql/t-sql/language-elements/try-catch-transact-sql?wt.mc_id=studentamb_657938
THROW	https://learn.microsoft.com/sql/t-sql/language-elements/throw-transact-sql?wt.mc_id=studentamb_657938
SET XACT_ABORT	https://learn.microsoft.com/sql/t-sql/statements/set-xact-abort-transact-sql?wt.mc_id=studentamb_657938
XACT_STATE	https://learn.microsoft.com/sql/t-sql/functions/xact-state-transact-sql?wt.mc_id=studentamb_657938
SCOPE_IDENTITY	https://learn.microsoft.com/sql/t-sql/functions/scope-identity-transact-sql?wt.mc_id=studentamb_657938
SARGability in the high-CPU guide	https://learn.microsoft.com/troubleshoot/sql/database-engine/performance/troubleshoot-high-cpu-usage-issues?wt.mc_id=studentamb_657938
Process JSON data	https://learn.microsoft.com/training/modules/write-advanced-sql-code/4-json-functions?wt.mc_id=studentamb_657938
Match patterns with regular expressions	https://learn.microsoft.com/training/modules/write-advanced-sql-code/5-regular-expressions?wt.mc_id=studentamb_657938
Fuzzy string functions	https://learn.microsoft.com/training/modules/write-advanced-sql-code/6-fuzzy-string-matching?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Graph queries	https://learn.microsoft.com/training/modules/write-advanced-sql-code/7-graph-queries?wt.mc_id=studentamb_657938
JSON_VALUE	https://learn.microsoft.com/sql/t-sql/functions/json-value-transact-sql?wt.mc_id=studentamb_657938
JSON_QUERY	https://learn.microsoft.com/sql/t-sql/functions/json-query-transact-sql?wt.mc_id=studentamb_657938
OPENJSON	https://learn.microsoft.com/sql/t-sql/functions/openjson-transact-sql?wt.mc_id=studentamb_657938
JSON_CONTAINS	https://learn.microsoft.com/sql/t-sql/functions/json-contains-transact-sql?wt.mc_id=studentamb_657938
JSON_ARRAYAGG	https://learn.microsoft.com/sql/t-sql/functions/json-arrayagg-transact-sql?wt.mc_id=studentamb_657938
FOR JSON	https://learn.microsoft.com/sql/relational-databases/json/format-query-results-as-json-with-for-json-sql-server?wt.mc_id=studentamb_657938
WITHOUT_ARRAY_WRAPPER	https://learn.microsoft.com/sql/relational-databases/json/remove-square-brackets-from-json-without-array-wrapper-option?wt.mc_id=studentamb_657938
Regular expressions overview	https://learn.microsoft.com/sql/relational-databases/regular-expressions/overview?wt.mc_id=studentamb_657938
REGEXP_LIKE	https://learn.microsoft.com/sql/t-sql/functions/regexp-like-transact-sql?wt.mc_id=studentamb_657938
REGEXP_REPLACE	https://learn.microsoft.com/sql/t-sql/functions/regexp-replace-transact-sql?wt.mc_id=studentamb_657938
REGEXP_SUBSTR	https://learn.microsoft.com/sql/t-sql/functions/regexp-substr-transact-sql?wt.mc_id=studentamb_657938
overview	https://learn.microsoft.com/sql/relational-databases/fuzzy-string-match/overview?wt.mc_id=studentamb_657938
EDIT_DISTANCE	https://learn.microsoft.com/sql/t-sql/functions/edit-distance-transact-sql?wt.mc_id=studentamb_657938
EDIT_DISTANCE_SIMILARITY	https://learn.microsoft.com/sql/t-sql/functions/edit-distance-similarity-transact-sql?wt.mc_id=studentamb_657938
JARO_WINKLER_DISTANCE	https://learn.microsoft.com/sql/t-sql/functions/jaro-winkler-distance-transact-sql?wt.mc_id=studentamb_657938
SQL Graph overview	https://learn.microsoft.com/sql/relational-databases/graphs/sql-graph-overview?wt.mc_id=studentamb_657938
MATCH	https://learn.microsoft.com/sql/t-sql/queries/match-sql-graph?wt.mc_id=studentamb_657938
SHORTEST_PATH	https://learn.microsoft.com/sql/relational-databases/graphs/sql-graph-shortest-path?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
security impact	https://learn.microsoft.com/training/modules/design-implement-sql-solutions-ai-assisted-tools/3-interpret-security-impact?wt.mc_id=studentamb_657938
instruction files	https://learn.microsoft.com/training/modules/design-implement-sql-solutions-ai-assisted-tools/6-create-configure-copilot-instruction-files?wt.mc_id=studentamb_657938
MCP endpoints	https://learn.microsoft.com/training/modules/design-implement-sql-solutions-ai-assisted-tools/7-connect-mcp-server-endpoints?wt.mc_id=studentamb_657938
GitHub Copilot in SSMS	https://learn.microsoft.com/ssms/github-copilot/overview?wt.mc_id=studentamb_657938
Execution context	https://learn.microsoft.com/ssms/github-copilot/execution-context?wt.mc_id=studentamb_657938
Agent mode	https://learn.microsoft.com/ssms/github-copilot/agent-mode?wt.mc_id=studentamb_657938
MCP servers in SSMS	https://learn.microsoft.com/ssms/github-copilot/mcp-servers?wt.mc_id=studentamb_657938
GitHub Copilot with the MSSQL extension	https://learn.microsoft.com/sql/tools/visual-studio-code-extensions/github-copilot/overview?wt.mc_id=studentamb_657938
Agent mode	https://learn.microsoft.com/sql/tools/visual-studio-code-extensions/github-copilot/agent-mode?wt.mc_id=studentamb_657938
Custom instructions	https://code.visualstudio.com/docs/agent-customization/custom-instructions?wt.mc_id=studentamb_657938
MCP servers in VS Code	https://code.visualstudio.com/docs/agent-customization/mcp-servers?wt.mc_id=studentamb_657938
Use MCP servers	https://learn.microsoft.com/visualstudio/ide/mcp-servers?wt.mc_id=studentamb_657938
Enable Copilot in Fabric	https://learn.microsoft.com/fabric/fundamentals/copilot-enable-fabric?wt.mc_id=studentamb_657938
Copilot for SQL database	https://learn.microsoft.com/fabric/database/sql/copilot-sql-database?wt.mc_id=studentamb_657938
Fabric data agent as an MCP server	https://learn.microsoft.com/fabric/data-science/data-agent-mcp-server?wt.mc_id=studentamb_657938
SQL MCP Server	https://learn.microsoft.com/azure/data-api-builder/mcp/overview?wt.mc_id=studentamb_657938
encryption	https://learn.microsoft.com/training/modules/implement-data-security-compliance/2-design-implement-data-encryption?wt.mc_id=studentamb_657938
masking	https://learn.microsoft.com/training/modules/implement-data-security-compliance/3-design-implement-dynamic-data-masking?wt.mc_id=studentamb_657938
RLS	https://learn.microsoft.com/training/modules/implement-data-security-compliance/4-design-implement-row-level-security?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
permissions	https://learn.microsoft.com/training/modules/implement-data-security-compliance/5-design-implement-object-level-permissions?wt.mc_id=studentamb_657938
Always Encrypted	https://learn.microsoft.com/sql/relational-databases/security/encryption/always-encrypted-database-engine?wt.mc_id=studentamb_657938
Always Encrypted cryptography	https://learn.microsoft.com/sql/relational-databases/security/encryption/always-encrypted-cryptography?wt.mc_id=studentamb_657938
Secure enclaves	https://learn.microsoft.com/sql/relational-databases/security/encryption/always-encrypted-enclaves?wt.mc_id=studentamb_657938
TDE in Azure SQL	https://learn.microsoft.com/azure/azure-sql/database/transparent-data-encryption-tde-overview?wt.mc_id=studentamb_657938
Encrypt a column of data	https://learn.microsoft.com/sql/relational-databases/security/encryption/encrypt-a-column-of-data?wt.mc_id=studentamb_657938
Dynamic data masking	https://learn.microsoft.com/sql/relational-databases/security/dynamic-data-masking?wt.mc_id=studentamb_657938
Row-level security	https://learn.microsoft.com/sql/relational-databases/security/row-level-security?wt.mc_id=studentamb_657938
CREATE SECURITY POLICY	https://learn.microsoft.com/sql/t-sql/statements/create-security-policy-transact-sql?wt.mc_id=studentamb_657938
Getting started with permissions	https://learn.microsoft.com/sql/relational-databases/security/authentication-access/getting-started-with-database-engine-permissions?wt.mc_id=studentamb_657938
Configure secure access to AI services	https://learn.microsoft.com/training/modules/implement-data-security-compliance/7-secure-model-endpoints?wt.mc_id=studentamb_657938
Secure data API endpoints	https://learn.microsoft.com/training/modules/implement-data-security-compliance/8-secure-graphql-rest-mcp-endpoints?wt.mc_id=studentamb_657938
Implement auditing	https://learn.microsoft.com/training/modules/implement-data-security-compliance/6-implement-auditing?wt.mc_id=studentamb_657938
Microsoft Entra authentication for Azure SQL	https://learn.microsoft.com/azure/azure-sql/database/authentication-aad-overview?wt.mc_id=studentamb_657938
Configure Entra authentication	https://learn.microsoft.com/azure/azure-sql/database/authentication-aad-configure?wt.mc_id=studentamb_657938
Managed identities	https://learn.microsoft.com/entra/identity/managed-identities-azure-resources/overview?wt.mc_id=studentamb_657938
Connect App Service with a managed identity	https://learn.microsoft.com/azure/app-service/tutorial-connect-msi-sql-database?wt.mc_id=studentamb_657938
Apps hosted on-premises	https://learn.microsoft.com/dotnet/azure/sdk/authentication/on-premises-apps?wt.mc_id=studentamb_657938
Network access controls	https://learn.microsoft.com/azure/azure-sql/database/network-access-controls-overview?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
IP firewall rules	https://learn.microsoft.com/azure/azure-sql/database/firewall-configure?wt.mc_id=studentamb_657938
Private Link	https://learn.microsoft.com/azure/azure-sql/database/private-endpoint-overview?wt.mc_id=studentamb_657938
Azure SQL auditing	https://learn.microsoft.com/azure/azure-sql/database/auditing-overview?wt.mc_id=studentamb_657938
SQL Server Audit	https://learn.microsoft.com/sql/relational-databases/security/auditing/sql-server-audit-database-engine?wt.mc_id=studentamb_657938
sp_invoke_external_rest_endpoint	https://learn.microsoft.com/sql/relational-databases/system-stored-procedures/sp-invoke-external-rest-endpoint-transact-sql?wt.mc_id=studentamb_657938
DAB authorization	https://learn.microsoft.com/azure/data-api-builder/concept/security/authorization-overview?wt.mc_id=studentamb_657938
Fabric API for GraphQL permissions	https://learn.microsoft.com/fabric/data-engineering/connect-apps-api-graphql?wt.mc_id=studentamb_657938
Share Fabric items	https://learn.microsoft.com/fabric/fundamentals/share-items?wt.mc_id=studentamb_657938
configurations	https://learn.microsoft.com/training/modules/optimize-database-performance/02-recommend-database-configurations?wt.mc_id=studentamb_657938
isolation levels	https://learn.microsoft.com/training/modules/optimize-database-performance/03-preserve-data-integrity-isolation-levels?wt.mc_id=studentamb_657938
plans and DMVs	https://learn.microsoft.com/training/modules/optimize-database-performance/04-evaluate-query-performance?wt.mc_id=studentamb_657938
Query Store	https://learn.microsoft.com/training/modules/optimize-database-performance/05-monitor-tune-queries-query-store?wt.mc_id=studentamb_657938
blocking and deadlocks	https://learn.microsoft.com/training/modules/optimize-database-performance/06-identify-resolve-blocking-deadlocks?wt.mc_id=studentamb_657938
vCore service tiers	https://learn.microsoft.com/azure/azure-sql/database/service-tiers-sql-database-vcore?wt.mc_id=studentamb_657938
Automatic tuning	https://learn.microsoft.com/azure/azure-sql/database/automatic-tuning-overview?wt.mc_id=studentamb_657938
SET TRANSACTION ISOLATION LEVEL	https://learn.microsoft.com/sql/t-sql/statements/set-transaction-isolation-level-transact-sql?wt.mc_id=studentamb_657938
Locking and row versioning guide	https://learn.microsoft.com/sql/relational-databases/sql-server-transaction-locking-and-row-versioning-guide?wt.mc_id=studentamb_657938
Optimized locking	https://learn.microsoft.com/sql/relational-databases/performance/optimized-locking?wt.mc_id=studentamb_657938
Monitor with Query Store	https://learn.microsoft.com/sql/relational-databases/performance/monitoring-performance-by-using-the-query-store?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Query Store usage scenarios	https://learn.microsoft.com/sql/relational-databases/performance/query-store-usage-scenarios?wt.mc_id=studentamb_657938
Query Store hints	https://learn.microsoft.com/sql/relational-databases/performance/query-store-hints?wt.mc_id=studentamb_657938
Query Performance Insight	https://learn.microsoft.com/azure/azure-sql/database/query-performance-insight-use?wt.mc_id=studentamb_657938
Diagnose high CPU	https://learn.microsoft.com/azure/azure-sql/database/high-cpu-diagnose-troubleshoot?wt.mc_id=studentamb_657938
Understand and resolve blocking	https://learn.microsoft.com/azure/azure-sql/database/understand-resolve-blocking?wt.mc_id=studentamb_657938
Deadlocks guide	https://learn.microsoft.com/sql/relational-databases/sql-server-deadlocks-guide?wt.mc_id=studentamb_657938
Analyze deadlocks in Azure SQL	https://learn.microsoft.com/azure/azure-sql/database/analyze-prevent-deadlocks?wt.mc_id=studentamb_657938
build and validate	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/02-create-build-validate-sql-database-projects?wt.mc_id=studentamb_657938
source control and reference data	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/03-configure-source-control-manage-reference-data?wt.mc_id=studentamb_657938
branching and PRs	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/04-manage-branching-pull-requests-conflict-resolution?wt.mc_id=studentamb_657938
schema drift	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/05-detect-resolve-schema-drift?wt.mc_id=studentamb_657938
pipelines	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/06-implement-cicd-pipelines?wt.mc_id=studentamb_657938
testing	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/07-design-implement-testing-strategy?wt.mc_id=studentamb_657938
What are SQL database projects?	https://learn.microsoft.com/sql/tools/sql-database-projects/sql-database-projects?wt.mc_id=studentamb_657938
System objects	https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/system-objects?wt.mc_id=studentamb_657938
Package references	https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/package-references?wt.mc_id=studentamb_657938
Pre/post-deployment scripts	https://learn.microsoft.com/sql/tools/sql-database-projects/concepts/pre-post-deployment-scripts?wt.mc_id=studentamb_657938
Automation	https://learn.microsoft.com/sql/tools/sql-database-projects/sql-projects-automation?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Troubleshoot builds	https://learn.microsoft.com/sql/tools/sql-database-projects/howto/troubleshoot-sql-project-build?wt.mc_id=studentamb_657938
Overview	https://learn.microsoft.com/sql/tools/sqlpackage/sqlpackage?wt.mc_id=studentamb_657938
Publish	https://learn.microsoft.com/sql/tools/sqlpackage/sqlpackage-publish?wt.mc_id=studentamb_657938
DeployReport and DriftReport	https://learn.microsoft.com/sql/tools/sqlpackage/sqlpackage-deploy-drift-report?wt.mc_id=studentamb_657938
Extract	https://learn.microsoft.com/sql/tools/sqlpackage/sqlpackage-extract?wt.mc_id=studentamb_657938
Connect to Azure SQL with GitHub Actions	https://learn.microsoft.com/azure/azure-sql/database/connect-github-actions-sql-db?wt.mc_id=studentamb_657938
OpenID Connect from GitHub Actions	https://learn.microsoft.com/azure/developer/github/connect-from-azure-openid-connect?wt.mc_id=studentamb_657938
configuration files	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/2-create-configuration-files-data-api-builder?wt.mc_id=studentamb_657938
entities	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/3-define-entities-rest-graphql?wt.mc_id=studentamb_657938
procedures and views	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/4-expose-database-objects-stored-procedures-views?wt.mc_id=studentamb_657938
deployment	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/5-deploy-data-api-builder-azure-services?wt.mc_id=studentamb_657938
Azure Monitor	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/6-recommend-azure-monitor-configurations?wt.mc_id=studentamb_657938
event-driven patterns	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/7-handle-changes-event-driven-patterns?wt.mc_id=studentamb_657938
Overview	https://learn.microsoft.com/azure/data-api-builder/overview?wt.mc_id=studentamb_657938
Configuration	https://learn.microsoft.com/azure/data-api-builder/configuration/?wt.mc_id=studentamb_657938
Entities	https://learn.microsoft.com/azure/data-api-builder/configuration/entities?wt.mc_id=studentamb_657938
@env function	https://learn.microsoft.com/azure/data-api-builder/concept/config/env-function?wt.mc_id=studentamb_657938
REST	https://learn.microsoft.com/azure/data-api-builder/concept/rest/overview?wt.mc_id=studentamb_657938
GraphQL relationships	https://learn.microsoft.com/azure/data-api-builder/concept/graphql/relationships?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
Deploy to Container Apps	https://learn.microsoft.com/azure/data-api-builder/deployment/azure-container-apps?wt.mc_id=studentamb_657938
Application Insights	https://learn.microsoft.com/azure/data-api-builder/concept/monitor/application-insights?wt.mc_id=studentamb_657938
Container Apps ingress	https://learn.microsoft.com/azure/container-apps/ingress-overview?wt.mc_id=studentamb_657938
Container Apps secrets	https://learn.microsoft.com/azure/container-apps/manage-secrets?wt.mc_id=studentamb_657938
Azure SQL monitoring with Azure Monitor	https://learn.microsoft.com/azure/azure-sql/database/monitoring-sql-database-azure-monitor?wt.mc_id=studentamb_657938
Database watcher	https://learn.microsoft.com/azure/azure-sql/database-watcher-overview?wt.mc_id=studentamb_657938
Azure SQL trigger for Functions	https://learn.microsoft.com/azure/azure-functions/functions-bindings-azure-sql-trigger?wt.mc_id=studentamb_657938
Enable change tracking	https://learn.microsoft.com/sql/relational-databases/track-changes/enable-and-disable-change-tracking-sql-server?wt.mc_id=studentamb_657938
CDC in Azure SQL Database	https://learn.microsoft.com/azure/azure-sql/database/change-data-capture-overview?wt.mc_id=studentamb_657938
sys.sp_cdc_enable_table	https://learn.microsoft.com/sql/relational-databases/system-stored-procedures/sys-sp-cdc-enable-table-transact-sql?wt.mc_id=studentamb_657938
Change event streaming	https://learn.microsoft.com/sql/relational-databases/track-changes/change-event-streaming/overview?wt.mc_id=studentamb_657938
Logic Apps SQL connector	https://learn.microsoft.com/azure/connectors/connectors-create-api-sqlazure?wt.mc_id=studentamb_657938
evaluate models	https://learn.microsoft.com/training/modules/design-implement-models-embeddings-with-sql/02-understand-evaluate-models-sql-database-workloads?wt.mc_id=studentamb_657938
external models	https://learn.microsoft.com/training/modules/design-implement-models-embeddings-with-sql/03-create-manage-external-model-sql?wt.mc_id=studentamb_657938
design embeddings	https://learn.microsoft.com/training/modules/design-implement-models-embeddings-with-sql/04-design-embeddings-sql-database-workloads?wt.mc_id=studentamb_657938
generate and maintain embeddings	https://learn.microsoft.com/training/modules/design-implement-models-embeddings-with-sql/05-generate-maintain-embeddings-sql-database-workloads?wt.mc_id=studentamb_657938
CREATE EXTERNAL MODEL	https://learn.microsoft.com/sql/t-sql/statements/create-external-model-transact-sql?wt.mc_id=studentamb_657938
ALTER EXTERNAL MODEL	https://learn.microsoft.com/sql/t-sql/statements/alter-external-model-transact-sql?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
AI_GENERATE_EMBEDDINGS	https://learn.microsoft.com/sql/t-sql/functions/ai-generate-embeddings-transact-sql?wt.mc_id=studentamb_657938
AI_GENERATE_CHUNKS	https://learn.microsoft.com/sql/t-sql/functions/ai-generate-chunks-transact-sql?wt.mc_id=studentamb_657938
vector data type	https://learn.microsoft.com/sql/t-sql/data-types/vector-data-type?wt.mc_id=studentamb_657938
Foundry models sold by Azure	https://learn.microsoft.com/azure/foundry/foundry-models/concepts/models-sold-directly-by-azure?wt.mc_id=studentamb_657938
Understand embeddings	https://learn.microsoft.com/azure/foundry-classic/openai/concepts/understand-embeddings?wt.mc_id=studentamb_657938
Structured outputs	https://learn.microsoft.com/azure/foundry/openai/how-to/structured-outputs?wt.mc_id=studentamb_657938
RAG chunking phase	https://learn.microsoft.com/azure/architecture/ai-ml/guide/rag/rag-chunking-phase?wt.mc_id=studentamb_657938
RAG generate-embeddings phase	https://learn.microsoft.com/azure/architecture/ai-ml/guide/rag/rag-generate-embeddings?wt.mc_id=studentamb_657938
Chunk large documents	https://learn.microsoft.com/azure/search/vector-search-how-to-chunk-documents?wt.mc_id=studentamb_657938
About change tracking	https://learn.microsoft.com/sql/relational-databases/track-changes/about-change-tracking-sql-server?wt.mc_id=studentamb_657938
About change data capture	https://learn.microsoft.com/sql/relational-databases/track-changes/about-change-data-capture-sql-server?wt.mc_id=studentamb_657938
choose an approach	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/02-choose-intelligent-search-approach?wt.mc_id=studentamb_657938
full-text	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/03-implement-full-text-search?wt.mc_id=studentamb_657938
prepare for vector search	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/04-prepare-sql-vector-search?wt.mc_id=studentamb_657938
vector query patterns	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/05-implement-vector-search-query-patterns?wt.mc_id=studentamb_657938
hybrid search and ranking	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/06-implement-hybrid-search-ranking?wt.mc_id=studentamb_657938
Vector search and vector index	https://learn.microsoft.com/sql/sql-server/ai/vectors?wt.mc_id=studentamb_657938
CREATE VECTOR INDEX	https://learn.microsoft.com/sql/t-sql/statements/create-vector-index-transact-sql?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
VECTOR_SEARCH	https://learn.microsoft.com/sql/t-sql/functions/vector-search-transact-sql?wt.mc_id=studentamb_657938
VECTOR_DISTANCE	https://learn.microsoft.com/sql/t-sql/functions/vector-distance-transact-sql?wt.mc_id=studentamb_657938
VECTOR_NORMALIZE	https://learn.microsoft.com/sql/t-sql/functions/vector-normalize-transact-sql?wt.mc_id=studentamb_657938
VECTORPROPERTY	https://learn.microsoft.com/sql/t-sql/functions/vectorproperty-transact-sql?wt.mc_id=studentamb_657938
Vectors FAQ	https://learn.microsoft.com/sql/sql-server/ai/vectors-frequently-asked-questions?wt.mc_id=studentamb_657938
Full-text search	https://learn.microsoft.com/sql/relational-databases/search/full-text-search?wt.mc_id=studentamb_657938
CREATE FULLTEXT INDEX	https://learn.microsoft.com/sql/t-sql/statements/create-fulltext-index-transact-sql?wt.mc_id=studentamb_657938
CONTAINS	https://learn.microsoft.com/sql/t-sql/queries/contains-transact-sql?wt.mc_id=studentamb_657938
CONTAINSTABLE	https://learn.microsoft.com/sql/relational-databases/system-functions/containstable-transact-sql?wt.mc_id=studentamb_657938
FREETEXTTABLE	https://learn.microsoft.com/sql/relational-databases/system-functions/freetexttable-transact-sql?wt.mc_id=studentamb_657938
Hybrid search	https://learn.microsoft.com/azure/search/hybrid-search-overview?wt.mc_id=studentamb_657938
Create a hybrid query	https://learn.microsoft.com/azure/search/hybrid-search-how-to-query?wt.mc_id=studentamb_657938
RRF scoring	https://learn.microsoft.com/azure/search/hybrid-search-ranking?wt.mc_id=studentamb_657938
Semantic ranking queries	https://learn.microsoft.com/azure/search/semantic-how-to-query-request?wt.mc_id=studentamb_657938
use cases and architecture	https://learn.microsoft.com/training/modules/design-implement-rag-with-sql/2-identify-rag-use-cases-architecture?wt.mc_id=studentamb_657938
prepare retrieval context	https://learn.microsoft.com/training/modules/design-implement-rag-with-sql/3-prepare-retrieval-context-augmentation?wt.mc_id=studentamb_657938
augment prompts	https://learn.microsoft.com/training/modules/design-implement-rag-with-sql/4-augment-prompts-database-context?wt.mc_id=studentamb_657938
generate and process responses	https://learn.microsoft.com/training/modules/design-implement-rag-with-sql/5-generate-process-rag-responses?wt.mc_id=studentamb_657938
JSON_OBJECT	https://learn.microsoft.com/sql/t-sql/functions/json-object-transact-sql?wt.mc_id=studentamb_657938

Resource	URL (includes Contributor ID)
RAG in Microsoft Foundry	https://learn.microsoft.com/azure/foundry/concepts/retrieval-augmented-generation?wt.mc_id=studentamb_657938
Intelligent applications with Azure SQL	https://learn.microsoft.com/azure/azure-sql/database/ai-artificial-intelligence-intelligent-applications?wt.mc_id=studentamb_657938
RAG design and evaluation guide	https://learn.microsoft.com/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide?wt.mc_id=studentamb_657938
Create and maintain database objects	https://learn.microsoft.com/training/modules/design-implement-database-objects/9-exercise-create-database-objects?wt.mc_id=studentamb_657938
Implement programmability objects in SQL Server	https://learn.microsoft.com/training/modules/implement-programmability-objects/8-exercise-implement-programmability-objects?wt.mc_id=studentamb_657938
Work with JSON functions	https://learn.microsoft.com/training/modules/write-advanced-sql-code/10-exercise-write-advanced-sql-code?wt.mc_id=studentamb_657938
Configure AI-assisted tools for database development	https://learn.microsoft.com/training/modules/design-implement-sql-solutions-ai-assisted-tools/8-exercise-design-implement-sql-solutions-ai-assisted-tools?wt.mc_id=studentamb_657938
Implement security features	https://learn.microsoft.com/training/modules/implement-data-security-compliance/9-exercise-implement-security-features?wt.mc_id=studentamb_657938
Optimize query performance	https://learn.microsoft.com/training/modules/optimize-database-performance/07-exercise-optimize-query-performance?wt.mc_id=studentamb_657938
Implement CI/CD by using SQL database projects	https://learn.microsoft.com/training/modules/implement-cicd-sql-database-projects/08-exercise-implement-cicd-sql-database-projects?wt.mc_id=studentamb_657938
Configure data API builder for a product catalog	https://learn.microsoft.com/training/modules/integrate-sql-solutions-azure-services/8-exercise-data-api-builder?wt.mc_id=studentamb_657938
Generate and update embeddings in Azure SQL Database	https://learn.microsoft.com/training/modules/design-implement-models-embeddings-with-sql/06-exercise-generate-update-embeddings-sql?wt.mc_id=studentamb_657938
Implement intelligent search	https://learn.microsoft.com/training/modules/design-implement-intelligent-search-with-sql/07-exercise-implement-intelligent-search?wt.mc_id=studentamb_657938
Implement a RAG solution	https://learn.microsoft.com/training/modules/design-implement-rag-with-sql/6-exercise-implement-rag-solution?wt.mc_id=studentamb_657938
Data Exposed	https://learn.microsoft.com/shows/data-exposed/?wt.mc_id=studentamb_657938
Microsoft Q&A	https://learn.microsoft.com/answers/products/?wt.mc_id=studentamb_657938